

Стюарт Рассел, Питер Норвиг

Искусственный интеллект: современный подход, 2-е изд.

Часть IV. Планирование

Содержание

Часть IV. Планирование	1
Глава 11. Основы планирования	1
Глава 12. Планирование и осуществление действий в реальном мире	56

Часть IV. Планирование

Основы планирования 512 Планирование и осуществление действий в реальном мире 564

Глава 11. Основы планирования

11 Основы В данной главе показано, каким образом агент может воспользоваться информацией о структуре задачи для создания сложных планов действий.

Планированием называется процесс выработки последовательности действий, позволяющих достичь цели. До сих пор в этой книге рассматривались два примера планирующих агентов: описанный в главе 3 агент, осуществляющий решение задач на основе поиска, и логический планирующий агент, который представлен в главе 10. Данная глава в основном посвящена описанию вопросов расширения области применения агентов и ее распространения на сложные задачи планирования, которые нельзя решить с помощью подходов, рассматривавшихся до сих пор.

В разделе 11.1 разрабатывается выразительный, но тщательно регламентированный язык для представления задач планирования, включая действия и состояния.

Этот язык тесно связан с представлениями действий в пропозициональной логике и логике первого порядка, которые рассматривались в главах 7 и 10. В разделе 11.2 описано, как можно использовать эти представления в алгоритмах прямого и обратного поиска в основном с применением точных эвристик, которые могут создаваться автоматически исходя из структуры представления. (Используемый при этом способ аналогичен способу формирования эффективных эвристик для задач удовлетворения ограничений в главе 5.) В разделах 11.3-11.5 описаны алгоритмы планирования, которые выходят за рамки прямого и обратного поиска и позволяют воспользоваться имеющимся представлением задачи. В частности, представлены подходы, позволяющие не ограничиваться рассмотрением только полностью упорядоченных последовательностей действий.

В данной главе рассматриваются лишь такие варианты среды, которые являются полностью наблюдаемыми, детерминированными, конечными, статическими (изменения происходят только в результате действий агента) и дискретными (сточки зрения времени, действий, объектов и результатов). Такая среда называется средой [^] классического планирования. В отличие от этого, неклассическое планирование предназначено для частично наблюдаемых или стохастических вариантов среды и для него требуется другой набор алгоритмов и проектов агентов, описанный в главах 12 и 17.

ILL Задача планирования Рассмотрим, что может произойти, когда обычный агент, решающий задачи с помощью стандартных алгоритмов поиска (поиска в глубину, поиска A^* и т.д.), сталкивается с крупными задачами реального мира. Это позволит нам научиться разрабатывать лучших планирующих агентов.

Наиболее очевидная сложность состоит в том, что агент, решающий задачи, может быть просто подавлен огромным количеством действий, не относящихся к делу.

Рассмотрим задачу покупки одного экземпляра англоязычного издания настоящей книги с названием AI: A Modern Approach в электронном книжном магазине.

Предположим, что агент-покупатель должен совершить одно действие, связанное с покупкой, в расчете на каждый возможный десятицифровой номер ISBN, что приводит к общему количеству действий, равному 10 миллиардам. В ходе применения алгоритма поиска агент должен исследовать состояния результатов всех 10 миллиардов действий, чтобы определить, какое из них соответствует цели, заключающейся в том, чтобы приобрести экземпляр книги с номером ISBN 0137903952. С другой стороны, разумный планирующий агент должен быть способным проработать процедуру покупки в обратном направлении, от явного описания цели, такого как $\text{Have}(\text{ISBN}0137903952)$, и непосредственно сформировать действие $\text{Buy}(\text{ISBN}0137903952)$. Для этого агенту требуется иметь общие знания о том, что действие $\text{Buy}(x)$ приводит к результату $\text{Have}(x)$. При наличии этих знаний и цели планировщик может определить в единственном шаге унификации, что правильным действием является $\text{Buy}(\text{ISBN}0137903952)$.

Еще одна сложность заключается в определении хорошей эвристической функции.

Предположим, что цель агента состоит в том, чтобы купить четыре разных книги в оперативном режиме. Количество планов только для четырех этапов покупки будет составлять 1040, поэтому поиск без точной эвристики даже нет смысла рассматривать. Для человека очевидно, что хорошей эвристической оценкой для стоимости состояния является количество книг, которые еще предстоит купить; к сожалению, эта идея не столь очевидна для агента, решающего задачи, поскольку он рассматривает процедуру проверки цели как "черный ящик", который возвращает истину или ложь в ответ на каждое состояние. Поэтому агент,

решающий задачи, не обладает автономностью; он требует, чтобы человек предоставлял ему эвристическую функцию для каждой новой задачи. С другой стороны, если планирующий агент имеет доступ к явному представлению цели как конъюнкции подцелей, то может использовать единственную эвристику, независимую от проблемной области, — количество невыполненных конъюнктов. Для задачи покупки книг цель будет представлять собой выражение Have (A) л Have (B) л Have {C} л Have (D), а состояние, содержащее выражение Have (A) л Have (C), будет иметь стоимость 2. Таким образом, агент автоматически получает правильную эвристику для этой задачи и для многих других.

Ниже в этой главе будет показано, как формировать более сложные эвристики, в которых учитываются не только структура цели, но и возможные действия.

Наконец, агент-решатель задач может оказаться неэффективным из-за того, что не способен воспользоваться $\wedge$ декомпозицией задачи. Рассмотрим задачу доставки множества пакетов ночной почты по соответствующим адресатам, которые разбросаны по всей Австралии. В данном случае имеет смысл найти аэропорт, ближайший к каждому адресату, и разделить общую задачу на несколько подзадач, по одной на

каждый аэропорт. А в самом множестве пакетов, доставляемых через каждый конкретный аэропорт, можно определить в зависимости от города адресата допустимость дальнейшей декомпозиции. Как было описано в главе 5, способность выполнять декомпозицию такого рода обеспечивает повышение эффективности решателей задач удовлетворения ограничений. Такое же утверждение остается справедливым для планировщиков: в наихудшем случае может потребоваться время $O(p!)$ для поиска наилучшего плана доставки p пакетов, но если существует возможность выполнить декомпозицию этой задачи на k равных частей, затраты времени будут составлять только $O((p/k)! \cdot k)$.

Как было отмечено в главе 5, идеально декомпозируемые задачи являются очень привлекательными, но встречаются редко¹. Проекты многих систем планирования (особенно планировщиков с частичным упорядочением, описанных в разделе 11.3) основаны на предположении, что большинство задач реального мира являются $\wedge$ Эк частично декомпозируемыми. Это означает, что планировщик может работать над подцелями независимо, но, скорее всего, ему потребуется также выполнить определенную дополнительную работу по объединению результирующих субпланов.

Применительно к некоторым задачам такое предположение не оправдывается, поскольку велика вероятность того, что работа над одной подцелью будет приводить к разрушению результатов работы над другой подцелью. Именно из-за

такого взаимодействия подцелей и являются трудноразрешимыми многие головоломки (подобные задаче игры в восемь).

Язык задач планирования Как следует из приведенного выше описания, сам способ представления задач планирования (состояний, действий и целей) должен обеспечивать возможность для алгоритмов планирования воспользоваться логической структурой задачи. Весь секрет состоит в том, чтобы найти язык, достаточно выразительный для описания широкого круга задач, но вместе с тем достаточно ограничительный для обеспечения функционирования на его основе эффективных алгоритмов. В этом разделе вначале рассматривается основной язык представления классических планировщиков, известный под названием ² Strips. Затем кратко представлены некоторые из многих возможных вариантов языков, подобных Strips.

- **Представление состояний.** В планировщиках применяется декомпозиция мира на логические условия, а состояние представляется в виде конъюнкции положительных литералов. Мы будем рассматривать пропозициональные литералы; например, выражение $\text{Poor} \wedge \text{Unknown}$ может представлять состояние агента неудачника (бедного и безвестного). В этой главе будут также использоваться литералы первого порядка; например, выражение $\text{At}(\text{Plane1}, \text{Melbourne}) \wedge \text{At}(\text{Plane2}, \text{Sydney})$ может представлять одно из состояний в задаче доставки ¹ Следует отметить, что даже задача доставки пакетов не является идеально декомпозируемой.

Могут возникать такие случаи, в которых лучше назначить пакеты для доставки в более отдаленный аэропорт, если это позволит исключить необходимость еще одного полета в более близкий аэропорт. Тем не менее большинство компаний, занимающихся доставкой почты, предпочитают решения, характеризующиеся вычислительной и организационной простотой, таким решениям, которые продиктованы требованиями декомпозиции.

² Strips — сокращение от STanford Research Institute Problem Solver.

пакетов. Литералы в описаниях состояния первого порядка должны быть базовыми и не содержащими функцш. Такие литералы, как $\text{At}(x, y)$ или $\text{At}(\text{Father}(\text{Fred}), \text{Sydney})$, не допускаются. Кроме того, используется предположение о замкнутом мире, которое означает, что любые условия, не упомянутые в описании состояния, считаются ложными.

- **Представление целей.** Цель — это частично заданное состояние, представленное в виде конъюнкции положительных базовых литералов, таких как $\text{Rich} \wedge \text{Famous}$ или $\text{At}(\text{P2}, \text{Tahiti})$. Пропозициональное состояние s удовлетворяет цели d , если s содержит все атомы из d (и, возможно, другие атомы). Например, состояние $\text{Rich} \wedge \text{Famous} \wedge \text{Miserable}$ (богатый, знаменитый и несчастный) удовлетворяет цели $\text{Rich} \wedge \text{Famous}$.
- **Представление действий.** Любое действие задается в терминах предусловий, которые должны соблюдаться до того, как оно может быть выполнено, и результатов, которые становятся следствием его

выполнения. Например, действие, состоящее в перелете самолета из одного места в другое, можно записать следующим образом:

Action{Fly(p, from, to), Precond: At(p, from) $\wedge$ Plane(p) $\wedge$ Airport(from) $\wedge$ Airport(to), Effect: $\neg$ At{p, from) $\wedge$ At{p, to)) где Precond обозначает предусловие, а Effect — результат.

Такое выражение следовало бы называть не просто действием, а $\wedge$ схемой действия, в том смысле, что в нем представлен целый ряд различных действий, которые могут явиться следствием конкретизации переменных p, from и to различными константами. Вообще говоря, любая схема действия состоит из перечисленных ниже трех частей. • Имя действия и список параметров (например, Fly(p, from, to)) служат для обозначения действия. • $\wedge$ Предусловием называется конъюнкция не содержащих функций положительных литералов, регламентирующая то, что должно быть истинным в некотором состоянии, прежде чем можно будет выполнить рассматриваемое действие. Любые переменные, заданные в предусловии, должны также присутствовать в списке параметров действия. • $\wedge$ Результатом является конъюнкция не содержащих функций литералов, представляющая собой описание того, как изменится состояние после выполнения действия. В состоянии, ставшем результатом действия, подтверждается истинность положительного литерала P в результате, тогда как применительно к отрицательному литералу $\neg P$ подтверждается его ложность. Переменные, присутствующие в результате, должны также присутствовать в списке параметров действия.

Для повышения удобства чтения в некоторых системах планирования предусмотрено разделение результата на $\wedge$ список добавления для положительных литералов и $\wedge$ список удаления для отрицательных литералов.

Определив синтаксис представлений задач планирования, можно приступить к определению семантики. Наиболее простой способ выполнения этой задачи состоит в том, чтобы описать, как действия влияют на состояния. (Альтернативный метод предусматривает определение непосредственного преобразования в аксиомы состояния-преемника, семантика которых основана на логике первого порядка; см. упр. 11.3.) Вначале необходимо отметить, что действие $\wedge$ применимо в любом состоянии, в котором выполняется предусловие; в противном случае действие не имеет эффекта. Для схемы действий первого порядка задача определения применимости связана с поиском подстановки G для переменных в предусловии. Например, предположим, что текущее состояние описано следующим образом:

At(Plf JFK) $\wedge$ At(P2, SFO) $\wedge$ Plane (Px) $\wedge$ Plane (P2) $\wedge$ Airport{JFK) $\wedge$ Airport(SFO) Это состояние удовлетворяет такому предусловию:

$At\{p, from\} \wedge Plane(p) \wedge Airport(from) \wedge Airport\{to\}$ с подстановкой $\{p/P1f\ from/JFK, to/SFO\}$ (кроме всего прочего; см. упр. 11.2).

Поэтому конкретное действие $Fly(P1, JFK, SFO)$ является применимым.

Начиная с состояния s , $\wedge$ результатом выполнения применимого действия a является состояние s' , представляющее собой то же самое, что и s , за исключением того, что любой положительный литерал P в результате a добавляется к s' , а любой отрицательный литерал $\neg P$ удаляется из s' . Таким образом, после выполнения действия $Fly(P\pm, JFK, SFO)$ показанное выше текущее состояние принимает вид $At(P1f\ SFO) \wedge At(P2i\ SFO) \wedge Plane(P1) \wedge Plane(P2) \wedge Airport(JFK) \wedge Airport(SFO)$. Следует отметить, что если какой-то положительный результат уже находится в состоянии s , то не добавляется повторно, а если какой-то отрицательный результат отсутствует в состоянии s , эта часть результата игнорируется. Такое определение воплощает в себе так называемое $\wedge$ предположение Strips, что каждый литерал, не упомянутый в результате, остается неизменным. Благодаря этому язык Strips позволяет избежать возникновения проблемы представительного окружения, описанной в главе 10.

Наконец, можно определить $\wedge$ решение для задачи планирования. В своей простейшей форме это всего лишь последовательность действий, которая, будучи выполненной в начальном состоянии, приводит к состоянию, удовлетворяющему цели. Далее в этой главе будет показано, как обеспечить возможность использовать решение, представляющее собой частично упорядоченные множества действий, при условии, что каждая последовательность действий, которая отвечает этому частичному упорядочению, является решением.

Выразительность и расширения языка Всевозможные ограничения, налагаемые в представлении Strips, были выбраны в надежде на то, что алгоритмы планирования станут более простыми и эффективными и вместе с тем при описании реальных задач не будут возникать слишком значительные трудности. Одно из самых важных ограничений состоит в том, что литералы должны быть свободными от функций. Благодаря наличию такого ограничения можно быть уверенным в том, что каждую схему действий для каждой конкретной задачи удастся пропозиционализировать, т.е. преобразовать в конечную коллекцию чисто пропозициональных представлений действий без переменных (дополнительную информа-

цию по этой теме см. в главе 9). Например, в проблемной области грузовых авиационных перевозок для задачи с десятью самолетами и пятью аэропортами можно преобразовать схему $Fly\{p, from, to\}$ в $10 \times 5 \times 5 = 250$ чисто пропозициональных действий. Планировщики, описанные в разделах 11.4 и 11.5, работают непосредственно с пропозиционализированными описаниями. Если же будут разрешены функциональные символы, то количество состояний и действий, которые могут быть сконструированы, станет бесконечно большим.

В последние годы стало очевидно, что язык Strips является недостаточно выразительным для некоторых реальных проблемных областей. В результате этого было разработано много вариантов данного языка. В табл. 11.1 кратко описан наиболее важный из них, язык $\wedge$ ADL (Action Description Language — язык описания действий), на основе его сравнения с базовым языком Strips. В языке ADL действие Fly можно записать следующим образом:

Action(Fly(p:Plane, from:Airport, to:Airport), Precond: At{p, from) $\wedge$ {from Φ to), Effect: -. At(p, from) $\wedge$ At{p, to))

Таблица 11.1. Сравнение языков представления задач планирования Strips и ADL.

Язык Strips	Язык ADL
Только положительные литералы	Положительные и отрицательные литералы
в состояниях: Poor л Unknown $\wedge$ Rich л $\wedge$ Famous	Предположение о замкнутом мире: неупомянутые литералы являются неизвестными
Предположение об открытом мире: неупомянутые литералы являются ложными	Результат р л -i(p означает добавление Р и -i(p и удаление бавление Р и удаление О -,р и Q
Только базовые литералы в цеКвалифицированные переменные в целях: цель лях: Ri ch л Famous 3x A t (Pi, x) $\wedge$ At (P2, x)	состоит в том, чтобы Pi и P2 находились в одном и том же месте
Цели представляют собой конъюнкций и д изъюнкции: Rich $\wedge$ Famous юнкций:-iPoor л {Famous v Smart)	Результаты представляют собой Разрешены условные результаты: when P: E означает, что E конъюнкции является результатом, только если выполнено условие Р
Отсутствует поддержка отношеПредикат равенства (x=y) является встроенным ния равенства	Отсутствует поддержка типов
Переменные могут иметь типы, как, например, в выражении (p:Plane)	Обозначение р: Plane в списке параметров представляет собой сокращение для Plane (p) в предусловии; применение такого обозначения не приводит к повышению выразительной мощи, но позволяет повысить удобство для чтения.

(Применение этого обозначения приводит также к сокращению количества возможных пропозициональных действий, которые могут быть сконструированы.) Предусловие (from&to) выражает тот факт, что полет не может быть совершен из некоторого аэропорта в сам этот аэропорт. Такое условие не может быть выражено кратко в языке Strips.

Различные формальные средства планирования, применяемые в искусственном интеллекте, были систематизированы в рамках стандартного синтаксиса, получившего название PDDL (Planning Domain Definition Language — язык определения проблемной области планирования). Этот язык позволяет исследователям обмениваться эталонными тестовыми задачами и сравнивать полученные результаты. Язык PDDL включает подязыки для Strips, ADL и

иерархических сетей задач (Hierarchical Task Network — HTN), которые будут рассматриваться в главе 12.

Системы обозначений Strips и ADL являются вполне приемлемыми для многих реальных проблемных областей. В приведенных ниже подразделах описаны некоторые простые примеры. Тем не менее эти языки все еще характеризуются некоторыми существенными ограничениями. Наиболее очевидным из них является то, что эти языки не позволяют представить естественным образом последствия действий.

Например, если в самолете есть люди, пакеты или частички пыли, то все они в процессе полета самолета также меняют свое местонахождение. Можно было бы представить эти изменения как прямой результат полета, но кажется более естественным представить изменение местонахождения содержимого самолета как логическое следствие изменения местонахождения самого самолета. Дополнительные примеры подобных ^ ограничений состояния приводятся в разделе 11.5. Кроме того, в классических системах планирования не предпринимается даже попытка решить проблему спецификации: проблему не представленных в определении задачи обстоятельств, которые могут вызвать неудачное завершение действия. Способы решения проблемы спецификации описаны в главе 12.

Пример: воздушный грузовой транспорт В листинге 11.1 показана одна из задач организации перевозок с помощью воздушного грузового транспорта, в которой предусматривается загрузка и разгрузка грузов в самолеты и из самолетов, а также перелет из одного места в другое. Эта задача может быть определена с помощью трех действий: Load, Unload и Fly Действия влияют на значения двух предикатов: предикат $1p(c, p)$ означает, что груз c находится в самолете p , а предикат $At(x, a)$ означает, что объект x (либо самолет, либо груз) находится в аэропорту a . Следует отметить, что предикат At больше вообще не обуславливает местонахождение груза, если предикат In определяет, что груз находится в самолете, поэтому At фактически означает "доступен для использования в указанном местонахождении". Для того чтобы научиться прорабатывать все эти детали должным образом, необходимо приобрести определенный опыт составления определений действий. Ниже приведен план, который является решением данной задачи. [Load {d, P±, SFO), Fly(Plt SFO, JFK), Unload { d , Plt JFK) , Load {C2, P2, JFK) , Fly{P2 , JFK, SFO) , Unload {C2, P2 , SFO)] Листинг 11.1. Задача Strips, в которой предусматривается транспортировка груза между аэропортами Init{At{Clт SFO) а At{C2, JFK) а At{Plт SFO) л At{P2, JFK) а Cargo (Ci) л Cargo (C2) л Plane (Pi) л Plane {P2) A Airport{JFK) а Airport{SFO)) Goal{At{Clт JFK) A At{C2, SFO)) Action{Load{c, p, a),

Precond: At {c, a) л At(p, a) л Cargo (c) л Plane (p) л Airport {a) Effect: -I At {c, a) л 1п(c, p)) Act ion{Unload{c, p, a), Precond: In {c, p) л At{p, a) л Cargo {c) л Plane (p) л Airport {a)

Effect: At(c, a) л -i Tп(c, p)) Action (Fly (p, from, to) , Precond: At{p, from) л Plane (p) л Airport {from) л Airport (to) Effect: -i At(p, fro/n) л At(p, to)) В применяемом здесь представлении используется чистый (т.е. не дополненный) язык Strips. В частности, это представление не исключает того, что самолет может вылететь и прилететь в один и тот же аэропорт. Возникновение такой ситуации могут предотвратить литералы неравенства, предусмотренные в языке ADL.

Пример: задача с запасным колесом Рассмотрим задачу смены колеса со стертой покрышкой. Точнее, цель состоит в том, чтобы на оси автомобиля было правильно смонтировано исправное запасное колесо, тогда как в начальном состоянии на оси имеется колесо со стертой покрышкой, а в багажнике находится исправное запасное колесо. Для того чтобы упростить эту задачу, рассмотрим чрезвычайно абстрактную ее версию, в которой не учитываются крепко прихваченные крепежные гайки или другие усложнения. Существуют только четыре действия: выемка запасного колеса из багажника, снятие колеса со стертой покрышкой с оси, установка запасного колеса на ось и оставление автомобиля без присмотра на ночь. Предполагается, что автомобиль находится в районе с исключительно неблагоприятной криминогенной обстановкой, поэтому результатом его пребывания ночью на улице становится исчезновение колес.

Описание ADL этой задачи приведено в листинге 11.2. Обратите внимание на то, что оно является исключительно пропозициональным. Это описание превосходит по своим возможностям описание на языке Strips в том, что в его предусловии для действия Put On {Spare, Axle) (поместить запасное колесо на ось) используется отрицаемый предикат -At (Flat, Axle) (отрицание предиката At (Flat, Axle) — на оси находится колесо со стертой покрышкой). Необходимости в этом можно избежать, применяя вместо него предикат Clear {Axle) (отсутствие колеса на оси), как будет показано в следующем примере.

Листинг 11.2. Простая задача с запасным колесом Init {At {Flat, Axle) а At {Spare, Trunk)) Goal{At{Spare,Axle)) Action{Remove{Spare,Trunk), Precond: At{Spare,Trunk) Effect: — At{Spare,Trunk) а At{Spare,Ground)) Action{Remove{Flat, Axle) , Precond: At{Flat,Axle) Effect: -iAt {Flat, Axle) а At {Flat, Ground)) Action{PutOn{Spare, Axle), Precond: At{Spare,Ground) а —At{Flat,Axle)

Effect: -At (Spare, Ground) л At {Spare, Axle)) Action (LeaveOvernight, Precond:

Effect: -At {Spare, Ground) л -At (Spare, Axle) л -At (Spare, Trunk) а -At (Flat, Ground) л - ^At (Flat, Axle)) Пример: мир блоков Одна из наиболее широко известных проблемных областей планирования известна под названием ^ мира блоков. Эта проблемная область состоит из множества блоков кубической формы, находящихся на столе³. Блоки можно укладывать в столбик, но на верхней поверхности одного блока может быть непосредственно размещен только еще один блок. Робот может

брать манипулятором блок и перемещать его в другую позицию — либо на стол, либо на верхнюю поверхность другого блока.

С помощью манипулятора может быть взят одновременно только один блок, поэтому невозможно взять блок, на котором стоит еще один блок. Цель всегда заключается в том, что должны быть построены один или несколько столбиков из блоков, а сами задачи формулируются в терминах того, какие блоки находятся над указанными другими блоками. Например, задача может состоять в том, чтобы поставить блок A на в и блок C на D.

Для указания на то, что блок b находится на блоке x , где x — либо другой блок, либо стол, используется предикат $On(b, x)$. Для перемещения блока b с верхней поверхности блока x на верхнюю поверхность блока y применяется действие $Move(b, x, y)$. Итак, одним из предусловий перемещения b является то, что на нем не стоит какой-то другой блок. В логике первого порядка оно может быть представлено с помощью выражения $\neg \exists x On(x, b)$ или альтернативного выражения $\forall x \neg On(x, b)$. Эти выражения могут быть сформулированы как предусловия на языке ADL. Но мы могли бы оставаться в рамках языка Strips, введя новый предикат, $Clear(x)$, который является истинным, если на блоке x ничего нет (верхняя поверхность блока x свободна).

Действие $Move$ позволяет переместить блок b с блока x на блок y , если свободны верхние поверхности и блока b , и блока y . После выполнения этого действия верхняя поверхность блока x свободна, а блока y — нет. Формальное описание действия $Move$ в языке Strips состоит в следующем:

Action($Move(b, x, y)$, Precond: $On(b, x) \wedge Clear(b) \wedge Clear(y)$, Effect: $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)$) К сожалению, в этом действии предикат $Clear$ не сопровождается должным образом, если блок x или блок y находится на столе. Если $x = Table$, то результатом этого действия будет $Clear(Table)$, но поверхность стола не должна становиться свободной (поскольку она и так всегда свободна), а если $y = Table$, то действие имеет предусловие $Clear(Table)$, но вся поверхность стола не обязана быть свободной для того, чтобы на нее можно было поместить блок (поскольку на столе всегда и 3 Мир блоков, используемый в этих исследованиях планирования, гораздо проще по сравнению с версией Shrdlu, которая показана на с. 60.

без того достаточно места). Для исправления такого положения необходимо предусмотреть два изменения. Во-первых, введем еще одно действие для перемещения блока b с блока x на стол:

Action ($MoveToTable(b, x)$, Precond: $On(b, x) \wedge Clear(b)$), Effect: $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x)$) Во-вторых, примем интерпретацию предиката $Clear(b)$ как означающую, что "на b есть достаточно свободного места, чтобы поместился один блок". При этой интерпретации предикат $Clear(Table)$ всегда будет истинным. Единственная

проблема состоит в том, что ничто не будет препятствовать планировщику использовать действие $\text{Move}(b, x, \text{Table})$ вместо действия $\text{MoveToTable}(b, x)$. Можно смириться с этой проблемой (она приводит к созданию пространства поиска с размерами больше необходимого, но не становится причиной получения неправильных ответов) или ввести в предусловие Move предикат Block и добавить в предусловие действия Move выражение $\text{Block}(b) \wedge \neg \text{Block}(y)$.

Наконец, существует проблема фиктивных действий, таких как $\text{Move}(B, C, C)$, которые должны представлять собой пустую операцию, но имеют противоречивые результаты. Обычно принято игнорировать подобные проблемы, поскольку они редко вызывают выработку неверных планов. Правильный подход состоит в добавлении предусловий в неравенство, как показано в листинге 11.3.

Листинг 11.3. Задача планирования в мире блоков: построение столбика из трех блоков. Одним из решений является последовательность действий $[\text{Move}(B, \text{Table}, C), \text{Move}(A, \text{Table}, B)]$ Init{On(A,Table) $\wedge$ On(B, Table) $\wedge$ On(C, Table) $\wedge$ Block(A) $\wedge$ Block(B) $\wedge$ Block(C) $\wedge$ Clear(A) $\wedge$ Clear(B) $\wedge$ Clear(C)} Goal(On(A,B) $\wedge$ On(B, C)) Action($\text{Move}(b, x, y)$, Precond: On(b,x) $\wedge$ Clear(b) $\wedge$ Clear(y) $\wedge$ Block(b) $\wedge$ ($b \neq x$) $\wedge$ ($b \neq y$) $\wedge$ ($x \neq y$), Effect: On(b,y) $\wedge$ Clear(x) $\wedge$ $\neg$ On(b,x) $\wedge$ $\neg$ Clear(y)) Action ($\text{MoveToTable}(b, x)$, Precond: On(b,x) $\wedge$ Clear(b) $\wedge$ Block(b) $\wedge$ ($b \neq x$), Effect: On(b, Table) $\wedge$ Clear(x) $\wedge$ $\neg$ On(b,x)) 11.2. Планирование с помощью поиска в пространстве состояний Данный раздел посвящен описанию алгоритмов планирования. Наиболее простой подход состоит в использовании поиска в пространстве состояний. Поскольку описания действий в задаче планирования определяют и предусловия, и результаты, существует возможность организовать поиск в обоих направлениях: либо в прямом, от начального состояния, либо в обратном, от цели, как показано на рис. 11.1.

Кроме того, явные представления действий и целей могут использоваться для автоматического вывода эффективных эвристик.

At{PbA) At(P2, A) $\vee$ (P\A,B) $\wedge$ U(P2,A,B) ЛЦРЬ B) At(P2,A) J At(PbA) M(P2, B) At(PhA) At{Pb B) Ну^Л) б) At{PbB) At{Pb A) ?\у(P2,A,B) At(Pb B) АКРЬ B) Рис. 11.1. Два подхода к организации поиска плана: прямой (прогрессивный) поиск в пространстве состояний, начинающийся с начального состояния, в котором используются действия задачи для прямого поиска целевого состояния (а); обратный (регрессивный) поиск в пространстве состояний: поиск доверительного состояния (см. с. 141), начинающийся с целевого состояния (состояний), в котором для обратного поиска начального состояния используются инверсии действий (б) Прямой поиск в пространстве состояний Подход к организации планирования с использованием прямого поиска в пространстве состояний аналогичен подходу к решению задач, описанному в главе 3.

Иногда его называют $\wedge$ прогрессивным планированием, поскольку оно предусматривает продвижение в прямом направлении. При этом мы начинаем с начального состояния задачи и рассматриваем последовательности действий до тех пор, пока не находим последовательность, позволяющую достичь целевого состояния. Формулировка задач планирования в виде задач поиска в пространстве состояний приведена ниже.

- Начальным состоянием поиска является начальное состояние, взятое из задачи планирования. Вообще говоря, каждое состояние должно представлять собой множество положительных базовых литералов; литералы, не присутствующие в определении задачи, являются ложными.
- Всеми действиями, применимыми в некотором состоянии, являются такие действия, для которых выполнены предусловия. Состояние-преемник, являющееся результатом выполнения действия, вырабатывается путем добавления положительных литералов результата и удаления отрицательных литералов результата. (В случае использования литералов первого порядка к литералам результата необходимо применять унификатор из предусловий.) Обратите внимание на то, что во всех задачах планирования используется единственная функция определения преемника, что является следствием применения явного представления действий.

- В ходе проверки цели осуществляется проверка того, удовлетворяет ли данное состояние цели условиям задачи планирования.
- Стоимость этапа для каждого действия обычно равна 1. Хотя было бы несложно предусмотреть использование различных стоимостей для разных действий, такая возможность в планировщиках Strips применяется редко.

Напомним, что при отсутствии функциональных символов пространство состояний задачи планирования является конечным. Поэтому полным алгоритмом планирования должен быть любой алгоритм поиска в графе, который является полным (например, A^*).

Начиная с самых первых дней исследований по планированию (примерно с 1961 года) и до сравнительно недавнего времени (примерно до 1998 года), предполагалось, что прямой поиск в пространстве состояний был бы слишком неэффективен для того, чтобы иметь практическое значение. Причины этого понять совсем несложно — достаточно вернуться в начало раздела 11.1. Во-первых, прямой поиск не решает проблему не относящихся к делу действий — рассматриваются все действия, применимые из каждого состояния. Во-вторых, без хорошей эвристики этот подход быстро приводит к чрезмерному увеличению объема вычислений. Рассмотрим задачу воздушных грузовых перевозок с 10 аэропортами, где в каждом аэропорту имеется 5 самолетов и 20 единиц груза. Цель состоит в том, чтобы переместить все грузы из аэропорта А в аэропорт В. Существует простое решение этой задачи: погрузить 20 единиц груза в один из самолетов в аэропорту А, полететь на этом самолете в аэропорт В и выгрузить груз. Но формальный поиск этого

решения может оказаться затруднительным, поскольку средний коэффициент ветвления является огромным: каждый из этих $5 \times 10 = 50$ самолетов может полететь в 9 других аэропортов, а каждая из $20 \times 10 = 200$ единиц груза может быть либо разгружена (если она загружена), либо загружена в любой самолет в аэропорту, где она находится (если она разгружена).

Допустим, что в среднем существует около 10 000 возможных действий, поэтому дерево поиска с глубиной, равной глубине очевидного решения, имеет около 1000^{41} узлов. Очевидно, что для обеспечения эффективности такого поиска потребуется очень точная эвристика. Мы рассмотрим некоторые возможные эвристики после описания обратного поиска.

Обратный поиск в пространстве состояний Обратный поиск в пространстве состояний был кратко описан в составе двунаправленного поиска в главе 3. В этой главе было отмечено, что задача организации обратного поиска может оказаться сложной, если целевые состояния описаны с помощью множества ограничений, а не перечислены явно. В частности, не всегда очевидно, как должно составляться описание возможных преемников множества целевых состояний. А в этой главе будет показано, что представление Strips позволяет решить такую задачу очень легко, поскольку множества состояний могут быть описаны с помощью литералов, которые должны быть истинными в этих состояниях.

Основным преимуществом обратного поиска является то, что он позволяет рассматривать только $\wedge$ релевантные действия. Действие релевантно конъюнктивной цели, если оно достигает одного из конъюнктов данной цели. Например, целью опи-

санной выше задачи по воздушной перевозке грузов с 10 аэропортами была доставка 20 единиц груза в аэропорт B или, точнее:

$At(C_i, B) \wedge L_{At\{C_2, B\}} \wedge \dots \wedge L_{At\{C_{20}, B\}}$ Теперь рассмотрим конъюнкт $At(C_{1f}B)$. Двигаясь в обратном направлении, можно найти действия, имеющие этот результат. Таковым является только одно действие: $Unload(C_{1f}, B)$, где самолет не задан.

Обратите внимание на то, что имеется также много нерелевантных действий, способных привести к целевому состоянию. Например, можно организовать перелет пустого самолета из аэропорта JFK в аэропорт SFO; это действие позволяет достичь целевого состояния из состояния-предшественника, в котором самолет находился в JFK и все целевые конъюнкты были удовлетворены. Обратный поиск, в котором допускаются нерелевантные действия, по-прежнему будет полным, но гораздо менее эффективным. Если решение существует, то должно быть найдено с помощью обратного поиска, допускающего только релевантные действия. Наличие такого ограничения, в котором допускаются только релевантные действия, означает, что процедура обратного поиска часто имеет гораздо более низкий коэффициент

ветвления по сравнению с прямым поиском. Например, в рассматриваемой задаче грузовых воздушных перевозок имеется около 1000 действий, ведущих в прямом направлении из начального состояния, но только 20 действий, позволяющих перейти в обратном направлении от цели.

Поиск в обратном направлении иногда называют $\wedge$ регрессивным планированием. Основной вопрос регрессивного планирования состоит в следующем: есть ли такие состояния, из которых применение некоторого действия приводит к цели?

Вычисление описаний таких состояний называется регрессией цели через действие. Для того чтобы определить, как можно найти ответ на указанный выше вопрос, рассмотрим пример с воздушными грузовыми перевозками. Мы имеем цель $At(C_i, V) \wedge AC(C_2, V) \wedge \dots \wedge At(C_{20}, V)$ и релевантное действие $Unload\{Clfpf\ V\}$, которое позволяет достичь первого конъюнкта. Соответствующее действие будет выполнимо, только если выполнены его предусловия. Поэтому любое состояние-преемник должно включать эти предусловия: $In\{Clfp\} \wedge At(p, V)$. Более того, подцель $At(C_{if}\ V)$ не должна быть истинной в состоянии-преемнике⁴. Поэтому описание состояния-преемника является таковым:

$In(C_i, p) \wedge At(p, V) \wedge At(C_2, V) \wedge \dots \wedge At(C_{20}/V)$ Кроме выполнения того требования, чтобы действия достигали некоторого желаемого литерала, должно также соблюдаться требование, чтобы действия не отменяли какие-либо желаемые литералы. Любое действие, удовлетворяющее этому ограничению, называется $\wedge$ совместимым. Например, действие $Load\{C_2, p\}$ не будет совместимым с текущей целью, поскольку оно отрицает литерал $At(c_2, V)$.

Составив определения понятий релевантности и совместимости, мы можем описать общий процесс формирования преемников для обратного поиска. Допустим, ⁴ Если подцель была истинной в состоянии-преемнике, то данное действие все равно должно привести к целевому состоянию. С другой стороны, подобные действия являются нерелевантными, поскольку они не могут сделать цель истинной.

что при наличии описания цели G имеется действие A , которое является релевантным и совместимым. Соответствующий преемник может быть определен, как описано ниже.

- Любые положительные результаты действия A , которые появляются в цели G , удаляются.
- Добавляется каждый литерал предусловия A , если он еще не присутствует в определении действия.

Для осуществления обратного поиска могут использоваться любые из стандартных алгоритмов поиска. Завершение работы происходит после выработки такого описания преемника, которое соответствует начальному состоянию задачи планирования.

В случае использования логики первого порядка для обеспечения соответствия с начальным состоянием может потребоваться подстановка переменных в описание преемника. Например, описание преемника, приведенное в предыдущем абзаце, будет соответствовать такому начальному состоянию после подстановки {p/ P12}:

In(Clf P12) Л At(Pi2,B) л At(C2, B) л ... л At(C20, B) Затем данная подстановка должна быть применена к действиям, ведущим из этого состояния к цели, что приводит к получению решения [Unload(Clf P12, B)].

Эвристики для поиска в пространстве состояний Итак, из описанного выше следует, что ни прямой, ни обратный поиск не может быть эффективным без хорошей эвристической функции. Как было указано в главе 4, эвристическая функция оценивает расстояние от некоторого состояния до цели; в планировании Strips стоимость каждого действия равна 1, поэтому расстояние измеряется количеством действий. Основная идея состоит в том, что нужно рассматривать результаты действий и цели, которые должны быть достигнуты, а после этого выдвигать предположение, сколько действий потребуется для достижения всех целей. Задача определения точного количества действий является NP-трудной, но чаще всего существует возможность найти приемлемые оценки без слишком большого объема вычислений. Необходимо также иметь возможность вывести допустимую эвристику, которая не приводит к переоценке. Такая эвристика может использоваться в сочетании с поиском A* для нахождения оптимальных решений.

Для составления эвристики могут быть опробованы два подхода. Первый из них состоит в том, чтобы вывести ослабленную задачу из заданной спецификации задачи (см. главу 4). Оптимальная стоимость решения для ослабленной задачи (которую можно надеяться очень легко решить) задает допустимую эвристику для первоначальной задачи. Второй подход состоит в том, что можно воспользоваться предположением о возможности применения алгоритма, действующего исключительно по принципу "разделяй и властвуй". Такое предположение называется предположением о ^ независимости подцелей, согласно которому стоимость решения конъюнкции подцелей аппроксимируется суммой стоимостей решения каждой подцели независимо друг от друга. Предположение о независимости подцелей может быть оптимистическим или пессимистическим. Оно является оптимистическим, если имеются отрицательные взаимодействия субпланов для каждой подцели, например, если действие в одном субплане удаляет цель, достигнутую в другом субплане. С другой сто-

роны, такое предположение является пессимистическим и поэтому недопустимым, если субпланы содержат избыточные действия, например, два таких действия, которые могут быть заменены одним действием в объединенном плане.

Рассмотрим способ вывода ослабленных задач планирования. Поскольку имеются явные представления предусловий и результатов, такой способ может предусматривать модификацию указанных представлений. (Сравните этот подход с задачами поиска, в которых функция определения преемника рассматривалась как "черный ящик".) Простейшая идея состоит в том, что задача может быть ослаблена путем удаления из действий всех предусловий. В таком случае каждое действие всегда будет применимым и любой литерал может быть достигнут за один шаг (если есть какое-либо применимое действие; в противном случае цели достичь невозможно). Из этого почти безусловно следует, что количество шагов, необходимых для решения конъюнкции цели, равно количеству невыполненных целей — почти, но не совсем, поскольку, во-первых, могут существовать два таких действия, каждое из которых удаляет литерал цели, достигнутый другим, и, во-вторых, некоторое действие может достигать нескольких целей. Если же ослабленная задача будет применяться в сочетании с предположением о независимости подцелей, это позволит воспользоваться предположением о том, что указанных проблем не существует, и поэтому результирующая эвристика будет точно соответствовать количеству невыполненных целей.

Во многих случаях существует возможность получить более точную эвристику, рассматривая, по меньшей мере, положительные взаимодействия, вытекающие из наличия действий, достигающих нескольких целей. Прежде всего, можно еще больше ослабить задачу, удаляя отрицательные результаты (см. упр. 11.6). Затем можно подсчитать минимальное количество требуемых действий таким образом, чтобы объединение положительных результатов этих действий позволяло выполнить данную цель. Например, рассмотрим следующее определение:

Goal {A л B л C} Action(X, Effect: A л P) Action {Y, Effect: B л C л Q} Action_iZ, Effect: B л P л Q) Минимальное множественное покрытие цели {A, B, C} задается действиями {X, Y}, поэтому эвристика множественного покрытия возвращает стоимость 2.

Такая оценка лучше по сравнению с предположением о независимости подцелей, которое позволяет получить эвристическое значение 3. Остается непреодоленной лишь одна небольшая неприятная особенность: задача поиска множественного покрытия является NP-трудной. Простой жадный алгоритм поиска множественного покрытия гарантирует возвращение значения, которое находится в пределах коэффициента $1/\log p$ от истинного минимального значения, где p — количество литералов в цели, но обычно на практике данный алгоритм действует намного лучше по сравнению с этой оценкой. К сожалению, жадный алгоритм не обеспечивает гарантий достижимости для такой эвристики.

Существует также возможность создавать ослабленные задачи, удаляя отрицательные результаты без удаления предусловий. Таким образом, если некоторое действие в первоначальной задаче имеет результат A л -pB, то в

ослабленной задаче будет иметь результат A. Это означает, что не нужно беспокоиться об отрицательных взаимодействиях субпланов, поскольку ни одно действие не способно удалить лите-

ралы, достигнутые другим действием. Стоимость решения полученной в конечном итоге ослабленной задачи задает то, что принято называть эвристикой с $\wedge$ пустым списком удаления. Эта эвристика является весьма точной, но для ее вычисления требуется действительно выполнить некоторый (простой) алгоритм планирования. Тем не менее на практике поиск в условиях ослабленной задачи часто происходит достаточно быстро для того, чтобы имело смысл применение этой оценки стоимости.

Описанные здесь эвристики могут использоваться либо в прогрессивном, либо в регрессивном направлении. Ко времени написания данной книги первые места в рейтинге эффективности занимали прогрессивные планировщики, в которых применялась эвристика с пустым списком удаления. Такая ситуация вполне может измениться в результате исследования новых эвристик и новых методов поиска.

Поскольку задачи планирования являются экспоненциально трудными⁵, ни один алгоритм не может быть достаточно эффективным для решения всех задач, но с использованием эвристических методов, описанных в этой главе, могут быть решены многие практические задачи, и таких задач стало гораздо больше по сравнению с теми, которые были решаемыми всего лишь несколько лет тому назад.

11.3. Планирование с частичным упорядочением Прямой и обратный поиск в пространстве состояний представляют собой особые формы поиска полностью упорядоченного плана. В них рассматриваются только строго линейные последовательности действий, непосредственно связанные с начальным или целевым состоянием. Это означает, что такие методы поиска не позволяют воспользоваться преимуществами декомпозиции задачи. Вместо того чтобы обеспечить отдельную проработку каждой подзадачи, эти методы вынуждены всегда поддерживать принятие решений о том, как упорядочить действия, относящиеся ко всем подзадачам. Но обычно более предпочтительным является подход, позволяющий работать над несколькими подцелями независимо, достигать их с помощью нескольких субпланов, а затем объединять эти субпланы.

Подобный подход обладает также тем преимуществом, что позволяет добиться большей гибкости при определении последовательности, в которой составляется окончательный план. Это означает, что планировщик вначале может работать над "очевидными" или "важными" решениями, не будучи вынужденным прорабатывать все этапы в хронологическом порядке. Например, планирующий агент, который находится в Беркли и желает попасть в Монте-Карло, может вначале попытаться найти рейс из Сан-Франциско в Париж; получив информацию о времени

отправления и прибытия этого рейса, он может затем заняться поиском способов того, как доехать и выехать из соответствующих аэропортов.

Общая стратегия, в которой в процессе поиска выбор определенных этапов откладывается на более позднее время, называется стратегией с $\wedge$ наименьшим вкладом (least commitment). Формального определения стратегии с наименьшим вкладом не существует, поскольку очевидно, что на любом этапе поиска должен быть сделан 5 Формально процедура планирования в стиле Strips является PSPACE-полной, если только не рассматриваются действия, которые имеют только положительные предусловия и лишь один литерал результата [214].

определенный вклад в окончательное решение, так как в противном случае поиск окажется непродуктивным. Несмотря на такую неформальность, наименьший вклад — это полезная концепция для анализа того, какие решения должны быть приняты в любой задаче поиска.

Приведенный в этом разделе первый конкретный пример будет гораздо проще по сравнению с планированием отпуска, проводимого в Монте-Карло. Рассмотрим простую задачу надевания пары туфель. Ее можно описать как формальную задачу планирования следующим образом:

GoalRightShoeOn л LeftShoeOn Init() Action(RightShoe,Precond: RightSockOn,Effect: RightShoeOn) Action(RightSock,Effect: RightSockOn) Action(LeftShoe,Precond: LeftSockOn,Effect: LeftShoeOn) Action{LeftSock,Effect: LeftSockOn) Планировщик должен быть способен предложить последовательность из двух действий, RightSock (Надеть правый носок), затем RightShoe (Надеть правую туфлю), чтобы достичь первого конъюнкта этой цели, и последовательность LeftSock (Надеть левый носок), затем LeftShoe (Надеть левую туфлю), чтобы достичь второго конъюнкта. После этого эти две последовательности могут быть объединены для создания окончательного плана. В ходе этого планировщик будет манипулировать двумя подпоследовательностями независимо, не задумываясь над тем, должно ли какое-то действие в одной последовательности быть выполнено до или после некоторого действия в другой. Любой алгоритм планирования, способный включить в план два действия без указания того, какое из них должно быть выполнено первым, называется $\wedge$ планировщиком с частичным упорядочением (partialorder planner). На рис. 11.2 показан план с частичным упорядочением, который представляет собой решение задачи надевания туфель и носков. Обратите внимание на то, что это решение представлено в виде графа, а не последовательности действий. Заслуживают также внимания "фиктивные" действия, называемые Start и Finish, которые отмечают начало и конец плана. Назвав эти ситуации действиями, мы можем упростить структуру плана, поскольку теперь каждый этап плана становится действием. Это решение с частичным упорядочением соответствует шести

возможным планам с полным упорядочением; каждый из них называется $\wedge$ линеаризацией плана с частичным упорядочением.

Планирование с частичным упорядочением может быть реализовано в виде поиска в пространстве планов с частичным упорядочением. (Начиная с этого момента, мы будем называть их просто "планами".) Это означает, что поиск начинается с пустого плана. После этого рассматриваются способы уточнения плана до тех пор, пока не удастся составить полный план, который решает данную задачу. Действия, рассматриваемые в этом поиске, являются не действиями в мире, а действиями в планах: добавление в план этапа; наложение упорядочения, согласно которому одно действие должно занять место перед другим; и т.д.

В данном разделе будет определен алгоритм POP (Partial-Order Planning) для планирования с частичным упорядочением. В соответствии с общепринятой традицией алгоритм POP оформляется в виде отдельной программы, но мы вместо этого определим задачу планирования с частичным упорядочением как экземпляр задачи поиска. Это позволит нам сосредоточиться на этапах уточнения плана, которые могут

быть применены, и не задумываться над тем, каким образом алгоритм осуществляет исследование пространства поиска. В действительности после формулировки задачи поиска может быть применен широкий перечень неинформированных или эвристических методов поиска. В действительности после формулировки задачи поиска может быть применен широкий перечень неинформированных или эвристических методов поиска.

Рис. 11.2. План надевания туфель и носков с частичным упорядочением и шесть соответствующих линеаризаций в виде планов с полным упорядочением

Напомним, что состояниями рассматриваемой задачи поиска должны быть планы (в основном незаконченные). Чтобы избежать путаницы с состояниями мира, мы будем вести речь о планах, а не о состояниях. Каждый план имеет описанные ниже четыре компонента, где первые два определяют этапы плана, а последние два выполняют функции учета, позволяющие определить, как может быть дополнен план.

- Множество действий, из которых состоят этапы плана. Эти действия берутся из множества действий в задаче планирования. "Пустой" план содержит только действия Start и Finish. Действие Start не имеет предусловий, а его результатом являются все литералы в начальном состоянии задачи планирования. Действие Finish не имеет результатов, а его предусловиями являются литералы цели в задаче планирования.
- Множество $\wedge$ ограничений упорядочения. Каждое ограничение упорядочения находится в форме $A \prec B$, которая читается как "A перед B" и означает, что действие A должно быть выполнено в какое-то время перед действием B, но не обязательно непосредственно перед ним. Ограничения упорядочения должны описывать правильный вариант частичного упорядочения. Любой цикл (такой как $A \prec B$ и $B \prec A$) представляет противоречие, поэтому ни одно ограничение

упорядочения не может быть добавлено в план, если оно создает цикл. ческих методов поиска.

План с частичным упорядочением Left Sock eftSo Start Right Sock J \ rlcOn RiehtSc Left 1 Shoe f ckOn Right Shoe \ / LeftSho eOn, Right Finish ShoeOn Планы с полным упорядочением Start Right Sock Left Sock Right Shoe i Left Shoe i Finish Start Right Sock i Left Sock Left Shoe Right Shoe Finish Start Left Sock f Right Sock Right Shoe i Left Shoe Fin ish

- Множество $\wedge$ причинных связей. Причинная связь между двумя действиями A и B в плане записывается как $A \rightarrow B$ и читается как "A $\wedge$ достигает p для B".

Например, в следующей причинной связи: RightSockRig $\wedge$ kOnRightShoe утверждается, что RightSockOn (надет правый носок) представляет собой результат действия RightSock и предусловие действия RightShoe. В ней также содержится утверждение, что предусловие RightSockOn должно оставаться истинным со времени действия RightSock до времени действия RightShoe. Другими словами, план не может быть дополнен путем добавления какого-либо нового действия C, которое Ж конфликтует с причинной связью. Действие C конфликтует со связью A $\rightarrow$?, если C имеет результат -лр и если C может (в соответствии с ограничениями упорядочения) происходить после A и перед B. Некоторые авторы называют причинные связи интервалами защиты, поскольку связь A $\rightarrow$? защищает предусловие p от его отрицания в интервале от A до в. • Множество $\wedge$ открытых предусловий. Предусловие является открытым, если оно не достигнуто с помощью какого-то действия в плане. Планировщики действуют по принципу сокращения множества открытых предусловий до пустого множества без внесения противоречия.

Например, окончательный план, показанный на рис. 11.2, имеет следующие компоненты (не показаны ограничения упорядочения, которые распространяются на каждое действие после действия Start и перед действием Finish):

Действия: {RightSock, RightShoe, LeftSock, LeftShoe, Start, Finish} Упорядочения: {RightSock < RightShoe, LeftSock -< LeftShoe} Связи: { RightSock Rig $\wedge$ kOn Right Shoe, Lef tSockhet- $\wedge$ On Left Shoe, RightShoe Rig $\wedge$ eOnFinish, LeftShoeLet- $\wedge$ eOnFinish} Открытые предусловия: {} Мы определяем $\wedge$ согласованный план (consistent plan) как план, в котором не имеется циклов в ограничениях упорядочения и нет конфликтов с причинными связями. Согласованный план без открытых предусловий представляет собой решение.

Даже краткие размышления должны убедить читателя в справедливости следующего утверждения: $\wedge$ каждая линейаризация решения с частичным упорядочением представляет собой решение с полным упорядочением, выполнение которого из начального состояния позволяет достичь целевого

состояния. Это означает, что можно распространить понятие "выполнения плана" с планов с полным упорядочением на планы с частичным упорядочением. План с частичным упорядочением выполняется путем повторного выбора любого из возможных следующих действий. Как будет показано в главе 12, такая гибкость, предоставляемая агенту в ходе выполнения плана, может быть очень полезной в тех обстоятельствах, когда мир к нему не благосклонен.

Гибкое упорядочение позволяет также упростить комбинирование меньших планов в более крупные, поскольку каждый из меньших планов допускает переупорядочение его действий для предотвращения конфликтов с другими планами.

Теперь мы готовы сформулировать задачу поиска, которую решает алгоритм POP, как показано ниже. Начнем с формулировки, подходящей для задач планирования в пропозициональной логике, оставляя на будущее осложнения, связанные с исполь-

зованием логики первого порядка. Как обычно, это определение включает начальное состояние, действия и проверку цели. • Начальный план содержит действия Start и Finish, ограничение упорядочения $\text{Start} < \text{Finish}$, не включает ни одной причинной связи и имеет все предусловия в действии Finish в качестве открытых предусловий. • Функция определения преемника произвольным образом выбирает одно открытое предусловие p действия B и вырабатывает план-преемник для каждого возможного согласованного способа выбора действия A , которое достигает p .

Согласованность обеспечивается принудительно следующим образом.

1. Причинная связь $A \rightarrow B$ и ограничение упорядочения $A < B$ добавляются к плану. Действие A может быть существующим действием в плане или новым действием. Если оно является новым, добавить его к плану, а также добавить $\text{Start} < A$ и $A < \text{Finish}$.

2. Разрешаются конфликты между новой причинной связью и всеми существующими действиями, а также между действием A (если оно является новым) и всеми существующими причинными связями. Конфликт между $A \rightarrow B$ и C разрешается путем обеспечения того, чтобы действие C происходило в какое-то время за пределами интервала защиты, либо за счет добавления ограничения $B < C$, либо путем добавления ограничения $C < A$. Для одного из них или обоих добавляются состояния-преемники, если они приводят к созданию согласованных планов. • В ходе проверки цели осуществляется проверка того, является ли текущий план решением первоначальной задачи планирования. Поскольку вырабатываются только согласованные планы, в проверке цели достаточно только проконтролировать, есть ли еще открытые предусловия.

Еще раз напомним, что действия, рассматриваемые в алгоритмах поиска при такой формулировке, представляют собой этапы уточнения плана, а не реальные действия из самой проблемной области. Поэтому, строго говоря, стоимость пути не имеет значения, поскольку важна только суммарная стоимость реальных действий в плане, к которому ведет этот путь. Тем не менее существует возможность определить функцию стоимости пути, которая отражает стоимости реальных планов; для этого можно назначать цену 1 за каждое реальное действие, добавленное к плану, и 0 — за все остальные этапы уточнения. В таком случае значение функции $d(p)$, где p — некоторый план, будет равно количеству реальных действий в плане. Может также использоваться эвристическая оценка $h(n)$.

На первый взгляд может показаться, что функция определения преемника должна вырабатывать преемников для каждого открытого предусловия p , а не только для одного из них. Но такой подход был бы избыточным и неэффективным по той же причине, по которой алгоритмы удовлетворения ограничений не включают преемников для каждой возможной переменной, поскольку порядок, в котором рассматриваются открытые предусловия (как и порядок, в котором рассматриваются переменные CSP), является коммутативным (см. с. 214). Таким образом, мы можем выбирать произвольное упорядочение и все еще иметь полный алгоритм. Выбор правильного упорядочения может привести к более быстрому поиску, но все упорядочения оканчиваются одним и тем же множеством решений-кандидатов.

Пример планирования с частичным упорядочением Теперь рассмотрим, как можно с помощью алгоритма POP решить задачу с запасным колесом, описанную в разделе 11.1. Описание этой задачи повторно приведено в листинге 11.4.

Листинг 11.4. Описание простой задачи замены колеса со стертой покрышкой
 Init{At(Flat,Axle) л At(Spare,Trunk)) Goal(At(Spare,Axle)) Act ion(Remove(Spare, Trunk) ,
 Precond: At(Spare, Trunk) Effect: -iAt (Spare, Trunk) а At (Spare, Ground))
 Action(Remove(Flat,Axle), Precond: At(Flat,Axle) Effect: -At{Flat,Axle) л At(Flat, Ground))
 Action(PutOn(Spare, Axle), Precond: At(Spare,Ground) а —| At(Flat,Axle) Effect: -
 At(Spare,Ground) а At(Spare,Axle)) Action(LeaveOvernight, Precond:

Effect: -iAt (Spare, Ground) а -^At (Spare, Axle) а —At (Spare, Trunk) а -At (Flat, Ground)
 а -At (Flat, Axle)) Поиск решения начинается с начального плана, содержащего
 действие Start с результатом At (Spare, Трипк) л At (Flat, Axle) и действие Finish с
 единственным предусловием At (Spare, Axle). Затем вырабатываются преемники
 путем взятия открытого предусловия для его проработки (не допускающей отмены)
 и выбора среди возможных действий для его достижения. На данный момент мы не
 будем задумываться над тем, что нужно воспользоваться какой-то эвристической
 функцией, которая могла бы помочь в выработке этих решений, и станем выбирать

варианты, которые внешне кажутся произвольными. Последовательность событий описана ниже.

1. Взять единственное открытое предусловие, At (Spare, Axle), действия Finish.

Выбрать единственное применимое действие, PutOn(Spare, Axle).

2. Взять предусловие At (Spare, Ground) действия PutOn(Spare, Axle).

Выбрать единственное применимое действие, Remove (Spare, Trunk), чтобы достичь его. Результирующий план показан на рис. 11.3.

At(Spare, Trunk) Remove(Spare, Trunk) Start At(Spare, Trunk) At(Spare, Ground) At(Flat, Axle) -, At(Flat, Axle) PutOn(Spare, Axle) At(Spare, Axle) Finish Рис. 11.3. Незаконченный план с частичным упорядочением для задачи замены колеса после выбора действий, соответствующих первым двум открытым предусловиям. В прямоугольниках показаны действия, предусловия которых находятся слева, а результаты — справа (результаты не показаны, кроме результатов действия Start/ Сплошные линии со стрелками показывают причинные связи, защищающие высказывание, находящееся у острия стрелки

Взять предусловие —At {Flat, Axle) действия PutOn{Spare, Axle).

Просто для того чтобы поступить вопреки здравому смыслу, выберем действие LeaveOvernight, а не действие Remove {Flat, Axle). Но обратите внимание на то, что действие LeaveOvernight также имеет результат -iAt (Spare, Ground), который означает, что оно конфликтует со следующей причинной связью:

Remove {Spare, Trunk) At(spare^round) PutOn (spare, Axle) Чтобы разрешить этот конфликт, добавим ограничение упорядочения, которое помещает действие LeaveOvernight перед действием Remove {Spare, Trunk). Возникающий при этом план показан на рис. 11.4.

(Предлагаем читателю ответить на вопросы, почему такое дополнение позволяет разрешить конфликт и почему нет другого способа его разрешить?) Start At(Spare, Trunk) At(Spare, Trunk) At(Flat, Axle) Remove(Spare, Trunk) At(Spare, Ground) -1 At{Flat, Axle) PutOn(Spare, Axle) + ~At(Spare, Axle) Finish LeaveOvernight -1 At(Flat, Axle) -1 At(Flat, Ground) At{Spare, Axle) ~> At(\$pare, Ground) -i At(\$pare, Trunk) Рис. 11.4. План, возникающий после выбора LeaveOvernight в качестве действия для достижения предусловия -At (Flat, Axle). Чтобы избежать конфликта с причинной связью, исходящей из действия Remove (Spare, Trunk) которая защищает действие At (Spare, Ground), на LeaveOvernight накладывается ограничение, чтобы это действие происходило перед Remove (Spare, Trunk), как показано штриховой линией со стрелкой 4. В этот момент единственным оставшимся открытым предусловием является предусловие At (Spare, Trunk) действия Remove {Spare, Trunk).

Единственным действием, позволяющим достичь этого предусловия, является существующее действие Start, но причинная связь от Start к Remove {Spare, Trunk} конфликтует с результатом -.At {Spare, Trunk} действия LeaveOvernight. В данный момент не существует способа разрешить конфликт с действием LeaveOvernight—его нельзя переупорядочить таким образом, чтобы оно находилось перед Start (поскольку ни одно действие не может происходить перед действием Start), и нельзя переупорядочить его так, чтобы оно находилось после Remove {Spare, Trunk} (поскольку уже имеется ограничение, которое упорядочивает его перед Remove{Spare, Trunk}). Поэтому необходимо вернуться к одному из предыдущих состояний, удалить действие LeaveOvernight и две последние причинные связи, возвратившись в состояние, показанное на рис. 11.3. Планировщик фактически доказал, что действие LeaveOvernight не может применяться в качестве способа замены колеса.

5. Еще раз рассмотрим предусловие —At(Flat,Axle) действия PutOn(Spare, Axle). На этот раз выберем действие Remove (Flat, Axle).

6. Снова возьмем предусловие At (Spare, Trunk) действия Remove(Spare, Trunk) и выберем действие Start, чтобы его достичь. На сей раз конфликты не возникают.

7. Возьмем предусловие At (Flat, Axle) действия Remove (Flat, Axle) и выберем действие Start для его достижения. Это дает нам полный согласованный план (иными словами, решение), как показано на рис. 11.5.

At(Spare, Trunk) Remove(Spare,Trunk) Start >pare, iрипк.) | кешцус^эpare, l пшк; [^ / \ At(Spare, Trunk) At{Spare,Ground) At(FlatAxle) —iAt{Flat,Axle) \ , / Attaint Avln\ Drtmn\o/'Plof Лл/1«=Л Т PutOn(Spare,Axle) - At(Spare,Axle) Finish Al(FlatMle) Remove(Flat,Axle) Рис. 11.5. Окончательное решение задачи замены колеса. Обратите внимание на то, что действия Remove (Spare, Trunk; и Remove (Flat, Axle) могут выполняться в любом порядке, при условии, что они завершаются перед действием PutOn (Spare, Axle.) Хотя данный пример очень прост, он иллюстрирует некоторые преимущества планирования с частичным упорядочением. Во-первых, причинные связи способствуют заблаговременному отсечению тех частей пространства поиска, которые не содержат решений из-за неустранимых конфликтов. Во-вторых, решение, приведенное на рис. 11.5, представляет собой план с частичным упорядочением. В данном случае преимущества невелики, поскольку имеются только две возможные линеаризации; тем не менее повышенная гибкость выбора вариантов может пойти агенту на пользу, например, если ему придется менять колесо в середине шоссе с интенсивным движением.

Этот пример также указывает на некоторые возможные усовершенствования, которые вполне могут быть осуществлены. Например, в данном случае налицо дублирование усилий: действие Start соединяется связью с действием Remove (Spare,

Trunk) до того, как конфликт вызывает возврат, а затем эта связь снова разрывается из-за возврата, даже несмотря на то, что она не участвует в конфликте. После этого в ходе продолжения поиска связь снова восстанавливается. Такое развитие событий является типичным для хронологических возвратов, и его последствия могут быть смягчены с помощью возвратов, управляемых с учетом зависимостей.

Планирование с частичным упорядочением и несвязанными переменными В этом разделе рассматриваются сложности, которые могут возникнуть, если алгоритм POP используется с представлениями действий в логике первого порядка, которые могут включать переменные. Предположим, что имеется задача в мире блоков (см. листинг 11.3) с открытым предусловием $On(A, B)$ и следующим действием:

Action (Move (b, x, y) , Precond: $On(b, x) \wedge Clear(b) \wedge Clear(y)$, Effect: $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)$)

Это действие достигает результата $On(A, B)$, поскольку результат $On(b, y)$ унифицируется с термом $On(A, B)$ с помощью подстановки $\{b/A, y/B\}$. Затем применим эту подстановку к действию и получим следующее:

Action {Move {A, x, B} Precond: $On(A, x) \wedge Clear(A) \wedge Clear(B)$ Effect: $On\{A, B\} \wedge Clear(x) \wedge \neg On\{A, x\} \wedge \neg Clear(B)$ } При этом переменная x остается несвязанной. Тем самым в данном действии определено, что блок A нужно переместить откуда-то, а откуда, не сказано. В этом заключается еще один пример реализации принципа наименьшего вклада: мы можем отложить выполнение выбора до какого-то этапа, в котором сам план укажет для нас этот выбор. Например, предположим, что в начальном состоянии имеется предусловие $On(A, D)$. В таком случае для достижения предусловия $On(A, x)$ может использоваться действие Start со связыванием x с D. Такая стратегия с ожиданием дополнительной информации перед выбором x часто является более эффективной по сравнению с опробованием любого возможного значения x и возвратом в ответ на неудачный выбор каждого из этих значений.

Из-за наличия переменных в предусловиях и действиях процесс обнаружения и разрешения конфликтов усложняется. Например, после добавления к плану действия Move (A, x, B) потребуется следующая причинная связь:

Move (A, x, B) $\circ^4$ Finish Если имеется другое действие M2 с результатом $\neg On(A, z)$, то M2 конфликтует, только если переменная z равна B. Чтобы учесть такую возможность, расширим представление планов для включения множества ограничений неравенства в форме $z \neq x$, где z — переменная, а x — либо другая переменная, либо константный символ. В данном случае можно было бы разрешить конфликт путем добавления ограничения $z \neq B$, которое означает, что в будущих дополнениях к плану разрешается конкретизировать переменную z любым значением, кроме B. При каждом применении к плану некоторой подстановки необходимо контролировать, чтобы подобные неравенства не противоречили этой

подстановке. Например, подстановка, которая включает x/u , конфликтует с ограничением неравенства $x \neq u$. Такие конфликты не могут быть разрешены, поэтому планировщику приходится выполнять возврат.

Более подробный пример планирования по алгоритму POP с переменными в мире блоков приведен в разделе 12.6.

Эвристики для планирования с частичным упорядочением В отличие от планирования с полным упорядочением, планирование с частичным упорядочением обладает явным преимуществом, поскольку позволяет выполнять декомпозицию задач на подзадачи. Оно имеет также определенный недостаток, который заключается в том, что состояния не представлены явно, поэтому труднее оценить, насколько далек план с частичным упорядочением от ситуации достижения цели. К тому же в настоящее время существует гораздо меньшее понимание того, как следует вычислять точные эвристики для планирования с частичным упорядочением, чем для планирования с полным упорядочением.

Наиболее очевидная эвристика состоит в подсчете количества различных открытых предусловий. Такая эвристика может быть улучшена путем вычитания из указанной величины количества открытых предусловий, которые согласуются с литералами в состоянии Start. Как и в случае с полным упорядочением, такая эвристика приводит к переоценке стоимости, если имеются действия, достигающие нескольких подцелей, и недооценке стоимости, если возникают отрицательные взаимодействия этапов плана. В следующем разделе представлен подход, позволяющий получать гораздо более точные эвристики из ослабленной задачи.

Эвристическая функция используется для выбора плана, подлежащего уточнению. При наличии такого выбора алгоритм вырабатывает преемников на основе определения единственного открытого предусловия, которое следует дополнительно проработать. Как и в случае выбора переменной в алгоритмах удовлетворения ограничений, такое решение оказывает большое влияние на эффективность. Для алгоритмов планирования может быть адаптирована эвристика с наиболее ограниченной переменной, применяемая в задачах CSP, и эта эвристика, по-видимому, неплохо действует. Ее идея состоит в том, чтобы определить открытое условие, которое может быть выполнено с помощью наименьшего количества способов. Возникают два особых случая применения этой эвристики. Во-первых, если какого-то открытого условия невозможно достичь с помощью любого действия, оно должно быть выбрано данной эвристикой; это — вполне целесообразно, поскольку раннее обнаружение неразрешимой ситуации позволяет сэкономить большой объем работы. Во-вторых, если открытое условие может быть достигнуто только одним способом, оно также должно быть выбрано, поскольку такое решение неизбежно и позволяет налагать дополнительные ограничения на другие варианты выбора, которые еще должны быть сделаны. Хотя процедура

полного вычисления количества способов выполнения каждого открытого условия является дорогостоящей и не всегда оправданной, эксперименты показали, что применение этих двух частных случаев обеспечивает весьма существенное ускорение.

11.4. Графы планирования Все предложенные выше эвристики для планирования с полным упорядочением и частичным упорядочением могут допускать неточности. В данном разделе показано, как можно воспользоваться специальной структурой данных, называемой " $\wedge$ графом планирования, для получения лучших эвристических оценок. Такие эвристики могут применяться в любом из методов поиска, рассмотренных нами до сих пор. Еще один вариант состоит в том, что решение может быть извлечено непосредственно из графа планирования с использованием специализированного алгоритма, например такого, который получил название Graphplan.

Граф планирования состоит из последовательности " $\wedge$ уровней, которые соответствуют временн $\ddot{y}$ м этапам в плане, где уровень 0 представляет собой начальное состояние. Каждый уровень содержит множество литералов и множество действий.

Грубо говоря, литералы представляют собой то, что может быть истинным на каждом временном этапе в зависимости от действий, выполненных на предыдущих временных этапах. Также грубо говоря, действиями являются все те действия, которые

могут иметь все свои предусловия выполненными на данном временном этапе в зависимости от того, какие из литералов фактически являются истинными. В этих двух определениях применяется выражение "грубо говоря", поскольку в графе планирования регистрируется только ограниченное подмножество возможных отрицательных взаимодействий между действиями; поэтому граф может давать оптимистическую оценку минимального количества временн $\ddot{y}$ х этапов, требуемых для того, чтобы некоторый литерал стал истинным. Тем не менее такое количество этапов в графе планирования предоставляет хорошую оценку того, насколько трудно достичь указанного литерала из начального состояния. Еще важнее то, что граф планирования определен таким образом, что может быть сформирован очень эффективно.

Графы планирования могут применяться только для решения задач планирования в пропозициональной логике, т.е. тех задач, в которых отсутствуют переменные.

Как упоминалось в разделе 11.1, в пропозициональную форму могут быть преобразованы и представления Strips, и представления ADL. Если в задаче имеется большое количество объектов, это может привести к весьма существенному возрастанию количества возможных схем действий. Несмотря на это, было

показано, что графы планирования представляют собой эффективное инструментальное средство решения трудных задач планирования.

Проиллюстрируем применение графов планирования на простом примере.

(Использование более сложных примеров привело бы к созданию графов, которые не поместились бы на одной странице книги.) В листинге 11.5 приведена задача, а на рис. 11.6 показан ее граф планирования. Начнем с уровня состояния s_0 , который представляет начальное состояние задачи. Затем перейдем на уровень действия A_0 и поместим на него все действия, предусловия которых были выполнены на предыдущем уровне. Каждое действие соединено с его предусловиями в состоянии S_0 и его результатами в состоянии B_g , а это в данном случае влечет за собой введение в $S_{\pm}$ новых литералов, которых не было в s_0 .

Листинг 11.5. Задача "получить кекс, а также съесть кекс"

Init(Have(Cake)) Goal(Have(Cake) а Eaten(Cake)) Action(Eat(Cake) Precond: Have(Cake) Effect: -^Have(Cake) а Eaten(Cake)) Action(Bake(Cake) Precond: —\Have(Cake) Effect: Have(Cake) Должны быть предусмотрены способы представлять на графе планирования не только действие, но и бездействие. Это означает, что необходимо иметь своего рода эквивалент аксиом окружения ситуационного исчисления, которые позволяют литералу оставаться истинным от одной ситуации к другой, если его не изменяет ни одно действие. В графе планирования это осуществляется с помощью множества $\wedge$ сохраняющих действий (persistence action). Для каждого положительного или отрицательного литерала C в задачу вводится сохраняющее действие с предусловием C и результатом C . На рис. 11.6 в состоянии A_0 показано одно "реальное" действие,

Eat {Cake} (съесть кекс), наряду с двумя сохраняющими действиями, обозначенными с помощью небольших квадратов.

Sq Aq S\ Aj \$2 Have(Cake) -OEat(Cake) Have(Cake) i Have(Cake) X A Bake(Cake) K^ Have(Cake) -i Have(Cake) Чу u —pr -i Have(Lake) Л Ea1(CakeTP<Г^ D —^ Eaten(Cake) П _ FntPn/Cnlfp) • -, Eaten(Cake) Eaten(Cake) !^Eaten(Cake) ? ^Eaten(Cake) Рис. 11.6. Граф планирования для задачи "получить кекс, а также съесть кекс" вплоть до уровня S_2 Прямоугольники обозначают действия (маленькие квадраты обозначают сохраняющие действия), а прямые линии указывают на предусловия и результаты. Взаимно исключающие связи обозначены кривыми серыми линиями Уровень A_0 содержит все действия, которые могут произойти в состоянии S_0 , но столь же важно то, что он показывает конфликты между действиями, способные воспрепятствовать тому, чтобы эти действия выполнялись вместе. Серыми линиями на рис. 11.6 показаны $\wedge$ взаимно исключающие связи (или связи, $\wedge$ характеризующиеся взаимным исключением — mutual exclusive, mutex). Например, действие Ea t {Cake} является взаимно исключающим с действием, обеспечивающим сохранение

состояния, вызванного действием Have {Cake} или —iEa ten {Cake}. Вскоре будет показано, как происходит вычисление взаимно исключающих связей.

На уровне S_i находятся все литералы, которые могут стать результатом принятия к исполнению любого подмножества действий уровня A_0 . На этом уровне показаны также взаимно исключающие связи (серые линии), обозначающие литералы, которые не могут появляться вместе, независимо от выбора действий. Например, взаимно исключающими являются литералы Have {Cake} и Eaten {Cake}: в зависимости от выбора действий на уровне A_0 результатом может стать один из них или другой, но не оба. Иными словами, на уровне S_i представлено несколько состояний, точно так же, как и при регрессивном поиске в пространстве состояний, а взаимно исключающие связи представляют собой ограничения, которые определяют множество возможных состояний.

Построение плана продолжается таким же образом, с чередованием между уровнями состояния S_i и уровнями действия A_i до тех пор, пока не будет достигнут уровень, на котором два последовательных уровня становятся идентичными. После достижения такой точки граф называется "выровненным" (leveled off). Каждый последующий уровень будет оставаться неизменным, поэтому дальнейшее развертывание не имеет смысла.

В конечном итоге образуется структура, в которой каждый уровень A_i содержит все действия, применимые на уровне S_{i+1} наряду с ограничениями, указывающими, какие пары действий не могут выполняться одновременно. Каждый уровень S_i содержит все литералы, которые могут стать результатом любого возможного выбора действий на уровне A_i , наряду с ограничениями, указывающими, какие пары литералов недопустимы. Важно отметить, что процесс построения графа планирования не требует выбора среди действий, который потребовал бы комбинаторного поиска. Вместо этого в

графе просто регистрируется невозможность осуществления некоторых выборов с использованием взаимно исключающих связей. Сложность формирования графа планирования оценивается полиномиальной зависимостью низкого порядка от количества действий и литералов, тогда как размеры пространства состояний определяются экспоненциальной зависимостью от количества литералов.

Теперь определим взаимно исключающие связи как для действий, так и для литералов. Между двумя действиями на данном конкретном уровне имеет место взаимно исключающее отношение, если соблюдается любое из трех перечисленных ниже условий. • Несогласованные результаты. Одно действие отрицает результат другого.

Например, действие Eat {Cake} и сохраняющее действие Have {Cake} имеют несогласованные результаты, поскольку они не согласуются в результате Have {Cake}

(съедение кекса приводит к его исчезновению). • Вмешательство. Один из результатов одного действия является отрицанием предусловия другого действия. Например, действие Eat (Cake) мешает осуществлению сохраняющего действия Have (Cake), поскольку отрицает тво предусловие (если кекса больше нет, то нечего и хранить). • Конкурирующие потребности. Одно из предусловий одного действия является взаимно исключающим по отношению к предусловию другого. Например, литералы Bake (Cake) (Испечь кекс) и Eat (Cake) (Съесть кекс) являются взаимно исключающими, поскольку конкурируют за значение предусловия Have (Cake) (в одном литерале кекс создается, а в другом — уничтожается).

Взаимно исключающее отношение имеет место между двумя литералами на одном и том же уровне, если один из них является отрицанием другого или если каждая возможная пара действий, которые позволяют достичь двух литералов, является взаимно исключающей. Такое условие называется несогласованной поддержкой.

Например, на уровне S1 литералы Have (Cake) и Eaten (Cake) являются взаимно исключающими, поскольку единственный способ достичь литерала Have (Cake) (применения сохраняющего действия) является взаимно исключающим с единственным способом достижения литерала Eaten (Cake), а именно Eat (Cake).

На уровне S2 эти два литерала не являются взаимно исключающими, поскольку существуют новые способы их достижения, такие как действие Bake (Cake) и сохраняющее действие Eaten (Cake), которые не являются взаимно исключающими.

Применение графов планирования для получения эвристической оценки Граф планирования после его построения становится богатым источником информации о задаче. Например, очевидно, что s^* литерал, который не появляется на заключительном уровне графа, не может быть достигнут с помощью любого плана.

Такое наблюдение может использоваться в обратном поиске следующим образом: любое состояние, содержащее недостижимый литерал, имеет стоимость $h(n) = \infty$.

Аналогичным образом, при планировании с частичным упорядочением любой план с недостижимым открытым условием имеет $p(n) = \infty$.

Эту идею можно сделать более обобщенной. Стоимость достижения любого целевого литерала может оцениваться как номер уровня, на котором он впервые появ-

ляется в графе планирования. Назовем эту оценку h^* уровневой стоимостью (level cost) цели. На рис. 11.6 литерал Have (Cake) имеет уровневую стоимость 0, а Eaten (Cake) — уровневую стоимость 1. Можно легко показать (упр. 11.9), что эти оценки являются допустимыми для отдельных целей. Однако сами эти оценки могут оказаться не очень качественными, поскольку графы планирования допускают наличие несколько действий на каждом уровне, тогда как в этой эвристике

учитывается только номер уровня, а не количество действий. По этой причине для вычисления эвристики принято использовать $\wedge$ последовательный граф планирования (serial planning graph). Последовательный граф требует, чтобы на каждом конкретном временном этапе фактически могло происходить только одно действие; такое требование осуществляется путем введения взаимно исключающих связей между каждой парой действий, кроме сохраняющих действий. Уровневые стоимости, извлеченные из последовательных графов, часто представляют собой вполне приемлемые оценки фактических стоимостей.

Чтобы оценивать стоимость достижения конъюнкции целей, можно воспользоваться одним из трех простых подходов. В эвристике $\wedge$ максимального уровня (max-level) просто берется максимальная уровневая стоимость любой из целей; такая эвристика является допустимой, но не обязательно очень точной. Эвристика $\wedge$ уровневой суммы (level sum), в основе которой лежит предположение о независимости подцелей, возвращает сумму уровневых стоимостей целей; эта эвристика является недопустимой, но очень хорошо действует на практике при решении задач, которые являются в значительной степени декомпозируемыми. Она характеризуется гораздо более высокой точностью по сравнению с эвристикой, в которой учитывается количество невыполненных целей, описанной в разделе 11.2. В рассматриваемой задаче эвристическая оценка для конъюнктивной цели Have (Cake) $\wedge$ Eaten(Cake) будет равна $0 + 1 = 1$, тогда как правильный ответ равен 2. Кроме того, если будет удалено действие Bake (Cake), эта оценка по-прежнему будет равна 1, но достижение этой конъюнктивной цели станет невозможной. Наконец, эвристика $\wedge$ множественного уровня (set-level) находит уровень, на котором все литералы в конъюнктивной цели появляются в графе планирования без любой пары из них, которая была бы взаимно исключающей. Эта эвристика дает правильное значение, равное 2, для первоначальной задачи и равное бесконечности для задачи без действия Bake (Cake). Она доминирует над эвристикой максимального уровня и действует чрезвычайно успешно в задачах, характеризующихся весьма существенным взаимодействием субпланов.

Для использования его в качестве инструментального средства формирования точных эвристик граф планирования можно рассматривать как ослабленную задачу, которая может быть эффективно решена. Чтобы понять характер этой ослабленной задачи, нужно точно определить, что означает появление литерала d на уровне S_i в графе планирования. В идеале желательно было бы иметь гарантию, что существует план с i уровнями действий, который достигает литерала d , а также, что литерал d не появится, если такого плана нет. К сожалению, предоставить такую гарантию столь же трудно, как и решить первоначальную задачу планирования. Поэтому граф планирования предоставляет вторую половину гарантии (если литерал d не появляется, то нет и плана его достижения), но если литерал d

появляется, то весь этот граф планирования становится залогом того, что существует план, который, возможно, позволяет достичь литерала d и не имеет "очевидных" недостатков. Очевид-

ный недостаток плана определяется как недостаток, который может быть выявлен путем рассмотрения двух действий или двух литералов одновременно; другими словами, путем проверки взаимно исключающих отношений. Могут существовать более трудно диагностируемые недостатки, охватывающие три, четыре или больше действий, но опыт показывает, что вычислительные затраты, связанные с отслеживанием этих возможных недостатков, не оправдываются. Этот вывод аналогичен уроку, усвоенному по результатам исследования задач удовлетворения ограничений, в которых часто целесообразно вычислить 2-совместимость (совместимость на уровне 2) перед поиском решения, но вычисление 3-совместимости или совместимости более высокой степени часто бывает менее целесообразным (см. раздел 5.2).

Алгоритм Graphplan В данном подразделе показано, как извлечь план непосредственно из графа планирования, а не просто использовать этот граф для получения эвристики. Алгоритм Graphplan (листинг 11.6) имеет два основных этапа, каждый из которых чередуется в цикле. Прежде всего в этом алгоритме проверяется, присутствуют ли все целевые литералы на текущем уровне без взаимно исключающих связей между любой парой из них. Если это требование соблюдается, то в текущем графе может существовать решение, поэтому в алгоритме выполняется попытка извлечь это решение. В противном случае граф расширяется путем добавления действий для текущего уровня и литералов состояния для следующего уровня. Процесс продолжается до тех пор, пока либо не обнаруживается решение, либо не выясняется, что решения не существует.

Листинг 11.6. Алгоритм Graphplan. В алгоритме Graphplan чередуются этап извлечения решения и этап расширения графа. Функция Extract-Solution определяет, может ли быть найден план, начиная с конца и выполняя поиск в обратном направлении. Функция Expand-Graph добавляет действия для текущего уровня и литералы состояния для следующего уровня

```
function Graphplan (problem) returns
  решение solution
  vlhvl индикатор неудачи failure
  graph <rInitial-Planning-Graph
  (problem) goals <rGoals [problem]
  loop do if все цели goals не являются взаимно
    исключающими на последнем уровне графа graph
    then do solution <— Extract-
      Solution (graph, goals, Length (graph) )
    if solution  $\Phi$  failure then return solution
    else if No-Solution-Possible(graph) then
      return failure
  graph <rExpand-Graph (graph, problem)
```

Теперь проследим за функционированием алгоритма Graphplan на примере задачи замены колеса, рассматриваемой в разделе 11.1. Полный граф показан на рис.

11.7. В первой строке алгоритма Graphplan граф планирования инициализируется значением одноуровневого графа (s_0), состоящего из пяти литералов, взятых из

начального состояния. Целевой литерал $At(Spare, Axle)$ в состоянии S_0 отсутствует, поэтому не требуется вызывать функцию $Extract-Solution$ — мы можем быть уверены, что решение еще не существует. Вместо этого вызывается функция

$Expand-Graph$, добавляющая три действия, предусловия которых существуют на уровне S_0 (т.е. все действия, за исключением $PutOn(Spare, Axle)$), наряду с сохраняющими действиями для всех литералов в S_0 . Результаты действий добавляются на уровне $S_{\pm}$. Затем функция $Expand-Graph$ отыскивает взаимно исключающие отношения и добавляет их к графу.

$At(Spare, Trunk) \wedge \neg At(Flat, Axle) \wedge Remove(Spare, Trunk) \wedge RsmnYSf \wedge FlaLM \wedge I \wedge D \wedge 1$
 $UavsQYernifiiiL \wedge \neg At(Spare, Axle) \wedge \neg At(Spare, Trunk) \wedge \neg At(Flat, Ade) \wedge \neg$
 $At(Flat, Axle) \wedge \neg At(Spare, Axle) \wedge \neg RemQvs(Sparg, Trunk) \wedge I \wedge RsmovglFlauAM \wedge D$
 $LsacvQvsmifht \wedge PutOn(Spare, Axle) \wedge D \wedge 2 \wedge At(Spare, Trunk) \wedge \neg At(Spare, Trunk) \wedge At(Flat, Axle)$
 $\wedge At(Flat, Axle) \rightarrow At(Spare, Axle) \wedge At(Spare, Axle) \wedge At(Flat, Ground) \wedge \neg At(Flat, Ground)$
 $\wedge B \wedge \neg \neg \neg \neg \neg \neg At(Flat, Ground) \wedge At(Flat, Ground) \rightarrow f \wedge \neg \neg \neg \neg \neg \neg At(Flat, Ground) \wedge \neg$
 $At(Spare, Ground) \wedge D \wedge f \wedge \neg \neg \neg \neg \neg \neg At(Spare, Ground) \wedge At(Spare, Ground) \wedge$
 $D \wedge At(Spare, Ground)$ Рис. 11.7. Граф планирования для задачи замены колеса после расширения до уровня S_2 . Взаимно исключающие связи показаны серыми линиями. Показаны только некоторые наиболее важные взаимно исключающие связи, поскольку граф был бы слишком загроможден, если бы были показаны все эти связи. Решение обозначено жирными линиями и жирными контурами прямоугольников B

в состоянии S_i литерал $At(Spare, Axle)$ все еще не присутствует, поэтому функция $Extract-Solution$ снова не вызывается. Вызов функции $ExpandGraph$ приводит к получению графа планирования, показанного на рис. 11.7. Теперь, после получения полного комплекта действий, целесообразно рассмотреть некоторые примеры взаимно исключающих отношений и их причин, как показано ниже.

- Несогласованные результаты. Литерал $Remove(Spare, Trunk)$ является взаимно исключающим по отношению к $LeaveOvernight$, поскольку один из них имеет своим результатом литерал $At(Spare, Ground)$, а другой — его отрицание.
- Вмешательство. Литерал $Remove(Flat, Axle)$ является взаимно исключающим по отношению к $LeaveOvernight$, поскольку один из них имеет своим предусловием $At(Flat, Axle)$, а другой имеет своим результатом его отрицание.
- Конкурирующие потребности. Литерал $PutOn(Spare, Axle)$ является взаимно исключающим по отношению к $Remove(Flat, Axle)$, поскольку один из них имеет в качестве предусловия литерал $At(Flat, Axle)$, а другой — его отрицание.
- Несогласованная поддержка. Литерал $At(Spare, Axle)$ является взаимно исключающим по отношению к $At(Flat, Axle)$ на уровне S_2 , поскольку единственным способом достижения цели $At(Spare, Axle)$ является выполнение действия $PutOn(Spare, Axle)$, а оно является взаимно исключающим с сохраняющим действием, которое представляет собой

единственный способ достижения литерала $At(Flat, Axle)$. Таким образом, взаимно исключаю-

щие отношения позволяют обнаруживать непосредственные конфликты, которые становятся следствием попыток поместить два объекта в одно и то же место одновременно.

После того как мы снова перейдем в начало цикла, на этот раз на уровне S_2 будут присутствовать все литералы из цели и ни один из них не будет взаимно исключающим по отношению к любому другому. Это означает, что может существовать решение и в функции Extract-Solution будет предпринята попытка его найти. По сути, функция Extract-Solution решает булеву задачу CSP, переменными которой являются действия на каждом уровне, а значениями для каждой переменной служат индикаторы, показывающие, принадлежит ли она или не принадлежит к плану. Для этого можно воспользоваться стандартными алгоритмами CSP или определить функцию Extract-Solution как задачу поиска, в которой каждое состояние в поиске содержит указатель на уровень в графе планирования и множество невыполненных целей. Определим эту задачу поиска, как описано ниже.

- Первоначальным состоянием является последний уровень графа планирования, S_n , наряду с множеством целей из задачи планирования.
- Действия, доступные в любом состоянии на уровне s_L , должны выбирать любое бесконфликтное подмножество действий на уровне AL_U результаты которых покрывают цели в этом состоянии. Результирующее состояние имеет уровень S_{i+1} и включает в качестве своего множества целей все предусловия для выбранного множества действий. Под термином "бесконфликтный" подразумевается множество таких действий, что никакие два из них не являются взаимно исключающими и никакие два из их предусловий не являются взаимно исключающими.
- Задача состоит в том, чтобы достичь на уровне S_0 такого состояния, чтобы все цели были выполнены.
- Стоимость каждого действия равна 1.

При решении данной конкретной задачи начнем с уровня s_2 , где имеется цель $At(Spare, Axle)$. Единственным вариантом, который может применяться для достижения этого множества целей, является $PutOn(Spare, Axle)$. В результате мы переходим в состояние поиска на уровне S_1 с целями $At(Spare, Ground)$ и $\neg Afc(Flat, Axle)$. Первой цели можно достичь только с помощью действия $Remove(Spare, Trunk)$, а последней — с помощью либо $Remove(Flat, Axle)$, либо $LeaveOvernight$. Но действие $LeaveOvernight$ является взаимно исключающим по отношению к $Remove(Spare, Trunk)$, поэтому единственное решение состоит в том, чтобы выбрать $Remove(Spare, Trunk)$ и $Remove(Flat, Axle)$. В результате мы переходим в состояние поиска на уровне S_0 с целями $At(Spare, Trunk)$ и $At(Flat, Axle)$. Обе из этих целей присутствуют в данном состоянии, поэтому налицо готовое решение: действия $Remove(Spare, Trunk)$ и

Remove {Flat, Axle) на уровне n , за которыми следует действие PutOn {Spare, Axle) на уровне $n+1$.

Известно, что задача планирования является PSPACE-полной и что для построения графа планирования требуется полиномиальное время, поэтому в наихудшем случае может возникнуть ситуация, в которой извлечение решения окажется неосуществимым. Таким образом, потребуется определенное эвристическое руководство

при выборе среди действий в ходе обратного поиска. Одним из подходов, хорошо зарекомендовавших себя на практике, является жадный алгоритм, основанный на учете уровневой стоимости литералов. Для каждого набора целей применение этой эвристики осуществляется в описанном ниже порядке.

1. Определить литерал с максимальной уровневой стоимостью.
2. Чтобы достичь этого литерала, выбрать в первую очередь действие с самыми легкими для выполнения предусловиями. Это означает, что действие нужно выбирать так, чтобы сумма (или максимум) уровневых стоимостей его предусловий была минимальной.

Завершение работы алгоритма Graphplan До сих пор мы обходили проблему завершения работы алгоритма. Если задача не имеет решения, можно ли быть уверенным в том, что алгоритм Graphplan не будет проходить по циклу до бесконечности, расширяя граф планирования при каждой итерации? Ответ на этот вопрос является положительным, но полное доказательство такого утверждения выходит за рамки настоящей книги. Здесь мы кратко опишем основные идеи, особенно те, которые проливают свет на графы планирования в целом.

На первом этапе необходимо отметить, что характеристики некоторых свойств графов планирования монотонно увеличиваются или уменьшаются. Выражение "характеристика X увеличивается монотонно" означает, что множество экземпляров X на уровне $i + 1$ является надмножеством (необязательно строгим) множества на уровне i . Соответствующие свойства графов перечислены ниже.

- Количество литералов увеличивается монотонно. После того как некоторый литерал появляется на данном конкретном уровне, он будет появляться на всех последующих уровнях. Это связано с наличием сохраняющих действий; после того как литерал появляется, сохраняющие действия вызывают его сохранение навечно.
- Количество действий увеличивается монотонно. После того как некоторое действие появляется на данном конкретном уровне, оно будет появляться на всех последующих уровнях. Это — следствие увеличения количества литералов: если предусловия некоторого действия появляются на одном уровне, они будут появляться и на последующих уровнях и поэтому то же происходит и с действиями.
- Количество взаимно исключающих связей уменьшается монотонно. Если два действия на данном

конкретном уровне AL являются взаимно исключающими, то они должны также быть взаимно исключающими на всех предыдущих уровнях, на которых они появлялись вместе. То же утверждение остается справедливым и по отношению к взаимно исключающим связям между литералами. Это не означает, что взаимно исключающие связи характеризуются такими же особенностями и на рисунках с изображением графов планирования, поскольку на рисунках допускаются упрощения: на них не показывают ни литералов, которые не могут быть истинными на уровне $Si5$ ни действий, которые не могут быть выполнены на уровне Ai . Можно убедиться в справедливости утверждения, что "количество взаимно исключающих связей умень-

шается монотонно", если учесть, что эти невидимые литералы и действия являются взаимно исключающими по отношению ко всем прочим.

Доказательство этих утверждений является довольно сложным, но может быть выполнено путем анализа отдельных случаев: если действия A и B являются взаимно исключающими на уровне a то должны быть таковыми из-за наличия одного из трех типов взаимно исключающих связей. Первые два из них, несогласованные результаты и вмешательство, обусловлены свойствами самих действий, поэтому, если действия являются взаимно исключающими на уровне Ai ? то будут взаимно исключающими на каждом уровне. Третий случай, с конкурирующими потребностями, зависит от условий на уровне SL : этот уровень должен содержать предусловие действия A , которое является взаимно исключающим по отношению к предусловию действия B . Теперь отметим, что эти два предусловия могут быть взаимно исключающими, если они являются отрицаниями друг друга (и в этом случае они будут взаимно исключающими на каждом уровне) или все действия по достижении одного из них являются взаимно исключающими по отношению ко всем действиям, необходимым для достижения другого. Но мы уже знаем, что количество допустимых действий увеличивается монотонно, поэтому по индукции количество взаимно исключающих связей должно уменьшаться.

Поскольку количество действий и литералов увеличивается, а количество взаимно исключающих связей уменьшается, и поскольку имеется только конечное количество действий и литералов, каждый граф планирования в конечном итоге выравнивается — все последующие уровни в нем становятся идентичными. После того как граф выровнялся, он может быть проанализирован, и если в нем отсутствует одна из целей задачи или две цели являются взаимно исключающими, то данная задача никогда не может быть решена, поэтому мы можем остановить работу алгоритма Graphplan и вернуть индикатор неудачи. Если же граф выравнивается со всеми присутствующими и не взаимно исключающими целями, но функция ExtractSolution оказывается не в состоянии найти решения, то может потребоваться снова расширить граф конечное количество раз, но так или иначе мы имеем

возможность остановить работу алгоритма. Последний вариант завершения является более сложным и здесь не рассматривается.

11.5. Планирование с помощью ПРОПОЗИЦИОНАЛЬНОЙ логики Как было описано в главе 10, планирование может осуществляться по принципу доказательства некоторой теоремы в рамках ситуационного исчисления. В подобной теореме утверждается, что при наличии начального состояния и аксиом состояния-преемника, которые описывают результаты действий, цель будет истинной в ситуации, которая является результатом некоторой последовательности действий. В такой ранний период развития искусственного интеллекта, как 1969 год, данный подход считался слишком неэффективным для того, чтобы с его помощью можно было находить интересные планы. Проведенные в последнее время разработки в области эффективных алгоритмов формирования рассуждений для пропозициональной ло-

гики (см. главу 7) привели к возрождению интереса к планированию с помощью логических рассуждений.

Подход, принятый в данном разделе, основан на проверке выполнимости логического высказывания, а не на доказательстве теорем. Мы будем отыскивать модели для пропозициональных высказываний, которые выглядят примерно так:

Начальное состояние $\wedge$ Все возможные описания действий $\wedge$ Цель Такое высказывание должно содержать пропозициональные символы, соответствующие каждому возможному проявлению действия; модель, в которой выполняется это высказывание, должна присваивать значение true действиям, являющимся частью правильного плана, и false — другим действиям. Присваивание, которое соответствует неправильному плану, не будет моделью, поскольку окажется несовместимым с утверждением, что цель истинна. Если задача планирования неразрешима, то данное высказывание будет невыполнимым.

Описание задач планирования в пропозициональной логике Процесс, который будет здесь применяться для перевода задач Strips на язык пропозициональной логики, представляет собой (так сказать) классический пример цикла представления знаний: мы начнем с множества аксиом, которое на первый взгляд кажется приемлемым, обнаружим, что эти аксиомы допускают появление фиктивных, не предусмотренных моделей, а затем запишем дополнительные аксиомы.

Начнем с очень простой задачи воздушной транспортировки. В начальном состоянии (время 0) самолет P₁ находится в аэропорту SFO, а самолет P₂ — в аэропорту JFK. Задача состоит в том, чтобы P₁ находился в JFK, а P₂ — в SFO\ иными словами, самолеты должны поменяться местами. Прежде всего, потребуются отдельные пропозициональные символы для формирования утверждений о каждом временном этапе. Для обозначения временного этапа будут использоваться верхние

индексы, как и в главе 7. Поэтому начальное состояние можно записать следующим образом:

$At(PltSFO)^\circ \wedge At(P2,JFK)^\circ$ (Напомним, что $At(Plf SFO)^\circ$ — это атомарный символ.) Поскольку в пропозициональной логике не принято предположение о замкнутом мире, необходимо также определить высказывания, которые не являются истинными в начальном состоянии. Если же некоторые высказывания в начальном состоянии не известны, они могут оставаться неопределенными (предположение об открытом мире). В данном примере зададим следующее: $\neg At(PltJFK)^\circ \wedge \neg At(P2,SFO)^\circ$. Сама цель должна быть связана с конкретным временным этапом. Поскольку мы не знаем заранее, какое количество этапов потребуется для достижения цели, то можно попытаться сформулировать утверждение, что цель истинна в начальном состоянии, во время $t=0$. Это означает, что мы вводим утверждение $At(P1, JFK)^\circ \wedge At(P2, SFO)^\circ$. Если эта попытка окажется неудачной, повторим ее снова во время $t=1$ и т.д. до тех пор, пока не достигнем осуществимого плана с минимальной длиной. Для каждого значения t база знаний будет включать только высказывания, покрывающие временн/е этапы от 0 вплоть до t . Чтобы обеспечить завершение работы алгоритма, можно наложить произвольный верхний предел,

Шах Этот алгоритм приведен в листинге 11.7. Альтернативный подход, позволяющий избежать использования многочисленных попыток решения, обсуждается в упр. 11.17.

Листинг 11.7. Алгоритм SATplan. Задача планирования преобразуется в высказывание в форме CNF, в котором подтверждается истинность цели на фиксированном временном этапе t , а аксиомы добавляются на каждом временном этапе вплоть до t . (Подробные сведения о преобразовании приведены в тексте главы.) Если алгоритм проверки выполнимости находит модель, то план извлекается путем поиска тех пропозициональных символов, которые относятся к действиям и которым в модели присвоено значение true. Если модель не существует, то процесс повторяется после передвижения цели на один этап дальше function SATplan(problem, Tmax) returns решение solution или индикатор неудачи failure inputs: problem, задача планирования Tmax, верхний предел длины плана for $\Gamma = 0$ to Tmax do cnf, mapping $\leftarrow$ Translate-To-SAT {problem, T} assignment $\leftarrow$ SAT-Solver {cnf} if присваивание assignment не является неопределенным then return Extract-Solution{assignment, mapping} return failure Следующая проблема состоит в том, как закодировать описания действий в пропозициональной логике. Наиболее прямолинейный подход состоит в том, чтобы было предусмотрено по одному пропозициональному символу на каждое проявление действия; например, символ $Fly(Plf SFO, JFK)^\circ$ является истинным, если самолет P1 вылетает из аэропорта SFO в аэропорт JFK во время 0. Как и в главе 7, мы запишем пропозициональные версии

аксиом состояния-преемника, разработанных для ситуационного исчисления в главе 10. Например, имеет место следующее:

$At\{Plt\ JFKI \leq \} (At\ (-Pi, JFKH \wedge \{Fly\{PIIJFK,SFO\}^\circ \wedge At\ (Pi, JFK)^\circ \}) \vee \{Fly\{PIISFO, JFKH \wedge At\{P1,SFO\}^\circ\} A1.1)$ Это означает, что самолет $p1$ должен находиться в аэропорту JFK во время 1, если он был в JFK во время 0 и не улетел или если он был в аэропорту SFO во время 0 и прилетел в аэропорт JFK. Необходимо иметь по одной такой аксиоме для каждого самолета, аэропорта и временного этапа. Более того, каждый дополнительный аэропорт добавляет еще один способ путешествия из каждого конкретного аэропорта или в этот аэропорт и поэтому вносит еще больше дизъюнктов в правую часть каждой аксиомы.

После подготовки всех этих аксиом можно вызвать алгоритм проверки выполнимости, чтобы найти план. Это должен быть план, который достигает цели во время $t=1$, а именно план, в котором эти два самолета меняются местами. Теперь предположим, что база знаний представляет собой следующее высказывание:

Начальное состояние $\wedge$ Аксиомы состояния-преемника $\wedge$ Цель1 A1.2) в котором утверждается, что цель истинна во время $t=1$. Читатель может проверить, что утверждение, в котором символы $Fly\{Pr, SFO, JFK\}^\circ$ и $Fly\{P2, JFK, SFO\}^\circ$

являются истинными, а все другие символы действий являются ложными, представляет собой модель этой базы знаний. До сих пор все идет хорошо. Но есть ли другие возможные модели, которые способен вернуть этот алгоритм проверки выполнимости? Безусловно, да. Являются ли все эти другие модели удовлетворительными планами? К сожалению, нет. Рассмотрим довольно глупый план, заданный с помощью указанных символов действий:

$Fly\{PlfSFO,JFK\}^\circ$ и $Fly\{Px.JFK.SFOH \wedge Fly\{P2IJFK,SFO\}^\circ$ Этот план глуп потому, что самолет pr вначале находится в аэропорту SFO, поэтому действие $Fly\{Plt\ JFK, SFO\}^\circ$ является неосуществимым. Тем не менее этот план действительно представляет собой модель высказывания из уравнения 11.2!

Иными словами, он согласуется со всем, что было сказано до сих пор об этой задаче.

Чтобы понять, чем может быть вызвано появление такого плана, мы должны проанализировать более тщательно, что сказано в аксиомах состояния-преемника (таких как уравнение 11.1) о действиях, предусловия которых не выполнены.

Аксиомы правильно предсказывают, что ничего не произойдет при попытке выполнить подобное действие (см. упр. 11.15), но в них ничего не сказано, что такое действие не может быть выполнено! Чтобы предотвратить выработку планов с недопустимыми действиями, мы должны добавить $\wedge$ аксиомы предусловий (precondition axiom), которые указывают, что для совершения любого действия

требуется, чтобы были выполнены его предусловия⁶. Например, требуется указать следующее:

$\text{Fly}(\text{P1JFK}, \text{SFO})^\circ \Rightarrow \text{At}(\text{P1fJFK})^\circ$ Поскольку указано, что в начальном состоянии символ $\text{At}(\text{P1fJFK})^\circ$ является ложным, эта аксиома гарантирует, что в каждой модели символу $\text{Fly}(\text{P1fJFK}, \text{SFO})^\circ$ также будет присвоено ложное значение. После введения аксиом предусловия существует одна и только одна модель, которая удовлетворяет всем аксиомам, если цель должна быть достигнута во время 1, а именно модель, в которой самолет P₁ летит в аэропорт JFK, а самолет P₂ — в аэропорт SFO. Обратите внимание на то, что это решение обеспечивает выполнение двух параллельных действий, как и при использовании алгоритма Graphplan или POP.

Неожиданности возникают после добавления третьего аэропорта, LAX. Теперь для каждого самолета имеются два действия, которые являются допустимыми в каждом состоянии. После применения алгоритма проверки выполнимости мы обнаружим, что модель с символами $\text{Fly}(\text{P1f SFO}, \text{JFK})$, а также $\text{Fly}(\text{P2}, \text{JFK}, \text{SFO})$ и $\text{Fly}(\text{P2}, \text{JFK}, \text{LAX})^\circ$ удовлетворяет всем аксиомам. Это означает, что аксиомы состояния-преемника и аксиомы предусловия разрешают самолету вылететь в два аэропорта назначения одновременно! Предусловия для двух полетов самолета P₂ выполнены в начальном состоянии; аксиомы состояния-преемника указывают, что самолет P₂ должен находиться и в аэропорту SFO, и в аэропорту LAX во время 1, поэтому цель выполнена.

Очевидно, что необходимо ввести дополнительные аксиомы для устранения таких фиктивных решений. Один из подходов состоит в том, что требуется ввести $\wedge$ аксиомы исключения действий (action exclusion axiom), которые предотвращают одновременное выполнение несовместимых действий. Например, можно принудительно ввести полное исключение, добавив все возможные аксиомы в такой форме:

⁶ Обратите внимание на то, что добавление аксиом предусловий означает, что не нужно будет включать предусловия для действий в аксиомы состояния-преемника.

$\wedge(\text{Fly}(\text{P2}, \text{JFK}, \text{SFO})^\circ \wedge \text{Fly}(\text{P2}, \text{JFK}, \text{LAX})^\circ)$ Эти аксиомы гарантируют, что никакие два действия не могут происходить одновременно. Они устраняют все фиктивные планы, но приведут также к полному упорядочению каждого плана. В результате гибкость частично упорядоченных планов будет потеряна; кроме того, в результате увеличения количества временных этапов в плане может увеличиться продолжительность вычислений.

Вместо полного исключения можно потребовать только частичного исключения, т.е. предотвращения одновременных действий, только если они мешают друг другу.

Применяемые при этом условия являются аналогичными условиям для взаимно исключающих действий: два действия не могут происходить одновременно, если одно из них отрицает предусловие или результат другого. Например, не могут происходить одновременно действия $\text{Fly}(P2, \text{JFK}, \text{SFO})$ и $\text{Fly}(P2, \text{JFK}, \text{LAX})$, поскольку каждое из них отрицает предусловие другого; с другой стороны, действия $\text{Fly}(P1, \text{SFO}, \text{JFK})$ и $\text{Fly}(P2, \text{JFK}, \text{SFO})$ могут осуществляться одновременно, поскольку в них два самолета не мешают друг другу. Частичное исключение позволяет устранять фиктивные планы, не вводя принудительно полное упорядочение.

Аксиомы исключения иногда выглядят как довольно слепой инструмент.

Поэтому вместо формирования утверждения о том, что ни один самолет не может вылетать в два аэропорта одновременно, можно просто выдвинуть требование, чтобы ни один объект не мог находиться в двух местах одновременно:

$\forall p, x, y, t \in \Phi \quad y \Rightarrow \neg (\text{At}(p/x)t \wedge \text{At}(p/y) t)$ Из этого факта, применяемого в сочетании с аксиомами состояния-преемника, следует, что самолет не может вылететь в два аэропорта одновременно. Факты, подобные этому, называются $\wedge$ ограничениями состояния (state constraint). Безусловно, в пропозициональной логике необходимо будет записать все базовые экземпляры каждого ограничения состояния. Для задачи с аэропортами этого ограничения состояния будет достаточно, чтобы исключить все фиктивные планы. Ограничения состояния часто являются гораздо более компактными по сравнению с аксиомами исключения действия, но их не всегда можно легко вывести из первоначального описания задачи на языке Strips.

Подводя итог, отметим, что планирование в форме доказательства выполнимости предусматривает поиск моделей для высказывания, включающего описание начального состояния, цели, аксиом состояния-преемника, аксиом предусловий, а также либо аксиом исключения действия, либо аксиом ограничения состояния. Можно показать, что эта коллекция аксиом является достаточной, в том смысле, что при их использовании больше не возникают какие-либо фиктивные "решения". Любая модель, в которой выполняется такое пропозициональное высказывание, будет представлять собой действительный план для первоначальной задачи; это означает, что любая линеаризация этого плана будет представлять собой допустимую последовательность действий, которая позволяет достичь цели.

Сложности, связанные с использованием пропозициональных кодировок Основным недостатком описанного пропозиционального подхода являются колоссальные размеры пропозициональной базы знаний, которая формируется на основе первоначальной задачи планирования. Например, схема действий $\text{Fly}(p, \text{alf } a2)$ преобразуется в $T \times | \text{Planes} | \times | \text{Airports} |^2$ различных пропозициональных символов. Вообще говоря, общее количество символов действий ограничено значением $t \times | \text{Act} | \times | O |^p$, где $| \text{Act} |$ — количество схем действий; $| O |$ — количество объектов в

проблемной области; P — максимальная арность (количество параметров) любой схемы действий. Количество выражений еще больше. Например, при 10 временных этапах, 12 самолетах и 30 аэропортах полная аксиома исключения действий состоит из 583 миллионов выражений.

Поскольку количество символов действий экспоненциально зависит от арности схемы действий, одним из способов преодоления указанного недостатка может оказаться попытка уменьшить арность. Это можно сделать, заимствовав одну идею из области семантических сетей (см. главу 10). В семантических сетях используются только бинарные предикаты; предикаты с большим количеством параметров сводятся к множеству бинарных предикатов, которые описывают каждый параметр отдельно. Применяя эту идею к символу действий, такому как $\text{Fly}\{\text{Plf SFO, JFKH}$, получим следующие три новых символа:

$\text{Flyi}(\text{PiH}$: самолет P_i прилетел во время 0 $\text{Fly2}\{\text{SFO}\}^\circ$: аэропортом отправления в этом полете был SFO $\text{Flyi}\{\text{JFK}\}\text{Q}$: аэропортом назначения в этом полете был JFK Этот процесс, называемый $\wedge$ расщеплением символов (symbol splitting), позволяет устранить необходимость в использовании экспоненциального количества символов. Теперь требуется только $t \times | \text{Act} | \times p \times |o|$ символов.

Расщепление символов само по себе позволяет сократить количество символов, но не приводит к автоматическому уменьшению количества аксиом в базе знаний.

Это означает, что если бы каждый символ действия в каждом выражении был просто заменен конъюнкцией трех символов, то общий размер базы знаний остался бы примерно тем же самым. Расщепление символов фактически приводит к уменьшению базы знаний потому, что некоторые из расщепленных символов станут нерелевантными для определенных аксиом и могут быть удалены. Например, рассмотрим аксиому состояния-преемника, приведенную в уравнении 11.1, модифицированную так, чтобы в нее был включен символ аэропорта LAX и исключены предусловия действия (которые будут учитываться с помощью отдельных аксиом предусловия):

$\text{At}(\text{Plt JFKI} \leq \Rightarrow (\text{At} (P_i, \text{JFKH} \wedge \text{Fly}\{P_1, \text{JFK, SFO}\}^\circ \wedge \neg \text{Fly}(\text{Plf JFK, LAXH}) \vee \text{Fly}(\text{PltSFO, JFK})^\circ \vee \text{Fly} \{ \text{Plt LAX, JFKH}$ В первом условии сказано, что самолет P_1 должен быть в аэропорту JFK, если он находился в нем во время 0 и не улетел из JFK в какой-то другой город, неважно, какой именно; во втором условии сказано, что он должен находиться в аэропорту JFK, если он прилетел туда из другого города, неважно, из какого именно. При использовании этих расщепленных символов мы можем удалить параметр, значение которого нас не интересует:

$\text{At}\{P_i, \text{JFKI} \leq \Rightarrow (\text{At} \{P_i, \text{JFKH} \wedge \neg (\text{Flyi}(\text{PiH} \wedge \text{Fly2}(\text{JFKH})) \vee (\text{Flyi}(\text{PiH} \wedge \text{Flyi}(\text{JFKH}))$ Обратите внимание на то, что в этой аксиоме аэропорты SFO и LAX больше не упоминаются. Вообще говоря, теперь расщепленные символы действий позволяют добиться того,

чтобы размер каждой аксиомы состояния-преемника был независимым от количества аэропортов. Аналогичные сокращения допускаются в аксиомах предусловия и ак-

сиомах исключения действий (см. упр. 11.16). Для описанного выше случая с 10 временными этапами, 12 самолетами и 30 аэропортами полная аксиома исключения действий сокращается с 583 миллионов выражений до 9360 выражений.

Но остается непреодоленным еще один недостаток: представление с помощью расщепленных символов не допускает описания параллельных действий.

Рассмотрим два параллельных действия, $\text{Fly}(P1, \text{SFO}, \text{JFK})^\circ$ и $\text{Fly}(P2, \text{JFK}, \text{SFO})^\circ$.

Преобразовав их в расщепленные представления, получим следующее: $\text{Fly}_i(\text{PiH} \text{ л } \text{Fly}_2(\text{SFO})^\circ \text{ л } \text{Fly}_3(\text{JFK})^\circ \text{ л } \text{Fly}_i(P2)^\circ \text{ л } \text{Fly}_2(\text{JFK})^\circ \text{ л } \text{Fly}_3(\text{SFO})^\circ$ Теперь больше нет возможности определить с помощью этого выражения, что произошло! Мы знаем, что самолеты P1 и P2 вылетели, но не можем выяснить места их отправления и назначения. Это означает, что должна использоваться полная аксиома исключения действий со всеми указанными выше недостатками.

Планировщики, основанные на проверке выполнимости, способны обрабатывать крупные задачи планирования, например находить оптимальные тридцатипятиэтапные решения для задач планирования в мире блоков с десятками блоков. Размер пропозиционального представления и стоимость решения в высшей степени зависят от задачи, но в большинстве случаев узким местом становится нехватка памяти, требуемой для хранения пропозициональных аксиом. Одним из интересных результатов этих исследований оказалось то, что алгоритмы поиска с возвратами, такие как DPLL, часто лучше решают задачи планирования по сравнению с алгоритмами локального поиска, подобными WalkSAT. Это связано с тем, что основная часть пропозициональных аксиом представляет собой хорновские выражения, которые эффективно обрабатываются с помощью метода распространения единичных выражений. Это наблюдение привело к разработке гибридных алгоритмов, в которых комбинируется некоторый метод случайного поиска с возвратами и метод распространения единичных выражений.

11.6. Анализ различных подходов к планированию Планирование — это область искусственного интеллекта, которая в настоящее время привлекает значительный интерес. Одна из причин такой ситуации состоит в том, что в планировании объединяются два основных направления развития искусственного интеллекта, которые рассматривались нами до сих пор, — поиск и логика.

Это означает, что любой планировщик может рассматриваться либо как программа, в которой осуществляется поиск решения, либо как такая программа, которая (конструктивно) доказывает существование решения. Такое перекрестное

обогащение идеями, взятыми из этих двух областей, привело не только к повышению производительности, которое за последнее десятилетие достигло нескольких порядков величины, но и к расширению использования планировщиков в производственных приложениях. К сожалению, у нас еще нет четкого понимания того, какие методы являются в наибольшей степени подходящими для задач того или иного типа.

Вполне возможно также, что появятся новые методы, которые вытеснят существующие.

Планирование главным образом представляет собой такое занятие, в котором приходится держать под контролем комбинаторный взрыв. Если в проблемной области имеется p примитивных высказываний, то количество состояний становится равным 2^p . В сложных проблемных областях величина p может стать весьма значительной. Следует также учитывать, что объекты в проблемной области характеризуются не только свойствами (Location, Color и т.д.), но и отношениями (At, On, Between и т.д.). При наличии d объектов в проблемной области с тернарными отношениями количество состояний достигает 2^d . На этом основании можно сделать вывод, что в наихудшем случае попытка решить задачу планирования становится безнадежной.

Мощным средством преодоления подобной пессимистической ситуации становится подход по принципу "разделяй и властвуй". В наилучшем случае (при полной декомпозируемости задачи) подход "разделяй и властвуй" обеспечивает экспоненциальное ускорение работы. Однако декомпозируемость нарушается из-за отрицательных взаимодействий между действиями. Планировщики с частичным упорядочением позволяют справиться с такой ситуацией с помощью причинных связей — мощного подхода к формированию представлений, но, к сожалению, каждый конфликт должен быть разрешен с помощью выбора определенного варианта (связанного с размещением конфликтующего действия до или после связи), а количество таких вариантов может увеличиваться экспоненциально. Алгоритм Graphplan позволяет избежать необходимости использования этих вариантов на этапе формирования графа, поскольку в нем для регистрации конфликтов используются взаимно исключающие связи, а выбор варианта, касающегося того, как должны быть разрешены эти конфликты, фактически не осуществляется. Алгоритм SATplan предоставляет аналогичный набор взаимно исключающих отношений, но в нем для этого применяется общая форма CNF, а не специальная структура данных. Насколько успешным окажется такой подход, зависит от области применения решателя задач SAT.

Иногда возможность эффективного решения задачи обеспечивается путем определения того, какие отрицательные взаимодействия могут быть исключены.

Задача называется имеющей Ж упорядочиваемые подцели, если существует такой порядок подцелей, что планировщик может достичь их в указанном порядке, не будучи вынужденным отменять какую-либо из ранее достигнутых подцелей. Например, если в мире блоков цель состоит в построении столбика (например, в котором блок А стоит на В, который, в свою очередь, стоит на С, стоящем, в свою очередь, на столе, Table), то подцели являются упорядочиваемыми снизу вверх: если вначале будет достигнута подцель "С на Table", то никогда не придется ее отменять в ходе достижения других подцелей. Планировщик, в котором используется такой прием с упорядочением снизу вверх, способен решить любую задачу в проблемной области мира блоков без возвратов (хотя и может оказаться, что он не всегда находит самый короткий план).

В качестве более сложного примера укажем, что для планировщика Remote Agent, который управлял космическим аппаратом Deep Space One агентства NASA, было определено, что высказывания, касающиеся управления космическим аппаратом, являются упорядочиваемыми. По-видимому, в этом нет ничего удивительного, поскольку космический аппарат проектируется его разработчиками так, чтобы была возможность управлять им как можно проще (разумеется, с учетом всех прочих ограничений). Пользуясь преимуществом последовательного упорядочения целей, планировщик Remote Agent был способен в процессе планирования устранять основную часть поиска. Это означает, что он оказался достаточно быстродействующим для того, чтобы с его помощью можно было управлять этим космическим аппаратом в реальном времени, а такая задача раньше казалась невыполнимой.

Существует несколько способов ограничения комбинаторного взрыва. Как было показано в главе 5, есть несколько методов управления возвратами в задачах удовлетворения ограничений (Constraint Satisfaction Problem— CSP), таких как возвраты, управляемые зависимостями. Все эти методы могут применяться и в планировании.

Например, задачу извлечения решения из графа планирования можно сформулировать как булеву задачу CSP, переменные которой указывают, должно ли данное конкретное действие произойти в данное конкретное время. Эту задачу CSP можно решить с использованием любого из алгоритмов, приведенных в главе 5, такого как алгоритм с минимальными конфликтами. Тесно связанный с этим метод, используемый в системе Blackbox, состоит в преобразовании графа планирования в выражение CNF, а затем извлечении плана с использованием решателя задач SAT. По-видимому, такой подход обладает более высокой производительностью, чем SATplan, и причиной этого, скорее всего, является то, что в графе планирования уже устранены многие невозможные состояния и действия из рассматриваемой задачи. Кроме того, такой подход действует лучше по сравнению с алгоритмом Graphplan,

по-видимому, из-за того, что поиск условий выполнимости, подобный алгоритму WalkSAT, характеризуется гораздо большей гибкостью, чем ограниченный поиск с возвратами, используемый в алгоритме Graphplan.

Нет никакого сомнения в том, что такие планировщики, как Graphplan, SATplan и Blackbox, привели к значительному прогрессу в области планирования, поскольку позволили, во-первых, поднять уровень производительности систем планирования, а во-вторых, дали возможность понять связанные с этим представительные и комбинаторные проблемы. Однако эти методы по своей сути являются пропозициональными и поэтому предназначены исключительно для использования в тех проблемных областях, которые они способны представить. (Например, задачи планирования экспедиторской доставки с несколькими десятками объектов и местонахождений могут потребовать гигабайтов памяти для хранения соответствующих выражений CNF.) Создается впечатление, что для достижения дальнейшего прогресса в этой области потребуются представления и алгоритмы в логике первого порядка, хотя такие структуры, как графы планирования, по-прежнему будут оставаться полезным источником эвристик.

11.7. Резюме В этой главе определена задача планирования в детерминированных, полностью наблюдаемых вариантах среды. В ней описаны основные представления, используемые для задач планирования, и представлено несколько алгоритмических подходов к их решению. Наиболее важные понятия, изложенные в этой главе, перечислены ниже.

- Системы планирования основаны на использовании алгоритмов решения задач, которые применяются к явным представлениям состояний и действий в пропозициональной логике (или логике первого порядка). Такие представления обеспечивают возможность получения эффективных эвристик, а также разработки мощных и гибких алгоритмов для решения задач планирования.

В языке Strips применяются описания действий в терминах их предусловий и результатов, а также описания начальных и целевых состояний в виде конъюнкций положительных литералов. В языке ADL некоторые ограничения языка Strips ослаблены и допускается использование дизъюнкции, отрицания и кванторов.

Поиск в пространстве состояний может действовать в прямом (прогрессивном) направлении или в обратном (регрессивном) направлении. Эффективные эвристики могут быть получены путем принятия предположения о независимости подцелей, а также с помощью различных ослаблений задачи планирования.

В алгоритмах планирования с частичным упорядочением (Partial-Order Planning — POP) пространство планов исследуется без стремления к созданию полностью упорядоченной последовательности действий. Такие алгоритмы действуют в обратном направлении от цели, добавляя в план действия, нужные для достижения

каждой подцели. Такой подход становится особенно эффективным при решении задач, приемлемых для использования подхода по принципу "разделяй и властвуй".

Граф планирования может формироваться инкрементно, начиная с начального состояния. Каждый его уровень содержит надмножество всех литералов или действий, которые могут обнаруживаться на данном временном этапе. Кроме того, на каждом уровне условно задаются отношения взаимного исключения, или взаимно исключающие отношения между литералами или действиями, которые не могут происходить одновременно. Графы планирования позволяют получать полезные эвристики для планировщиков в пространстве состояний и планировщиков с частичным упорядочением и могут использоваться непосредственно в алгоритме Graphplan.

Алгоритм Graphplan обрабатывает граф планирования, используя обратный поиск для извлечения плана. Этот алгоритм допускает определенное частичное упорядочение между действиями.

В алгоритме SATplan задача планирования преобразуется в пропозициональные аксиомы и после этого к ним применяется алгоритм проверки выполнимости для поиска модели, соответствующей действительному плану. Было разработано несколько разных пропозициональных представлений, обладающих различной степенью компактности и эффективности.

Каждый из основных подходов к планированию имеет своих сторонников, и еще не достигнуто полное согласие в том, какой из этих подходов является наилучшим. Конкуренция между этими подходами и их взаимное обогащение привели к существенному повышению эффективности систем планирования.

Библиографические и исторические заметки Направление планирования в области искусственного интеллекта сформировалось на основе исследований в части поиска в пространстве состояний, доказательства теорем и теории управления, а также на основании практических потребностей робототехники, составления расписаний и других проблемных областей. Первой

важной системой планирования стала система Strips [466], которая наглядно иллюстрирует продуктивность взаимодействия этих научных направлений. Система Strips была разработана как планирующий компонент программного обеспечения для проекта создания робота Shakey в институте SRI. Модель ее общей структуры управления была создана на основе программы GPS (General Problem Solver — общий решатель задач) [1129] — системы поиска в пространстве состояний, в которой используется анализ целей и средств (means—ends analysis). В системе Strips применялась одна из версий системы доказательства теорем QA3 [592] в качестве процедуры определения истинности предусловий для действий. Точные определения для языка Strips и анализ этого языка представлены Лифшицем [928].

Байлендер [213] показал, что простые задачи планирования Strips являются PSPACE-полными. В [467] приведена историческая ретроспектива проекта Strips и дан краткий обзор того, как этот проект связан с более современными разработками в области планирования.

Способ представления действий, использовавшийся в системе Strips, оказал гораздо более значительное влияние на дальнейшие разработки, чем ее алгоритмический подход. С тех пор почти во всех системах планирования применяется тот или иной вариант языка Strips. К сожалению, из-за огромного разнообразия вариантов задача их сравнения стала чрезмерно трудной. Со временем возникло лучшее понимание ограничений и компромиссов между формальными подходами. В языке ADL (Action Description Language — язык описания действий) [1195] ослаблены некоторые ограничения языка Strips и создана возможность представлять более реалистичные задачи. В [1119] рассматриваются схемы, применимые для компиляции определений ADL в определения Strips. Для использования в качестве стандартизированного синтаксиса, допускающего синтаксический анализ с помощью компьютера, который предназначен для представления определений на языках Strips, ADL и других языках, был предложен язык PDDL (Problem Domain Description Language — язык описания проблемной области) [548]. PDDL использовался в качестве стандартного языка для соревнований по планированию на конференции AIPS, начиная с 1998 года.

В первой половине 1970-х годов планировщики в основном применялись для получения полностью упорядоченных последовательностей действий. Декомпозиция задачи осуществлялась путем вычисления субплана для каждой подцели с последующей увязкой субпланов в единую цепочку в некотором порядке. Вскоре было обнаружено, что такой подход, названный Ж линейным планированием в [1337], является неполным. Он не позволяет решать некоторые очень простые задачи, такие как аномалия Зюссмана (см. упр. 11.11), обнаруженная Алленом Брауном во время экспериментов с системой Hacker [1474]. Полный планировщик должен обеспечивать $\wedge$ чередование действий из различных субпланов в пределах единой последовательности. Понятие упорядочиваемых подцелей [837] точно соответствует множеству задач, для которых планировщики без чередования позволяют получить полное решение.

Одним из решений проблемы чередования оказалось планирование с регрессией от цели. Это — метод, в котором этапы полностью упорядоченного плана переупорядочиваются так, чтобы можно было избежать конфликта между подцелями.

Данный подход был предложен Уолдингером [1551], кроме того, использовался в системе Warplan Уоррена [1554]. Система Warplan замечательна также тем, что была первым планировщиком, написанным на языке логического программирования (Prolog), и остается одним из лучших примеров невероятной экономии объема кода,

которая иногда может быть достигнута с использованием логического программирования: программа Warplan состоит только из 100 строк кода, что составляет лишь небольшую часть размера сравнимых с ней планировщиков на то время. Система Interplan [1493], [1494] обеспечивала также произвольное чередование этапов плана, что дало возможность преодолеть аномалию Зюссмана и связанные с ней проблемы.

К основным идеям, лежащим в основе планирования с частичным упорядочением, относятся обнаружение конфликтов [1493] и защита достигнутых условий от вмешательства [1474]. Первыми примерами средств создания планов с частичным упорядочением (которые в то время назывались сетями задач) были планировщик Noah Сакердоти [1337], [1338] и система Nonlin Тейта [1494], [1495]7.

Планирование с частичным упорядочением в течение следующих 20 лет стало ведущим направлением разработок, несмотря на то, что в течение основной части этого периода специфика данной области не нашла широкого понимания. Система Tweak Чепмена [234] стала образцом логической реконструкции и упрощения работ по планированию, проводимых в то время; формулировки, используемые в этой системе, были достаточно ясными для того, чтобы можно было доказывать полноту и разрешимость (либо NP-трудность и неразрешимость) различных формулировок задач планирования. Эта работа Чепмена привела к созданию Макаллестером и Розенблиттом [1008] полного планировщика с частичным упорядочением, который впервые можно было вполне обоснованно считать имеющим достаточно простое и доступное для чтения описание [1008]. Одна из реализаций алгоритма Макаллестера и Розенблитта, разработанная Содерлендом и Уэлдом и получившая название SNLP [1444], нашла широкое распространение и впервые позволила многим исследователям изучать и проводить эксперименты по планированию с частичным упорядочением. Алгоритм POP, описанный в этой главе, основан на алгоритме SNLP.

Кроме того, группа Уэлда разработала UCPOP [1202], первый планировщик для задач, представленных на языке ADL. В планировщике UCPOP применяется эвристика, в которой учитывается количество невыполненных целей. Применяемый в нем алгоритм работал немного быстрее, чем SNLP, но редко оказывался способным находить планы, состоящие больше чем примерно из десяти этапов. Хотя для планировщика UCPOP были разработаны усовершенствованные эвристики [543], [751], направление планирования с частичным упорядочением постепенно вытеснялось из перспективных исследований в 1990-х годах по мере появления более быстрых методов. Но в [1135] было показано, что эта область исследований вполне заслуживает возобновления к ней интереса: предложенный в этой работе планировщик RePOP при использовании точной эвристики, полученной из графа планирования, оказался намного более масштабируемым по сравнению с

планировщиком Graphplan и сумел составить конкуренцию самым быстрым планировщикам в пространстве состояний.

Работы Аврима Блюма и Меррика Фурста [139], [140] способствовали дальнейшему пробуждению интереса к этой области планирования, поскольку созданная ими система Graphplan оказалась на несколько порядков величин быстрее по сравнению с планировщиками с частичным упорядочением, применявшимися в то время. В терминологии существует некоторая путаница. Многие авторы используют термин нелинейный (nonlinear) вместо "частично упорядоченный" (partially ordered). Такое употребление терминов немного отличается от первоначального употребления, которое в работах Сакердоти относилось к чередующимся планам.

Вскоре последовали разработки других систем с графами планирования, такие как IPP [813], Stan [491] и SGP [1569]. Структура данных, весьма напоминающая граф планирования, была разработана немного раньше Галлабом и Ларуэллом [549], которые использовали такую структуру данных в планировщике с частичным упорядочением IxTeT для получения точных эвристик, применяемых для управления поиском. В [1136] приведен очень подробный анализ эвристик, полученных на основе графов планирования. Приведенное в этой главе описание графов планирования главным образом основано на этой работе и на конспектах лекций, подготовленных Суббарао Камбхампати (Subbarao Kambhampati). Как уже указывалось в этой главе, граф планирования позволяет управлять поиском решения задачи с помощью многих способов. Например, победителем соревнования по планированию на конференции AIPS 2002 года стал алгоритм LPG [544], в котором осуществляется поиск в графах планирования с использованием метода локального поиска, основанного на алгоритме WalkSAT.

Подход к планированию как проверке выполнимости и алгоритм SATplan были предложены Каутцем и Селманом [778], которых натолкнул на эту идею поразительный успех в использовании жадного локального поиска для решения задач проверки выполнимости (см. главу 7). Кроме того, Каутц и др. [777] исследовали различные формы пропозициональных представлений для аксиом Strips и обнаружили, что наиболее компактные формы не обязательно способствуют достижению наименьшей продолжительности вычисления решения. Систематический анализ был проведен Эрнстом и др. [442], которые разработали также автоматический "компилятор" для формирования пропозициональных представлений из формулировок задач на языке PDDL. Планировщик Blackbox, в котором объединены идеи алгоритмов Graphplan и SATplan, был разработан Каутцем и Селманом [779].

Повторное пробуждение интереса к планированию в пространстве состояний было в основном вызвано появлением программы UnPOP, разработанной Макдермоттом [1024], который впервые предложил эвристику с учетом расстояния, основанную на

ослабленной задаче с исключенными списками удаления. Само название UnPOP (отказ от POP) стало реакцией на чрезмерное в то время сосредоточение работ на планировании с частичным упорядочением; Макдермотт считал, что другие подходы не получают того внимания, которого они заслуживают. В алгоритме HSP (Heuristic Search Planner — планировщик с эвристическим поиском) Бонета и Геффнера [147] и его разработанных позже аналогах впервые удалось добиться практического применения поиска в пространстве состояний при решении крупных задач планирования. В то время самым успешным алгоритмом поиска в пространстве состояний был алгоритм FastForward, или FF, Хоффмана [667], который стал победителем в соревновании по планированию на конференции AIPS в 2000 году. В алгоритме FF используется упрощенная эвристика для графа планирования в сочетании с очень быстрым алгоритмом поиска, представляющим собой новаторское объединение прямого и локального поиска.

В дальнейшем появился интерес к использованию представления планов в виде $\wedge$ бинарных диаграмм решения (binary decision diagram — BDD) — компактного описания конечных автоматов, которое широко исследовалось в сообществе разработчиков средств проверки аппаратного обеспечения [266], [1031]. Существуют методы доказательства свойств бинарных диаграмм решения, включая свойство быть решением задачи планирования. В [261] представлен планировщик, основанный на этом

подходе. Применялись также другие представления; например, в [1549] приведен обзор использования целочисленного программирования для планирования.

Точки над "/" в этом вопросе еще окончательно не расставлены, но уже появились некоторые интересные сопоставления различных подходов к планированию.

В [646] проанализировано несколько классов задач планирования и показано, что подходы на основе ограничений, такие как Graphplan и SATplan, являются наилучшими для NP-трудных проблемных областей, а подходы на основе поиска лучше подходят для проблемных областей, в которых приемлемые решения могут быть найдены без поиска с возвратами. Алгоритмы Graphplan и SATplan сталкиваются с затруднениями при использовании в проблемных областях со многими объектами, поскольку наличие большого количества объектов означает, что в этих алгоритмах приходится создавать много действий. В некоторых случаях возникновение такой проблемы можно отсрочить или вообще ее избежать, вырабатывая пропозиционализированные действия динамически, по мере необходимости, а не конкретизируя их все до начала поиска.

В [1567], [1568] приведены два превосходных обзора современных алгоритмов планирования. Любопытно наблюдать за тем, какие изменения произошли за пять лет, которые прошли за время между этими двумя обзорами: первый из них

сосредотачивался на планировании с частичным упорядочением, а во втором были представлены алгоритмы Graphplan и SATplan. В книге Readings in Planning [19] приведена всеобъемлющая антология многих из самых лучших ранних статей в этой области, включая несколько хороших обзоров. В [1629] приведен обзор методов планирования с частичным упорядочением, который занимает целую книгу.

Исследования по планированию играли центральную роль в искусственном интеллекте со времени появления этого научного направления, а статьи по планированию занимают основной объем ведущих журналов и материалов конференций по искусственному интеллекту. Проводятся также специализированные конференции по планированию, такие как International Conference on AI Planning Systems (AIPS), International Workshop on Planning and Scheduling for Space и European Conference on Planning.

Упражнения 11.1. Опишите различие и сходство между решением задач и планированием.

11.2. При наличии аксиом, приведенных в листинге 11.1, определите все применимые конкретные экземпляры действия Flyip, from, to) в состоянии, описанном следующим выражением:

At(Plt JFK) л At{P2t SFO) л P1ane{Px) а Plane{P2) а Airport{JFK) А Airport{SFO) 11.3. Рассмотрим, как можно было бы преобразовать множество схем Strips в аксиомы состояния-преемника ситуационного исчисления (см. главу 10). • Изучите схему для действия Fly(p, from, to). Запишите логическое определение для предиката FlyPrecond(p, from, to, s), который является

истинным, если предусловия для действия Flyip, from, to) выполнены в ситуации s. • Затем, предположив, что Fly(p, from, to) является единственной схемой действия, доступной для агента, запишите аксиому состояния-преемника для литерала At (p, x, s), который представляет ту же информацию в виде схемы действия. • Теперь предположим, что есть еще один метод перемещения в пространстве—телепортация: Teleportip, from, to). Он имеет дополнительное предусловие - iWarped(p) (пространство не искривлено) и дополнительный результат Warped(p) (пространство искривлено). Объясните, как должна быть модифицирована база знаний ситуационного исчисления. • Наконец, разработайте общую и точно заданную процедуру для осуществления преобразования из множества схем Strips в множество аксиом состояния-преемника.

11.4. Лабораторная обезьяна сталкивается с задачей "обезьяна и бананы", в которой с потолка свисают бананы, не достижимые непосредственно. Имеется ящик, позволяющий обезьяне достать бананы, если она на него заберется.

Первоначально обезьяна находится в позиции А, бананы — в позиции В, а ящик — в позиции С. Обезьяна и ящик имеют высоту Low, но после того как обезьяна заберется на ящик, она будет иметь высоту High, такую же, как и у бананов. Действия, доступные для обезьяны, включают Go (Переход с одного места в другое), Push (Перемещение объекта из одного места в другое), ClimbUp (Подъем на объект) или ClimbDown (Спуск с объекта), а также Grasp (Схватывание объекта) или Ungrasp (Отпускание объекта).

Схватывание приводит к овладению объектом, если обезьяна и объект находятся в одном и том же месте и на одной и той же высоте. а) Составьте начальное описание состояния. б) Запишите определения указанных шести действий в стиле Strips. в) Предположим, что обезьяна хочет поставить в тупик ученых (которые перестали за ней следить и отправились пить чай), схватив бананы, но оставив ящик на первоначальном месте. Запишите эту задачу как общую цель (т.е. не предполагая, что ящик обязательно находится в позиции С) на языке ситуационного исчисления. Может ли задача достижения этой цели быть решена с помощью системы в стиле Strips? г) Можно не сомневаться в том, что аксиома для действия по перемещению, составленная читателем, будет неправильной, поскольку, если объект слишком тяжел, то его позиция после применения оператора Push должна оставаться неизменной. Является ли это примером проблемы распространения последствий или проблемы спецификации? Исправьте составленное вами описание задачи, чтобы в нем учитывались тяжелые объекты.

11.5. Объясните, почему процесс формирования преемников в обратном поиске не требует добавления литералов, представляющих собой отрицательные результаты в рассматриваемом действии.

11.6. Объясните, почему удаление отрицательных результатов из каждой схемы действия в задаче Strips приводит к получению ослабленной задачи.

11.7. Изучите определение двунаправленного поиска в главе 3. а) Была бы идея двунаправленного поиска в пространстве состояний перспективной для использования в планировании? б) А что можно сказать применительно к двунаправленному поиску в пространстве планов с частичным упорядочением? в) Разработайте версию планирования с частичным упорядочением, в которой любое действие может быть добавлено к плану, если выполнения его предусловий можно добиться с помощью результатов действий, уже имеющих в плане. Объясните, как следует поступать с конфликтами и ограничениями упорядочения. Является ли этот алгоритм по сути идентичным прямому поиску в пространстве состояний? г) Рассмотрите планировщик с частичным упорядочением, в котором сочетается метод, описанный в упр. 11.7, в, со стандартным методом добавления действий для достижения открытых условий. Будет ли результирующий алгоритм таким же, как

описано в упр. 11.7, 61 11.8. Сконструируйте уровни 0, 1 и 2 графа планирования для задачи, приведенной в листинге 11.1.

11.9. Докажите приведенные ниже утверждения о графе планирования. • Литерал, который не появляется на заключительном уровне графа, не может быть достигнут. • Уровневая стоимость литерала в последовательном графе не больше фактической стоимости оптимального плана его достижения.

11.10. Авторы подчеркнули различие между планировщиками с прямым и обратным поиском в пространстве состояний и планировщиками с частичным упорядочением, указав, что последние представляют собой средства поиска в пространстве планов. Объясните, на основании чего средства прямого и обратного поиска в пространстве состояний можно также рассматривать как средства поиска в пространстве планов, и укажите, каковыми являются операторы уточнения плана.

11.11. На рис. 11.8 показана задача в мире блоков, известная как $\wedge$ аномалия Зюссмана. Эта задача рассматривается как аномальная, поскольку планировщики без чередования, применявшиеся в начале 1970-х годов, не могли ее решить.

Запишите определение этой задачи в системе обозначений Strips и решите ее либо вручную, либо с использованием программы планирования.

Планировщиком без чередования является такой планировщик, который после получения двух подцелей, G_1 и G_2 , вырабатывает либо план для G_1 , который соединяется с планом G_2 , либо наоборот. Объясните, почему планировщик без чередования не мог решить эту задачу.

В в с Начальное состояние Целевое состояние Рис. 11.8. Задача планирования в мире блоков, получившая название аномалии Зюссмана 11.12. Рассмотрите задачу надевания туфель и носков, которая определена в разделе 11.3. Примените для решения этой задачи алгоритм Graphplan и покажите полученное решение. Добавьте действия для надевания пальто и шляпы.

Продемонстрируйте план с частичным упорядочением, который представляет собой одно из решений, и покажите, что существует 180 различных линеаризации плана с частичным упорядочением. Каково минимальное количество различных решений с графом планирования, необходимых для представления всех 180 линеаризаций?

11.13. Первоначальная программа Strips была разработана для управления роботом Shakey. На рис. 11.9 показана версии мира Shakey, состоящего из четырех комнат (Room), расположенных вдоль коридора (Corridor), где в каждой комнате имеется дверь (Door) и выключатель света (Switch).

Действия в мире робота Shakey включают перемещение из одного места в другое, передвижение подвижных объектов (таких как ящики), подъем и спуск с прочных

объектов (таких как ящики), а также включение и выключение выключателей света. Сам этот робот никогда не был достаточно находчивым, чтобы забраться на ящик или щелкнуть выключателем, но планировщик Strips оказался способным находить и выводить на печать планы, превосходящие мыслительные способности робота. Ниже перечислены шесть действий робота Shakey. • Действие $Go(x,y)$, которое требует, чтобы Shakey был в позиции x , и определяет, что позиции x и y находятся в одной и той же комнате. В соответствии с общепринятым соглашением дверь между двумя комнатами считается находящейся одновременно в обеих этих комнатах. • Действие по перемещению ящика b из позиции x в позицию y в пределах одной той же комнаты, $Push(b, x,y)$. Нам потребуются предикат Box и константы для описания ящиков. • Действия по подъему на ящик, $ClimbUp(b)$, и спуску с ящика, $ClimbDown(b)$. Нам потребуются предикат On и константа $Floor$ (Пол). • Действия по включению выключателя света, $TurnOn(s)$, и его выключению, $TurnOff(s)$. Для того, чтобы включить или выключить свет, робот Shakey должен стоять на ящике, расположенном в том месте, где находится выключатель света.

Рис. 11.9. Мир робота Shakey. Робот Shakey может передвигаться между отметками в пределах комнаты, проходить через двери между комнатами и коридором, взбираться на те объекты, на которые ему разрешено взбираться, и двигать те объекты, которые ему разрешено двигать, а также может щелкать выключателями света. Опишите в системе обозначений Strips шесть действий робота Shakey и показанное на рис. 11.9 начальное состояние. Составьте план, с помощью которого робот Shakey мог бы переместить ящик $Box2$ в комнату $Room2$.

11.14. В этой главе встречались только такие графы планирования, которые позволяли использовать лишь пропозициональные действия. А что было бы, если возникла необходимость применять графы планирования для задач с переменными в цели, таких как $At(P_1, x) \wedge At(P_2, x)$, где x пробегает по конечной области определения местонахождений? Как можно было бы представить такую задачу, чтобы для ее решения могли использоваться графы планирования? (Подсказка. Вспомните действие $Finish$ из области планирования POP. Какие предусловия оно должно иметь?) 11.15. Вплоть до настоящего момента предполагалось, что действия выполняются только в подходящих для этого ситуациях. Рассмотрим, что должны утвер-

ждать пропозициональные аксиомы состояния-преемника, такие как уравнение 11.1, в отношении действий, предусловия которых не выполнены. а) Покажите, что эти аксиомы предсказывают, что ничего не произойдет, если действие будет осуществлено в таком состоянии, в котором не выполнены его предусловия. б) Рассмотрите план p , который содержит действия, требуемые для достижения цели, но включает также недопустимые действия. Является ли истинным или ложным приведенное ниже высказывание?

Начальное состояние л Аксиомы состояния-преемника л р И Цель в) Возможно ли с помощью аксиом состояния-преемника первого порядка в ситуационном исчислении (как в главе 10) доказать, что план, содержащий недопустимые действия, позволяет достичь цели?

11.16. Приводя примеры из проблемной области грузового аэропорта, объясните, каким образом метод расщепления символов позволяет уменьшить размеры аксиом предусловия и аксиом исключения действий. Выведите общую формулу для оценки размера каждого множества аксиом в терминах количества временных этапов, количества схем действий, их аристностей и количества объектов.

11.17. В алгоритме SATplan, приведенном в листинге 11.7, предусмотрено, что каждый вызов алгоритма проверки выполнимости подтверждает цель d_t , где t находится в пределах от 0 до $T_{\max}$. Предположим вместо этого, что алгоритм проверки выполнимости вызывается только один раз, с целью $d^0 \vee d^1 \vee \dots \vee d^{T_{\max}}$. а) Будет ли такой вызов всегда возвращать план с длиной, меньшей или равной $T_{\max}$, если таковой существует? б) Приведет ли такой подход к появлению каких-либо новых фиктивных "решений"? в) Обсудите, как можно было бы модифицировать алгоритм проверки выполнимости, такой как WalkSAT, чтобы он находил короткие решения (если они существуют) после получения дизъюнктивной цели в указанной выше форме.

Глава 12. Планирование и осуществление действий в реальном мире

12.1. ДЕЙСТВИЯ В этой главе показано, что более выразительные представления и более интерактивные архитектуры агентов обеспечивают возможность создания планировщиков, применимых в реальном мире.

В предыдущей главе описывались наиболее важные понятия, представления и алгоритмы для планирования. Планировщики, применяемые в реальном мире для решения таких задач, как планирование наблюдений с помощью космического телескопа Хаббл, управление заводами и фабриками, а также осуществление поставок для военных кампаний, являются более сложными; они превосходят свои более простые аналоги и с точки зрения языка представления, и с точки зрения того способа, который применяется в планировщике для взаимодействия с его средой. Такие различия описаны в настоящей главе. В разделе 12.1 рассматриваются задачи планирования и составления расписаний с учетом временных и ресурсных ограничений, а в разделе 12.2 описывается планирование с помощью предопределенных субпланов. В разделах 12.3—12.6 рассматривается ряд архитектур агентов, предназначенных для осуществления действий в неопределенных вариантах среды. В разделе 12.7 показано, как должны составляться планы, когда среда содержит и других агентов.

12.1. Время, расписания и ресурсы В представлении на языке Strips говорится о том, какие действия должны быть выполнены, но это представление основано на ситуационном исчислении, поэтому с его помощью нельзя показать, сколько времени потребуется на проведение того или иного действия, или даже обозначить время, когда должно произойти это действие, помимо указаний на то, что оно должно быть осуществлено до или после дру-

гого действия. Но применительно к некоторым проблемным областям было бы желательно также указывать, когда должны начинаться и оканчиваться действия.

Например, в проблемной области транспортировки грузов может потребоваться знать, когда именно прибывает самолет, перевозящий некоторый груз, а не просто пользоваться информацией о том, что он прибудет, когда закончится полет.

Время является существенно важным фактором для крупного семейства приложений, называемых задачами планирования производства. Для выполнения таких задач требуется осуществление ряда работ, каждая из которых состоит из последовательности действий, а каждое действие имеет определенную продолжительность и может потребовать определенных ресурсов. Проблема состоит в разработке расписания, которое минимизирует общее время, требуемое для выполнения всех работ, при соблюдении ограничений на ресурсы.

Пример задачи планирования производства приведен в листинге 12.1. Это — в высшей степени упрощенная задача сборки автомобиля. В ней представлены две работы: сборка автомобилей C1 и C2. Каждая работа состоит из трех действий: установка двигателя, установка колес и проверка результатов. Двигатель должен устанавливаться в первую очередь (поскольку в автомобиле этой модели с установленными передними колесами затрудняется доступ к двигательному отсеку), а проверка, безусловно, должна проводиться в последнюю очередь.

Листинг 12.1. Задача планирования производства, связанная со сборкой двух автомобилей.

Обозначение Duration(d) показывает, что для выполнения некоторого действия требуется d минут, а обозначение Engine(Elc 60) показывает, что E1 — это двигатель, который устанавливается в шасси Ci, и для его установки требуется 60 минут Init (Chassis (d) л Chassis (C2) л Engine(Elc 60) л Engine(E2, C2, 60) л WheelsWi, Clc 30) л Wheels(W2, C2, 15)) Goal {Done(Ci) л Done{C2)) Action(AddEngine(e, c) Precond: Engine(e, c, d) л Chassis(c) л -nEngineIn(c) , Effect: EngineIn(c) л Duration(d)) Action{AddWheels(w, c) , Precond: EngineIn(c) л Wheels(w, c, d) л Chassis(c) , Effect: WheelsOn(c) л Durationid)) Action(Inspect(c), Precond: EngineIn(c) л WheelsOn(c) л Chassis(c), Effect: Done(c) л DurationA0)) Задача, приведенная в листинге 12.1, может быть решена с помощью любого из планировщиков, которые уже описывались в настоящей книге. На рис. 12.1 показано решение, которое было бы получено с

помощью планировщика рор с частичным упорядочением (если игнорируются некоторые числовые данные). Теперь для преобразования этой задачи в задачу составления расписания, а не в задачу планирования, необходимо определить, когда должно начаться и закончиться каждое действие, с учетом продолжительности действия, а также их упорядочения. Обозначение Duration (d) в спецификации результата действия (где переменная d должна быть привязана к некоторому числу) показывает, что для выполнения действия требуется d минут.

```
[0,0] Start И0Д5] \ddEngine1 [0,0] ,*|AddEngine2| [30,45] |AddWheels| [60,60]
|AddWheels2| [60,75] •| Inspect [75,75] Inspect 2 И [85,85] Finish i AddWheels| b_
AddEngine1 r [inspect 1[:
```

Add Engine2 —| Рис. 12.1. Решение задачи планирования производства, приведенной в листинге 12.1. В верхней части этого рисунка решение показано в виде плана с частичным упорядочением. Продолжительность каждого действия обозначена в нижней части каждого прямоугольника, а значения самого раннего и самого позднего времени начала, условно обозначаемые как /"ES, LS7, показаны в верхней левой части. Разность между этими двумя числами представляет собой резерв времени для действия; действия с нулевым резервом находятся на критическом пути, который обозначен жирными стрелками.

В нижней части рисунка то же решение приведено в виде временного графика. Серые прямоугольники показывают интервалы времени, в течение которых может быть выполнено некоторое действие, при условии, что соблюдаются ограничения упорядочения. Незанятая часть серого прямоугольника обозначает резерв времени. Определив частичное упорядочение действий с учетом их продолжительности, как показано на рис. 12.1, можно применить ^ метод критического пути (Critical Path Method — CPM) для выяснения допустимых значений времени начала и окончания каждого действия. Путем через план с частичным упорядочением называется линейно упорядоченная последовательность действий, начинающаяся в состоянии Start и оканчивающаяся в состоянии Finish (например, в плане с частичным упорядочением, показанном на рис. 12.1, есть два пути). ^ Критическим называется путь, имеющий максимальную суммарную продолжительность; этот путь называется "критическим", поскольку от него зависит продолжительность выполнения всего плана, — сокращение продолжительности других путей не приведет к сокращению времени выполнения всего плана в целом, но задержка начала любого действия в критическом пути замедлит осуществление всего плана. На этом рисунке критический путь показан жирными линиями. Для осуществления всего плана за минимальное суммарное время действия в критическом пу-

ти следует выполнять без задержки между ними. С другой стороны, действия, лежащие вне критического пути, имеют определенный запас времени — временное окно, в течение которого они могут быть выполнены. Это окно задается в терминах

самого раннего возможного времени начали, ES , и самого позднего возможного времени начала, LS . Разность $LS - ES$ называется Δ резервом времени для действия. Как показано на рис. 12.1, для выполнения всего плана потребуется 85 минут, каждое действие в критическом пути имеет резерв времени 0 (такое условие имеет место во всех расписаниях), а каждое действие в работе по сборке Q имеет десятиминутное окно, в пределах которого оно может быть начато. Значения времени ES и LS для всех действий, вместе взятые, составляют Δ расписание, представляющее собой решение данной задачи.

Ниже приведены формулы, которые служат определением значений ES и LS , а также лежат в основе алгоритма динамического программирования, применяемого для их вычисления.

$ES(\text{Start}) = 0$ $ES(B) = \max_{A \in \text{BES}(B)} \{ES(A) + \text{Duration}(A)\}$ $LS(\text{Finish}) = ES(\text{Finish})$ $LS(A) = \min_{B \in \text{BLS}(A)} \{LS(B) - \text{Duration}(A)\}$ Идея этого алгоритма состоит в том, что вначале следует присвоить терму $ES(\text{Start})$ значение 0. Затем, как только будет выявлено действие B , такое, что все действия, непосредственно предшествующие B , имеют присвоенные значения ES , терму $ES(B)$ присваивается максимальное из самых ранних значений времени завершения этих непосредственно предшествующих действий, где самое раннее время завершения действия определяется как самое раннее время начала плюс его продолжительность. Этот процесс повторяется до тех пор, пока каждому действию не присваивается значение ES . Значения LS вычисляются аналогичным образом, в ходе передвижения в обратном направлении от действия Finish . Проработку деталей этого алгоритма оставляем читателю в качестве упражнения.

Сложность алгоритма критического пути составляет всего лишь $O(Nb)$, где N — количество действий; b — максимальный коэффициент ветвления на входе или выходе любого действия. (Чтобы убедиться в этом, достаточно отметить, что вычисления значений LS и ES осуществляются по одному разу для каждого действия, а в каждом вычислении происходит итерация, самое большее, по b другим действиям.) Поэтому, если дано частичное упорядочение действий, задача поиска расписания с минимальной продолжительностью решается очень просто.

Составление расписаний с ресурсными ограничениями Реальные задачи составления расписаний усложняются из-за наличия ограничений на Δ ресурсы. Например, для установки в автомобиль двигателя требуется лебедка для двигателя. Если есть только одна лебедка, то нельзя одновременно устанавливать двигатель E_1 в автомобиль d и двигатель E_2 в автомобиль C_2 , поэтому расписание, показанное на рис. 12.1, будет неосуществимым. В данном примере лебедка для двигателя представляет собой пример Δ повторно применяемого ресурса — ресурса, который "занят" во время действия, но снова становится доступным после завершения этого действия. Следует отметить, что повторно применяемые ре-

ресурсы невозможно учесть в нашем стандартном описании действий в терминах предусловий и результатов, поскольку количество доступных ресурсов после завершения действия остается неизменным¹. По этой причине дополним наше представление, включив в него поле в форме Resource: R(k), которое означает, что для выполнения данного действия требуются k единиц ресурса R. Требования к ресурсам являются одновременно и предпосылкой (действие не может быть выполнено, если ресурс недоступен), и временным результатом, в том смысле, что во время выполнения действия доступность ресурса г сокращается на k единиц. В листинге 12.2 показано, как дополнить задачу сборки автомобилей для включения трех ресурсов — лебедки для двигателя, с помощью которой устанавливаются двигатели, станции монтажа колес, на которой устанавливаются колеса, и двух контролёров. На рис. 12.2 показано решение с самым быстрым временем завершения, 115 минут. Это время больше по сравнению с 80 минутами, которые требовались для выполнения расписания без ресурсных ограничений. Следует отметить, что нет такого промежутка времени, в который требовались бы оба контролёра, поэтому, составив данное расписание, можно сразу же перевести одного из двух контролёров на другой участок, где он будет приносить больше пользы.

EngineHoists(l) [AddEngineT WheelStations(l) InspectorsB) i 1 1 1 1 1 1 1 1 1 1 1 0 10
20 30 40 50 60 70 80 90 100 110 120

Рис. 12.2. Решение задачи составления производственного расписания с ресурсами, приведенной в листинге 12.2. На левом поле перечислены три ресурса, а действия показаны с выравниванием по горизонтали с теми ресурсами, которые в них используются. Существуют два возможных расписания, зависящих от того, при сборке какого автомобиля первой используется станция установки двигателя; здесь показано оптимальное решение, которое занимает 115 минут. Листинг 12.2. Задача составления производственного расписания для сборки двух автомобилей с учетом ресурсов. Доступными ресурсами являются одна станция сборки двигателя, одна станция сборки колес и два контролёра. Обозначение Resource: г указывает, что ресурс г используется во время выполнения действия, но снова становится свободным после завершения этого действия

```
Init {Chassis (Ci) л Chassis (C2) л Engine {Ei, Clt 30) л Engine {E2l C2, 60) л
Wheels {Wlt Clt 30) л Wheels {W2, C2, 15) л EngineHoistsA) л WheelStationsA) л
InspectorsB)) Goal {Done {Ci) л Done{C2)) Action{AddEngine{e, c) , Precond: Engine{e, c,
d) л Chassis {c) л —iEngineIn(c), Effect: EngineIn(c) л Duration(d), 1 В отличие от этого,
потребляемые ресурсы, такие как винты для монтажа двигателя, могут учитываться
в рамках стандартной инфраструктуры; см. упр. 12.2.
```

```
Resource: EngineHoistsA)) Action(AddWheels(w, c), Precond: EngineIn(c) л Wheels(w, c, d)
л Chassis(c), Effect: WheelsOn(c) л Duration(d), Resource: Wheel StationsA))
Action(Inspect(c) , Precond: EngineIn(c) л fo#3eeisOn(c) , Effect: Done(c) л DurationA0) ,
```

Resource: InspectorsA)) Представление ресурсов в виде числовых количественных значений, таких как InspectorsB), а не именованных сущностей, таких как Inspector (J/J и Inspector {J2), может служить примером очень продуктивного метода, называемого $\wedge$ агрегированием. Основная идея агрегирования состоит в том, что отдельные объекты должны группироваться в количественные величины, если все объекты неразличимы применительно к рассматриваемому назначению. В данной задаче сборки не имеет значения, какой именно контролёр проверит автомобиль, поэтому нет необходимости проводить между ними различие. (Та же идея может применяться при решении задачи с миссионерами и каннибалами, приведенной в упр. 3.9.) Агрегирование представляет собой важный способ уменьшения сложности задач.

Рассмотрим, что произойдет, если будет предложено расписание, в котором имеется 10 одновременных действий по проверке Inspect, но в наличии есть только 9 контролёров. Если контролёры представлены с помощью количественных величин, неудача будет обнаружена немедленно и алгоритм выполнит возврат, чтобы попытаться составить другое расписание. А если бы контролёры были представлены как отдельные лица, то в алгоритме пришлось бы выполнять возвраты для опробования всех $10!$ способов назначения контролёров для выполнения действий Inspect, притом что положительный результат так и не был бы получен.

Несмотря на их преимущества, ресурсные ограничения приводят к усложнению задач планирования, поскольку в них вводятся дополнительные зависимости между действиями. К тому же составление расписания без ограничений с использованием метода критического пути является несложным, а задача поиска расписания с ресурсными ограничениями, характеризующегося самым ранним возможным временем завершения, является NP-трудной. Такая сложность часто наблюдается не только в теории, но и на практике. Одно из заданий, которое предложили исследователям для проверки их сил в 1963 году (найти оптимальное расписание для задачи, в которой рассматриваются только 10 машин и 10 работ по 100 действий каждая), не удавалось решить в течение 23 лет [900]. Для его решения было опробовано много подходов, включая метод ветвей и границ, алгоритм эмуляции отжига, поиск с запретами, метод удовлетворения ограничений и другие методы, описанные в части II этой книги. Одной из простых, но широко применяемых эвристик является алгоритм с $\wedge$ минимальным резервом. В нем планирование действий осуществляется в режиме жадного поиска. Во время каждой итерации в этом алгоритме рассматриваются не введенные в расписание действия, для которых в расписании заданы все их преемники, и в расписание вводится то действие, которое имеет наименьший резерв для самого раннего возможного времени начала. После этого в алгоритме обновляются значения времени ES и LS для каждого затронутого действия и повторя-

ется итерация. Эвристика основана на том же принципе, что и эвристика с "наиболее ограниченной переменной" в задачах удовлетворения ограничений. Этот алгоритм хорошо работает на практике, но применительно к рассматриваемой задаче сборки он привел к получению 130-минутного решения, а не 115-минутного решения, показанного на рис. 12.2.

Подход, принятый в этом разделе, состоит в том, что "вначале следует оставлять план, а затем расписание". Это означает, что общая задача подразделяется на этап планирования, в котором выбираются и частично упорядочиваются действия для достижения целей данной задачи, и на следующий за ним этап составления расписания, в котором в план вводится временная информация для обеспечения того, чтобы он соответствовал ограничениям по ресурсам и срокам. Такой подход широко применяется в организациях, занимающихся реальным планированием производства и поставок, в которых этап планирования часто выполняется экспертами-людьми.

Однако, если существуют жесткие ограничения на ресурсы, могут возникать такие ситуации, что одни допустимые планы приводят к получению гораздо более лучших графиков, чем другие. В этом случае имеет смысл интегрировать этапы планирования и составления расписаний, принимая во внимание продолжительности и совпадения действий во времени на этапе создания плана с частичным упорядочением. Для учета такой информации могут быть дополнены некоторые алгоритмы планирования, описанные в главе 11. Например, планировщики с частичным упорядочением могут обнаруживать нарушения ресурсных ограничений во многом с помощью такого же способа, который в них используется для обнаружения конфликтов с помощью причинных связей. А эвристики могут быть модифицированы для оценки общего времени завершения плана, а не просто суммарной стоимости всей действий. В настоящее время в этой области исследований ведется активная работа.

12.2. Планирование иерархической сети задач Одной из наиболее привлекательных идей в области решения сложных задач является ^ иерархическая декомпозиция. На основе иерархии процедур или классов объектов создается сложное программное обеспечение; из иерархии боевых подразделений складываются армии; иерархии отделов, филиалов и дочерних организаций лежат в основе правительств и корпораций. Основное преимущество иерархической структуры состоит в том, что на каждом уровне иерархии вычислительная задача, военная операция или административная функция сводится к небольшому количеству действий, выполняемых на более низком уровне, поэтому вычислительная стоимость поиска правильного способа упорядочения этих действий для решения текущей задачи очень невелика. С другой стороны, в неиерархических методах задача сводится к очень большому количеству отдельных действий; при решении

крупномасштабных задач такой подход становится полностью неприменимым. Но в наилучшем случае (в котором высокоуровневые решения всегда сводятся к решениям, имеющим удовлетворительные низкоуровневые реализации) иерархические методы могут привести к созданию алгоритмов планирования с линейными, а не экспоненциальными затратами времени.

В настоящем разделе рассматривается метод планирования, основанный на $\wedge$ иерархических сетях задач, или сетях HTN (Hierarchical Task Network). В приня-

том нами подходе объединены идеи из области планирования с частичным упорядочением (раздел 11.3) и из области, известной под названием "планирование в иерархических сетях задач", или планирование HTN. В планировании HTN первоначальный план, который описывает задачу, рассматривается как описание на очень высоком уровне того, что должно быть сделано, например строительство дома. Планы уточняются путем применения $\wedge$ декомпозиций действий. В каждой декомпозиции действия одно действие высокого уровня сводится к частично упорядоченному множеству действий низкого уровня. Поэтому в декомпозициях действий воплощены знания о том, как осуществляются действия. Например, строительство дома может быть сведено к получению разрешения, найму подрядчика, осуществлению строительства и оплате работы подрядчика (такая декомпозиция показана на рис. 12.3). Этот процесс продолжается до тех пор, пока в плане не остаются только $\wedge$ примитивные действия. Как правило, примитивными считаются такие действия, которые могут быть выполнены агентом автоматически. Например, для генерального подрядчика такое действие, как "оформление ландшафтной архитектуры", может оказаться примитивным, поскольку оно сводится к привлечению подрядчика по ландшафтной архитектуре, а для подрядчика по ландшафтной архитектуре могут рассматриваться как примитивные действия, подобные "посадке на этом участке рододендронов".

Рис. 12.3. Одна из возможных декомпозиций для действия BuildHouse в "чистом" планировании HTN планы разрабатываются только путем последовательной декомпозиции действий. Поэтому планирование HTN может рассматриваться как процесс конкретизации описания некоторой деятельности, а не как процесс создания описания деятельности, начиная с пустого действия (как в случае планирования в пространстве состояний и планирования с частичным упорядочением). Как оказалось, каждое описание действия Strips может быть преобразовано в декомпозицию действия (см. упр. 12.6), а планирование с частичным упорядочением может рассматриваться как частный случай чистого планирования HTN. Однако для некоторых задач (особенно для тех, в которых применяется так называемая "новаторская" постановка с конъюнктивными целями) подход с использованием чистого планирования HTN становится не совсем естественным. Поэтому авторы предпочитают применять гибридный подход, в

котором декомпозиции действий используются как уточнения плана в планировании с частичным упорядочением, в до-

полнение к стандартным операциям определения открытых условий и разрешения конфликтов путем введения ограничений упорядочения. (Подход, в котором планирование HTN рассматривается как расширение планирования с частичным упорядочением, имеет дополнительное преимущество в том, что могут использоваться те же соглашения по системе именования вместо введения полностью нового набора обозначений.) Начнем с более подробного описания декомпозиций действий. Затем рассмотрим, как можно модифицировать алгоритм планирования с частичным упорядочением для учета декомпозиций, и, наконец, рассмотрим вопросы полноты, сложности и практической применимости алгоритмов.

Представление декомпозиций действий Общие описания методов декомпозиции действий хранятся в $\wedge$ библиотеке планов, из которой они извлекаются и конкретизируются для обеспечения потребностей формирования текущего плана. Каждый метод представляет собой выражение в форме `Decompose {a, d}`. Это выражение указывает, что может быть выполнена декомпозиция действия *a* на план *d*, представленный в виде плана с частичным упорядочением, как описано в разделе 11.3.

Строительство дома — это прекрасный, конкретный пример, поэтому мы будем использовать его для иллюстрации концепций декомпозиции действия. На рис. 12.3 показана одна возможная декомпозиция действия `BuildHouse` на четыре действия низкого уровня, а в листинге 12.3 приведены некоторые из описаний действий для данной проблемной области, а также декомпозиция действия `BuildHouse` в том виде, в каком она могла бы присутствовать в библиотеке планов. В этой библиотеке могут находиться также другие возможные декомпозиции.

Листинг 12.3. Декомпозиции действий для задачи построения дома и подробная декомпозиция для действия `BuildHouse`. В этих декомпозициях приняты упрощенное представление о деньгах, а также оптимистическое представление строителей в отношении перспектив оплаты `Action(BuyLand, Precond: Money, Effect: Land a -лMoney)` `Action(GetLoan, Precond: GoodCredit, Effect: Money a Mortgage)` `Action{BuildHouse, Precond: Land, Effect: House}` `Action(GetPermit, Precond: Land, Effect: Permit)` `Action(HireBuilder, Effect: Contract)` `Action(Construction, Precond: Permit a Contract, Effect: HouseBuilt a -^Permit)` `Action{PayBuilder, Precond: Money a HouseBuilt, Effect: -лMoney a House a -^Contract)` `Decompose(BuildHouse, Plan(Steps: {S1: GetPermit, S2: HireBuilder, S3: Construction, S4: PayBuilder}) Orderings: {Start -< S1 -< S3 -< S4 -< Finish, Start -< S2 -< S3}, Links: { Start-^S1, Start^^S4, <-, Permit,-, c. Contract r .-, HouseBuilt.-, 01 ^03, 62 => 03, b3 => ba , Sj^Finish, S.^^Finish) )`

В действии Start этой декомпозиции должны быть предусмотрены все те предусловия действий в плане, которые не предусмотрены в других действиях. Такие предусловия принято называть $\wedge$ внешними предусловиями. В данном примере внешними предусловиями декомпозиции являются Land (Земельный участок) и Money (Деньги).

Аналогичным образом, все $\wedge$ внешние результаты, являющиеся предусловиями действия Finish, представляют собой такие результаты действий в плане, которые не отрицаются другими действиями. В рассматриваемом примере внешними результатами действия BuildHouse являются House (Дом) и -lMoney (Отсутствие денег). В некоторых планировщиках HTN проводится также различие между $\wedge$ первичными результатами, такими как House, и $\wedge$ вторичными результатами, такими как -iMoney. Для достижения целей могут использоваться только первичные результаты, тогда как оба рода результатов могут вызывать конфликты с другими действиями; они позволяют в значительной степени сократить пространство поиска².

Декомпозиция должна представлять собой правильный способ осуществления действия. План d является правильным способом осуществления действия, если d — полный и согласованный план с частичным упорядочением для задачи достижения результатов действия a при наличии предусловий действия a. Очевидно, что декомпозиция будет правильной, если она стала результатом применения непротиворечивого планировщика с частичным упорядочением.

Библиотека планов может содержать несколько декомпозиций для любого конкретного действия высокого уровня; например, может существовать еще одна декомпозиция для действия BuildHouse, которая описывает такой процесс, что агент возводит дом-самостройку из камней и торфа голыми руками. Каждая декомпозиция должна представлять собой правильный план, но может иметь дополнительные предусловия и результаты, кроме тех, что указаны в описании действия высокого уровня. Например, декомпозиция для действия BuildHouse, показанная на рис.

12.3, кроме земельного участка (Land), требует денег (Money), и имеет такой результат, как -lMoney. С другой стороны, вариант с самостройкой не требует денег, но требует большого запаса камней (Rocks) и торфа (Turf), а также может привести к результату BadBack (Болезная спина).

Учитывая то, что действия высокого уровня, такие как BuildHouse, могут иметь несколько возможных декомпозиций, нельзя избежать такой ситуации, что их описания действий Strips будут скрывать некоторые из предусловий и результатов декомпозиций этих действий. Предусловия действия высокого уровня должны представлять собой пересечение внешних предусловий возможных декомпозиций

этого действия, а результаты — пересечение внешних результатов его декомпозиций.

Иначе говоря, необходимо обеспечить, чтобы предусловия и результаты высокого уровня представляли собой подмножества истинных предусловий и результатов каждой примитивной реализации действия высокого уровня.

Следует также отметить две другие формы сокрытия информации. Во-первых, в описании высокого уровня полностью игнорируются все $\wedge$ внутренние результаты декомпозиций. Например, в декомпозиции действия BuildHouse имеются времен2 Это позволяет также предотвратить обнаружение неочевидных планов. Например, некоторое лицо, которому грозит процедура банкротства, может избавиться от всех ликвидных активов (т.е. достичь цели $\neg \text{Money}$), купив или построив дом. Этот план полезен, поскольку современные законы запрещают отчуждать в пользу кредиторов основное жилье.

нб/е внутренние результаты Permit (Наличие разрешения) и Contract (Заключение контракта). Во-вторых, в описании высокого уровня не указаны интервалы "внутри" действия, в течение которых должны иметь место предусловия и результаты высокого уровня. Например, предусловие Land должно оставаться истинным (в нашей очень упрощенной модели) только до выполнения действия GetPermit (Получение разрешения), а результат House становится истинным только после выполнения действия PayBuilder (Оплата работы подрядчика).

Сокрытие информации такого рода становится очень важным, если иерархическое планирование применяется для уменьшения сложности; мы должны иметь возможность рассуждать о действиях высокого уровня, не заботясь о бесчисленных подробностях реализации. Однако за такую возможность приходится платить.

Например, могут существовать конфликты между внутренними условиями одного действия высокого уровня и внутренними действиями другого, но способа обнаружить эту ситуацию с помощью высокоуровневых описаний не существует. Такая проблема оказала значительное влияние на алгоритмы планирования HTN. По сути, примитивные действия могут рассматриваться в алгоритме планирования как точечные события, а действия высокого уровня имеют временную протяженность, в пределах которой могут происходить другие всевозможные события.

Модификация планировщика для его использования в сочетании с декомпозициями В этом разделе описано, как модифицировать алгоритм POP для его применения в планировании HTN. Для этого мы модифицируем функцию определения преемника POP (с. 530), чтобы иметь возможность применять методы декомпозиции к текущему частичному плану P. Новые планы определения преемника формируются так, что вначале выбирается некоторое непримитивное действие a' в плане P, а затем для любого метода Decompose(a, d) из библиотеки планов, такого, что a и a^*

унифицируются с помощью подстановки Э, действие a' заменяется декомпозицией $d' = \text{Subst}(8\#d)$.

Один из примеров применения такого метода показан на рис. 12.4. В верхней части приведен план p возведения дома. Для декомпозиции выбирается действие высокого уровня $a' = \text{BuildHouse}$. В качестве декомпозиции d берется план, приведенный на рис. 12.3, и действие BuildHouse заменяется этой декомпозицией.

Затем вводится дополнительный этап Get Loan (Получение ссуды) для разрешения нового открытого условия Money , которое создается на данном этапе декомпозиции.

Замена действия его декомпозицией немного напоминает пересадку органов в хирургии: мы должны вынуть новый субплан из его упаковки (этапов Start и Finish), вставить его в нужное место и правильно связать сосуды, ткани и нервы.

Для решения такой задачи может применяться несколько методов. Точнее, для каждой возможной декомпозиции d' должны быть выполнены описанные ниже этапы.

3 Действие Construction отрицает результат Permi t , поскольку в противном случае одно и то же разрешение можно было бы использовать для строительства нескольких домов. Но, к сожалению, действие Construction не завершает действие Contract , поскольку вначале необходимо расстаться с деньгами и совершить действие PayBuilder .

$\text{Start Money BuyLand Land Build House House Finish GoodCredit}$ Рис. 12.4. Декомпозиция действия высокого уровня в существующем плане. Действие BuildHouse заменяется его декомпозицией, взятой из плана, показанного на рис. 12.3. Внешнее предусловие Land выполняется с помощью существующей причинной связи от действия BuyLand (Покупка земельного участка). После выполнения этого этапа декомпозиции внешнее предусловие Money остается открытым, поэтому вводится новое действие, GetLoan (Получение ссуды) 1. Вначале действие a' удаляется из плана P . Затем для каждого этапа s в декомпозиции $d1$ необходимо выбрать действие для выполнения соответствующей роли в s и добавить его к плану. Это может быть либо новый экземпляр этапа s , либо существующий этап s' из плана P , который унифицируется с этапом s . Например, декомпозиция действия MakeWine (Выращивание винограда) может потребовать, чтобы мы купили земельный участок, BuyLand ; но, по-видимому, достаточно будет использовать то же действие BuyLand , которое уже было предусмотрено в плане строительства дома. Такая ситуация будет называться $\wedge$ совместным использованием подзадач.

На рис. 12.4 возможности совместного использования подзадач отсутствуют, поэтому создаются новые экземпляры действий. После выбора действий в них

копируются все внутренние ограничения из декомпозиции d' , например, такие, что действие `Get Permit` должно быть выполнено перед действием `Construction` и что есть причинная связь между этими двумя этапами, обуславливающая выполнение предусловия `Permit` действия `Construction`. На этом задача замены a' конкретизацией декомпозиции d на основе подстановки 9 завершается.

2. На следующем этапе необходимо связать ограничения упорядочения для a' в первоначальном плане с этапами декомпозиции d' . Вначале рассмотрим любое ограничение упорядочения в плане P , которое задано в форме $v -< a'$.

Как должно быть упорядочено действие v по отношению к этапам декомпозиции d' ? Наиболее очевидное решение состоит в том, что действие v должно быть завершено перед выполнением любого этапа декомпозиции d' , а этого можно добиться, заменяя каждое ограничение в форме $\text{Start} -< s$ в декомпозиции d' ограничением $v -< s$. С другой стороны, этот подход может оказаться слишком жестким! Например, действие `BuyLand` должно быть выполнено перед действием `BuildHouse`, но нет безусловной необходимости,

чтобы в расширенном плане покупка земли `BuyLand` осуществлялась перед наймом подрядчика `HireBuilder`. Наложив слишком строгое упорядочение, мы можем исключить возможность обнаружения некоторых решений. Поэтому лучший подход состоит в том, чтобы в каждом ограничении упорядочения записывались причины введения этого ограничения; в таком случае при развертывании действия высокого уровня можно будет применять настолько более ослабленные новые ограничения упорядочения, насколько это возможно, в полном соответствии с причиной введения первоначального ограничения. Точно такие же соображения могут применяться при замене ограничений в форме $au < C$.

3. Конечный этап состоит в увязке причинных связей. Если одной из причинных связей в первоначальном плане была $B \ P > a^*$, она заменяется множеством причинных связей от B ко всем этапам декомпозиции d' с предусловиями p , которые выполнены на этапе `Start` декомпозиции d (т.е. ко всем этапам декомпозиции d' , для которых p является внешним предусловием).

В данном примере причинная связь `BuyLandhand>BuildHouse` заменяется связью `BuyLandhand>Permit`. (Предусловие `Money` для действия `PayBuilder` в этой декомпозиции становится открытым условием, поскольку ни в одном из действий в первоначальном плане не выполнено действие по получению денег `Money` для строительства дома `BuildHouse`.) Аналогичным образом, для каждой причинной связи $a' \text{---} C$ в плане необходимо предусмотреть ее замену множеством причинных связей к C от каждого этапа декомпозиции d' , в котором выполняется предусловие p для этапа `Finish` в декомпозиции d (т.е. от этапа v в декомпозиции d' , для которого

предусловие p является внешним результатом). В данном примере связь $\text{BuildHouscHous} \leftrightarrow \text{Finish}$ заменяется связью $\text{PayBuidej} \wedge \wedge \text{Finish}$.

На этом завершаются 4 дополнения, требуемые для формирования декомпозиций в контексте применения планировщика POP.

Необходимость в дополнительных модификациях алгоритма POP связана с тем, что действия высокого уровня скрывают информацию об их конечных примитивных реализациях. В частности, в первоначальном алгоритме POP осуществляется возврат с индикатором неудачи, если текущий план включает неразрешимый конфликт, т.е. если одно из действий конфликтует с причинной связью, но не может быть переупорядочено так, чтобы оно находилось до или после этой связи (подобный пример приведен на рис. 11.4). При использовании действий высокого уровня, с другой стороны, с виду неразрешимые конфликты иногда могут быть разрешены путем декомпозиции конфликтующих действий и чередования их этапов. Соответствующий пример приведен на рис. 12.5. Таким образом, может возникнуть такая ситуация, что путем декомпозиции может быть получен полный и согласованный план из примитивных действий, даже если не существует полного и согласованного плана из действий высокого уровня. Такая возможность означает, что полный планировщик HTN не должен использовать многие варианты отсечения, которые предусмотрены 4. Должны быть также предусмотрены некоторые другие небольшие модификации, которые требуются для разрешения конфликтов в действиях высокого уровня; заинтересованный читатель может ознакомиться со статьями, указанными в конце данной главы.

для стандартного планировщика POP. Еще один способ организации работы может состоять в том, чтобы отсечение применялось в любом случае, в надежде на то, что ни одно решение не будет упущено.

Watch | Hair Start Watch Happy(He) Hair Happy (She) \ Finish Start Give Comb Watch | Hair Give Chain -i Watch Comb Happy(She) Happy(He) Chain -i Hair а) Первоначальная проблема б) Несогласованный абстрактный план Watch Hair Hair Watch GiveChain (OnCredit) Hair Owe(Hair) в) Декомпозиция плана б) для получения согласованного решения Рис. 12.5. Задача "Дары волхвов", взятая из одноименного рассказа О'Генри, может служить примером несогласованного абстрактного плана, который, тем не менее, может быть декомпонован в согласованное решение. На рис. 12.5, а приведена постановка задачи: у пары бедняков есть только две ценные вещи — у него золотые часы, а у нее прекрасные длинные волосы. Каждый из супругов собирается купить подарок, чтобы сделать другому приятное. Он собирается продать свои часы, чтобы купить серебряный гребень для ее волос, а она собирается продать свои волосы, чтобы купить золотую цепочку для его часов.

Частичный план, показанный на рис. 12.5, б, является несогласованным, поскольку не существует способа упорядочить абстрактные этапы GiveComb (Подарить гребень) и GiveChain (Подарить цепочку) без конфликта. (Мы предполагаем, что действие GiveComb имеет предусловие Hair (Волосы), поскольку если бы у жены к тому времени уже не было ее длинных волос, то действие не имело намеченного результата — сделать ей приятное, и аналогично для действия GiveChain.) На рис. 12.5, в показана декомпозиция этапа GiveComb с помощью метода Give. . . (OnCredit) (Получение. . . под залог).

На первом этапе декомпозиции муж получает в свою собственность гребень и вручает его своей жене, дав обязательство передать часы в качестве оплаты в последующую дату. На втором этапе он вручает свои часы и выполняет данное обязательство. Декомпозиция этапа GiveChain осуществляется с помощью аналогичного метода. При условии, что оба этапа дарения упорядочиваются перед этапами доставки залога, такая декомпозиция становится решением задачи. (Обратите внимание на то, что теперь задача определена так, что супруги получают удовольствие от использования цепочки вместе с часами и гребня вместе с волосами даже после того, как потеряют право владения на часы и волосы.) Обсуждение вопроса Вначале необходимо отметить определенные сложности: чистая задача планирования HTN неразрешима (если единственным допустимым способом уточнения

плана является декомпозиция), даже если соответствующее пространство состояний конечно! На первый взгляд такая ситуация может показаться весьма бесперспективной, поскольку основной смысл планирования HTN состоит в обеспечении высокой эффективности составления планов. Указанные сложности возникают потому, что декомпозиции действий могут оказаться рекурсивными (например, если выход на прогулку рассматривается как выполнение одного шага с последующим выходом на прогулку), поэтому планы HTN могут приобретать произвольную длину. В частности, даже кратчайшее решение HTN может оказаться неопределенно длинным, так что становится невозможным нахождение способа завершения поиска за какое-то постоянное время. Тем не менее существуют по меньшей мере три описанных ниже способа исправления указанного положения.

1. Исключить рекурсию, поскольку она действительно требуется лишь в очень немногих проблемных областях. В таком случае все планы HTN приобретают конечную длину и могут быть успешно исследованы.
2. Ограничить длину решений, которые нас интересуют. Поскольку пространство состояний является конечным, план, включающий больше этапов, чем имеется состояний в пространстве состояний, обязательно должен включать цикл, в котором неоднократно посещается одно и то же состояние. Мы ничего не потеряем, исключив решения HTN такого рода, поэтому следует контролировать длину поиска.

3. Принять гибридный подход, в котором сочетается планирование POP и HTN.

Для определения того, существует ли план, достаточно применить планирование с частичным упорядочением, отдельно взятое, поэтому, безусловно, задача планирования с помощью гибридного подхода является разрешимой.

При использовании третьего метода необходимо соблюдать определенную осторожность. В планировании POP примитивные действия могут соединяться в цепочки произвольными способами, поэтому иногда приходится сталкиваться с такими решениями, которые очень трудно понять и которые не имеют такой аккуратной иерархической организации, как планы HTN. Приемлемым компромиссом является управление гибридным поиском таким образом, чтобы операции декомпозиции действий стали предпочтительными по сравнению с операциями добавления новых действий, хотя и не до такой степени, чтобы вырабатывались планы HTN произвольной длины, прежде чем появится возможность добавления каких-либо примитивных действий. Один из способов осуществления такого управления состоит в использовании функции стоимости, которая предоставляет благоприятные условия действиям, введенным путем декомпозиции; чем более благоприятными являются эти условия, тем больше поиск будет напоминать чистое планирование HTN и тем более иерархическим будет решение. Иерархические планы обычно намного проще для выполнения в реальных условиях, поэтому их легче исправить, если что-то при их осуществлении нарушается.

Еще одной важной характерной особенностью планов HTN является возможность совместного использования подзадач. Напомним, что совместным использованием подзадач называется применение одного и того же действия для реализации двух разных этапов в декомпозиции планов. Если совместное использование подзадач запрещено, то каждая конкретизация декомпозиции d' должна быть выполнена

только одним способом, а не многими, что приводит к отсечению значительной части пространства поиска. Обычно такое отсечение обеспечивает некоторую экономию времени и в худшем случае приводит к решению, лишь немного более длинному, чем оптимальное. Но в некоторых случаях возникают более существенные проблемы. Например, рассмотрим цель: "Насладись медовым месяцем и создай большую семью". Библиотека планов может содержать решение "вступи в брак и отправляйся на Гавайи" для первой подцели и "вступи в брак и заведи двух детей" для второй. Без совместного использования подзадач план будет включать два разных действия по вступлению в брак для одного человека, но такой вариант многие считают весьма нежелательным.

Интересным примером анализа затрат и результатов совместного использования подзадач является применение этого метода при оптимизации компиляторов.

Рассмотрим задачу компиляции выражения $\tan(x) - \sin(x)$. В большинстве компиляторов такая задача выполняется путем слияния результатов двух отдельных вызовов процедур тривиальным способом: все этапы процедуры вычисления тангенса $\tan$ выполняются перед каким-либо из этапов вычисления синуса $\sin$. Но рассмотрим следующие аппроксимации для $\sin$ и $\tan$ с помощью рядов Тэйлора: $\sin x \sim x - \frac{x^3}{6} + \frac{x^5}{120} - \frac{x^7}{5040}$; $\tan x \sim x + \frac{x^3}{3} + \frac{2x^5}{15} + \frac{17x^7}{315}$. Планировщик HTN с совместным использованием подзадач может выработать более эффективное решение, поскольку он способен выбрать вариант реализации многочисленных этапов вычисления $\sin$ в сочетании с существующими этапами вычисления $\tan$. В большинстве компиляторов межпроцедурное совместное использование этапов такого рода не осуществляется, поскольку для них потребовалось бы слишком много времени на рассмотрение всех возможных совместно используемых планов. Вместо этого в большинстве компиляторов каждый субплан вырабатывается независимо и только после этого, возможно, происходит модификация результата с помощью локального оптимизатора.

Если учесть все эти дополнительные сложности, вызванные введением декомпозиций действий, можно ли рассчитывать на то, что планирование HTN окажется эффективным? Фактические причины сложностей трудно проанализировать на практике, поэтому рассмотрим идеализированный случай. Предположим, например, что необходимо составить план с p действиями. Для неиерархического планировщика с прямым поиском в пространстве состояний при наличии b допустимых действий в каждом состоянии затраты составят $O(b^p)$. А применительно к планировщику HTN предположим, что структура декомпозиции является регулярной: каждое непримитивное действие имеет d возможных декомпозиций, каждая из которых сводится к k действиям на следующем, более низком уровне. Необходимо определить, сколько разных деревьев декомпозиции существует в этой структуре. Итак, если имеется p действий на примитивном уровне, то количество уровней ниже корня равно $\log_k p$, поэтому количество внутренних узлов декомпозиции определяется выражением $1 + k + k^2 + \dots + k^{(\log_k p)-1} = \frac{k^{\log_k p} - 1}{k - 1}$. Каждый внутренний узел имеет d возможных декомпозиций, поэтому существует $C^d_{\frac{k^{\log_k p} - 1}{k - 1}}$ возможных регулярных деревьев декомпозиции, которые могут быть сформированы. Исследуя эту формулу, можно определить, что уменьшение значения d и увеличение значения k позволяет получить огромную экономию: по сути затраты измеряются корнем k -й степени от

стоимости неиерархического решения, если значения b и d являются сопоставимыми. С другой стороны, составление библиотеки планов, которая включает небольшое количество длинных декомпозиций, но тем не менее позволяет решить любую задачу, не всегда возможно. В другой формулировке эту

мысль можно выразить так, что длинные декомпозиционные макроподстановки, применимые для решения широкого ряда задач, являются чрезвычайно ценными.

Еще одной и, возможно, более важной причиной, позволяющей убедиться в том, что планирование HTN эффективно, является то, что эти методы хорошо зарекомендовали себя на практике. Почти все планировщики для крупномасштабных приложений являются планировщиками HTN, поскольку планирование HTN позволяет людям-экспертам применять свои крайне важные знания о том, как следует выполнять сложные задания, чтобы можно было формировать большие планы с малыми вычислительными издержками. Например, система O-Plan [93], в которой планирование HTN сочетается с составлением расписаний, использовалась для разработки производственных планов в компании Hitachi. При этом типичная задача охватывала производственную линию с 350 различными изделиями, 35 сборочными агрегатами и с более чем 2000 различных операций. Планировщик O-Plan вырабатывает тридцатисуточные расписания с тремя восьмичасовыми сменами в сутки, включающие миллионы операций.

Таким образом, ключом к планированию HTN является составление библиотеки планов, в которой зашифрованы известные методы осуществления сложных, высокоуровневых действий. Одним из способов составления такой библиотеки является изучение требуемых методов на опыте решения задач. Накопив тяжелым трудом опыт планирования в ходе разработки какого-то плана с нуля, агент может сохранить этот план в библиотеке как метод осуществления высокоуровневого действия, определяемого данным конкретным заданием. Таким образом, агент может становиться со временем все более и более компетентным по мере того, как будет находить новые методы на основе старых. Одной из важных характеристик такого процесса обучения является способность обобщать созданные методы, устраняя детали, характерные для данного экземпляра задачи (например, в задаче строительства дома таковыми являются имя подрядчика или адрес земельного участка), и сохраняя в плане только ключевую информацию. Методы осуществления такого рода обобщения описаны в главе 19. Авторы не могут себе даже представить, что человечество смогло бы достичь современного уровня компетентности без какого-то подобного механизма.

12.3. Планирование и осуществление действий в недетерминированных проблемных областях До сих пор в этой главе рассматривались только проблемные области классического планирования, которые являются полностью наблюдаемыми, статическими и детерминированными. Кроме того, предполагалось, что описания действий являются правильными и полными. В таких обстоятельствах любой агент получает возможность вначале составлять план, а затем выполнять этот план буквально "с закрытыми глазами". С другой стороны, в неопределенной среде

агент должен использовать результаты своих восприятий для обнаружения того, что происходит в процессе вы-

полнения плана, а также, возможно, модифицировать или заменять этот план, если случается что-то непредвиденное.

Агентам приходится также иметь дело и с неполной, и с неправильной информацией. Неполнота возникает из-за того, что мир является частично наблюдаемым, или недетерминированным, или обладающим и тем и другим свойством. Например, дверца шкафа с канцелярскими принадлежностями может быть либо закрыта, либо не закрыта на замок; один из моих ключей может открыть или не открыть дверцу, если она на замке, а я могу знать или не знать, в чем имеющаяся у меня информация об этой ситуации является неполной. Поэтому моя модель мира является слабой, но правильной. С другой стороны, неправильность часто возникает, поскольку мир не всегда соответствует моей модели мира; например, я могу быть уверенным в том, что мой ключ откроет шкаф с канцелярскими принадлежностями, но окажусь неправ, если в нем сменили замок. Не обладая способностью справляться с неправильной информацией, агент становится таким же неинтеллектуальным, как и навозный жук (с. 81), который пытается заткнуть свою норку несуществующим шариком навоза даже после того, как шарик вынули из его лапок.

Возможность получения полной или правильной информации зависит от того, какой степенью недетерминированности характеризуется мир. При $\wedge$ ограниченной недетерминированности действия могут иметь непредсказуемые результаты, но все возможные результаты можно перечислить в аксиомах описания действия.

Например, при подбрасывании монеты вполне резонно считать, что результатом будет Heads (Орел) или Tails (Решка). Агент получает способность справляться с ограниченной недетерминированностью, составляя планы, применимые во всех возможных обстоятельствах. При $\wedge$ неограниченной недетерминированности, с другой стороны, множество возможных предусловий или результатов либо неизвестно, либо слишком велико для того, чтобы в нем можно было выполнить полный поиск.

Такие ситуации могут возникать в случае очень сложных или динамичных проблемных областей, таких как вождение автомобиля, экономическое планирование и разработка военной стратегии. Агенту удастся справиться с неограниченной недетерминированностью, только если он способен пересматривать свои планы и/или свою базу знаний. Неограниченная недетерминированность тесно связана с проблемой спецификации (qualification problem), описанной в главе 10, — с проблемой, определяемой невозможностью

перечислить все предусловия, требуемые для того, чтобы какое-то действие в реальном мире имело свой намеченный результат.

Существуют четыре описанных ниже метода планирования для осуществления действий в условиях недетерминированности. Первые два из них применимы для ограниченной недетерминированности, а последние два — для неограниченной недетерминированности.

- $\wedge$ Планирование без использования датчиков. Этот метод, называемый также совместимым планированием, предусматривает создание стандартных, последовательных планов, которые должны выполняться без учета результатов восприятия. Алгоритм планирования без использования датчиков должен обеспечивать, чтобы цель достигалась в плане при всех возможных обстоятельствах, независимо от истинного начального состояния и фактических результатов действий. Планирование без использования датчиков основано на идее принуждения, согласно которой мир может быть принудительно переве-

ден в данное конкретное состояние, даже если агент обладает лишь частичной информацией о текущем состоянии. Принуждение не всегда возможно, поэтому планирование без использования датчиков часто является неприменимым. Методы решения задач без использования датчиков, предусматривающие поиск в пространстве доверительных состояний, были описаны в главе 3.- $\wedge$ Условное планирование. В этом подходе, называемом также планированием с учетом непредвиденных ситуаций, действия в условиях ограниченной недетерминированности осуществляются путем создания условного плана с различными ответвлениями для самых разных непредвиденных ситуаций, какие только могут возникнуть. Так же как и в классическом планировании, агент вначале составляет план, а затем выполняет подготовленный план. Агент определяет, какая часть плана должна быть выполнена, включив в план $\wedge$ действия по восприятию для проверки соответствующих условий.

Например, в проблемной области воздушных перевозок могут быть предусмотрены планы, в которых указано "проверить, работает ли аэропорт SFO (СанФранциско); в случае положительного ответа полететь в этот аэропорт, в противном случае полететь в Окленд". Условное планирование рассматривается в разделе 12.4.- $\wedge$ Контроль выполнения и перепланирование. При этом подходе агент может использовать любой из описанных выше методов планирования (классический, без использования датчиков или условный) для формирования плана, но использует также контроль выполнения для оценки того, предусмотрена ли в плане фактически сложившаяся текущая ситуация, или план должен быть пересмотрен. Перепланирование осуществляется, если что-то происходит не так, как надо. Благодаря этому агент приобретает способность справляться с неограниченной недетерминированностью. Например, даже если агент, способный к перепланированию, не предвидел возможность закрытия аэропорта SFO, то может

обнаружить эту ситуацию после ее возникновения и снова вызвать планировщик для поиска нового пути к цели.

Перепланирующие агенты рассматриваются в разделе 12.5. • ^ Непрерывное планирование. Все планировщики, рассматривавшиеся до сих пор, спроектированы так, что они достигают цели, а затем останавливаются, а непрерывный планировщик предназначен для того, чтобы заниматься планированием в течение всего срока своего существования. Он способен справляться с непредвиденными ситуациями в своей среде, даже если они возникают в ходе того, как агент занимается составлением плана. Он способен также обеспечить отказ от целей и создание дополнительных целей с помощью формулировки цели. Непрерывное планирование описано в разделе 12.6.

Рассмотрим один пример, позволяющий выяснить различия между агентами разных типов. Задача состоит в следующем: дано начальное состояние, в котором имеются стул, стол и несколько банок с краской, притом что все они имеют неизвестный цвет; достичь состояния, в котором стул и стол имеют одинаковый цвет.

Классический планирующий агент не может справиться с этой задачей, поскольку начальное состояние задано не полностью, — мы не знаем, какой цвет имеет мебель.

Агент, осуществляющий планирование без использования датчиков, должен найти план, который работает, не требуя применения каких-либо датчиков во время его выполнения. Решение состоит в том, чтобы открыть любую банку с краской и нанести эту краску и на стул, и на стол, тем самым принудительно переведя их в такое состояние, в котором они имеют одинаковый цвет (даже несмотря на то, что агент так и не узнает, каким стал этот цвет). Принуждение является наиболее приемлемым, если обработка высказываний требует больших затрат или не существует возможности обеспечить восприятие. Например, врачи часто предписывают антибиотик с широким спектром действия, а не используют условный план, согласно которому нужно выполнить анализ крови, затем дождаться получения результатов и только после этого выписать более специализированный антибиотик. Они принимают такое решение потому, что задержки и затраты, связанные с выполнением анализа крови, обычно слишком велики.

Агент, занимающийся условным планированием, может создать лучший план. Он вначале определяет цвет стола и стула; если этот цвет является одинаковым, то план выполнен. В противном случае агент просматривает надписи на банках с краской; если имеется банка с краской такого же цвета, как и у одного из предметов мебели, то агент наносит эту краску и на другой предмет. Если же оба эти предположения оказываются недействительными, агент окрашивает и стол, и стул произвольно выбранной краской.

Перепланирующий агент может выработать тот же план, что и условный планировщик, или вначале предусмотреть меньше ответвлений и по мере необходимости заполнять остальные фрагменты плана в ходе его выполнения. Он также способен справляться с неточностями в описании его действий. Например, предположим, что действие `Paint (obj, color)` рассматривается как имеющее детерминированный результат `Color{obj, color}`. Условный планировщик будет просто считать, что желаемый результат достигнут непосредственно после выполнения действия, а перепланирующий агент способен проверить результат, и если он не был истинным (возможно, потому, что агент допустил небрежность и не покрасил полностью какой-то участок), то имеет возможность затем перепланировать свои действия, чтобы снова покрасить дефектный участок. Мы вернемся к данному примеру на с. 596.

Непрерывно планирующий агент, кроме учета непредвиденных событий, способен пересматривать свои планы должным образом, если, допустим, мы внесем дополнительную цель, чтобы сегодняшний обед прошел еще за этим столом, поэтому нужно будет отложить выполнение плана окрашивания.

В реальном мире в агентах применяется сочетание нескольких подходов.

Производители автомобилей продают запасные колеса и надувные подушки безопасности, которые представляют собой физические воплощения ответвлений условного плана, позволяющих справляться с такими ситуациями, как проколы колес или аварии; с другой стороны, большинство водителей автомобилей никогда не учитывают такие возможности, поэтому отвечают на проколы и аварии как перепланирующие агенты.

Как правило, агенты создают условные планы только для таких непредвиденных ситуаций, которые имеют важные последствия, а шансами на то, что дела пойдут не лучшим образом, нельзя пренебречь. Таким образом, водителю автомобиля, предпринимающему поездку через пустыню Сахара, придется явно учесть возможность поломок в пути, тогда как поездка в соседний супермаркет требует гораздо меньшего заблаговременного планирования.

Агенты, описанные в этой главе, предназначены для осуществления действий в условиях недетерминированности, но не обладают способностью учитывать компромиссы между вероятностью успеха и стоимостью составления плана. В главе 16 рассматриваются дополнительные инструментальные средства, позволяющие справляться с этими проблемами.

12.4. Условное планирование Условное планирование представляет собой один из способов учета неопределенности путем проверки того, что фактически происходит в среде при выполнении заранее заданных пунктов плана. Условное планирование

можно проще всего объяснить на примере полностью наблюдаемых вариантов среды, поэтому начнем его описание с такого случая. Случай с частично наблюдаемой средой является более сложным, но и более интересным.

Условное планирование в полностью наблюдаемых вариантах среды Полная наблюдаемость означает, что агент всегда знает текущее состояние.

Но если среда является недетерминированной, то агент не будет способен предвидеть результат своих действий. Агент, занимающийся условным планированием, преодолевает такую недетерминированность, встраивая в свой план (на этапе планирования) условные этапы, в которых проверяется состояние среды (на этапе выполнения), для определения того, что делать дальше. Поэтому проблема состоит в том, как создавать такие условные планы.

Мы будем использовать в качестве примера проблемной области почтенный мир пылесоса, пространство состояний которого для данного детерминированного случая описано на с. 117. Напомним, что доступными действиями являются Left, Right и Suck. Нам потребуются определенные высказывания для описания этих состояний. Допустим, что предикат AtL (AtR) будет истинным, если агент находится в левом (правом) квадрате, а предикат CleanL (CleanR) истинен, если левый (правый) квадрат чист. Прежде всего необходимо дополнить язык Strips для того, чтобы он позволял учитывать недетерминированность. Для этого предусмотрено, чтобы действия имели $\wedge$ дизъюнктивные результаты, а это означает, что действие может иметь два или несколько различных результатов при каждом его выполнении.

Например, допустим, что действие по перемещению Left (Влево) иногда оканчивается неудачей. В таком случае следующее обычное описание действия:

Action (Left, Precond: AtR, Effect: AtL $\vee$ $\neg$ AtR) должно быть модифицировано так, чтобы в него был включен следующий дизъюнктивный результат:

Action (Left, Precond: At#, Effect: AtL $\vee$ AtR) A2.1) Авторы сочли также целесообразным, чтобы действия имели $\wedge$ условные результаты, когда результат действия зависит от состояния, в котором оно выполняется.

Условные результаты присутствуют в слоте Effect действия и имеют синтаксис 5. Очевидно, что предикат AtR является истинным тогда и только тогда, когда предикат $\neg$ AtL является истинным, и наоборот. Мы используем два высказывания в основном для того, чтобы улучшить удобство чтения.

"when <condition>: <effect>". Например, чтобы промоделировать действие Suck, необходимо записать следующее:

Action(Suck, Precond:, Effect: (when AtL: CleanL) $\wedge$ (when AtR: CleanR)) Условные результаты не вносят недетерминированность, но позволяют ее моделировать. Например, предположим, что в нашем распоряжении имеется не совсем исправный

пылесос, который иногда оставляет мусор в квадрате назначения в процессе своего передвижения, но только если этот квадрат чист. Такую ситуацию можно промоделировать с помощью следующего описания, которое одновременно является и дизъюнктивным, и условнымб:

ActioniLeft,Precond: At#,Effect: AtL v {AtL л when CleanL: -^CleanL)) Для создания условных планов требуются ^ условные этапы. Мы будем записывать их с использованием синтаксиса "if <test> then planA else pialв", где <test> — булева функция переменных состояния. Например, условным этапом для мира пылесоса может быть следующий: "if AtL л CleanL then Right else Suck". Выполнение такого этапа осуществляется очевидным способом. В результате вложения условных этапов линейные планы превращаются в древовидные.

Нам требуются условные планы, которые работают невзирая на то, какие результаты действий фактически будут получены. Такая проблема уже встречалась раньше в данной книге, но немного в другом виде. В играх с двумя игроками (см. главу 6) была поставлена задача найти ходы, которые приводят к выигрышу независимо от того, какие ходы делает противник. По этой причине задачи недетерминированного планирования часто называют ^ играми против природы.

Рассмотрим конкретный пример из мира пылесоса. В начальном состоянии робот-агент находится в правом квадрате мира, в котором нет мусора; поскольку среда полностью наблюдаема, агент знает полное описание состояния, AtR л CleanL л CleanR. В целевом состоянии робот должен находиться в левом квадрате мира, в котором нет мусора. Такая задача была бы совершенно тривиальной, если бы пылесос не подчинялся "двойному закону Мэрфи" и не оставлял иногда мусор при его перемещении в чистый квадрат назначения, а иногда— при выполнении действия Suck в чистом квадрате.

"Дерево игры" для этой среды показано на рис. 12.6. Действия выполняются роботом в узлах "состояния" этого дерева, а природа решает, каким должен быть результат в узлах "жеребьевки", обозначенных кружками. Решением является поддереву, в котором, во-первых, рядом с каждым листовым узлом имеется целевой узел, во-вторых, задается одно действие в каждом из узлов "состояния" и, в третьих, включена каждая ветвь результата в каждом из узлов "жеребьевки". Решение на этом рисунке показано жирными линиями; оно соответствует плану [Left, if AtL л CleanL л CleanR then [] else Suck]. (На данное время, поскольку использ Условный результат when CleanL: -i CleanL может выглядеть немного странным.

Но следует учитывать, что здесь CleanL относится к ситуации до выполнения действия, а -iCleanL — к ситуации после выполнения действия.

зуется планировщик в пространстве состояний, проверки в условных этапах приводят к получению полных описаний состояния.) У\ У\ ЦЕЛЬ 7V 7V ЦИКЛ Рис.

12.6. Первые два уровня дерева поиска для мира пылесоса с "двойным законом Мэрфи". Узлами состояния являются узлы OR, в которых должно быть выбрано некоторое действие. Узлы жеребьевки, показанные в виде кружков, являются узлами AND, для которых требуется учитывать все результаты, как показывает кривая линия, соединяющая исходящие ветви. Решение показано жирными линиями. Для получения точных решений в играх используется алгоритм минимакса (см. листинг 6.1), а для условного планирования обычно применяются две его модификации. Во-первых, узлы MAX и MIN могут быть преобразованы в узлы OR и AND. Интуитивно кажется очевидным, что в плане требуется предпринимать некоторые действия в каждом достигнутом в нем состоянии, но должны учитываться все результаты предпринятого действия. Во-вторых, алгоритм должен возвращать условный план, а не просто отдельный ход. В узле OR план состоит в выборе действия, за которым следует то, что должно произойти. А в узле AND план представляет собой вложенный ряд этапов "if—then—else" с определением субпланов для каждого результата; проверки в этих этапах возвращают полные описания состояния⁷.

Формально говоря, определенное нами пространство поиска представляет собой граф AND-OR. В главе 7 графы AND—OR рассматривались в контексте пропозиционального вывода на основе хорновских выражений, а в этой главе ветвями являются действия, а не этапы логического вывода, но алгоритм остается тем же самым.

В листинге 12.4 приведен рекурсивный алгоритм поиска в глубину, предназначенный для поиска в графе AND—OR.

Одной из ключевых особенностей этого алгоритма является тот способ, с помощью которого он справляется с циклами, часто возникающими в задачах недетерминированного планирования (например, если какое-то действие иногда не имеет ⁷ Такие планы можно также составлять с использованием конструкции case.

результата или если может быть исправлен нежелательный результат). Если текущее состояние идентично одному из состояний в пути от корня, то алгоритм выполняет возврат с индикатором неудачи. Это не означает, что нет решения, достижимого из текущего состояния; просто такая ситуация свидетельствует о том, что если есть нециклическое решение, оно должно быть достижимым из более раннего воплощения текущего состояния, поэтому его новое воплощение может быть отброшено. С помощью этой проверки можно гарантировать, что алгоритм завершит свою работу в любом конечном пространстве состояний, поскольку каждый путь должен достигнуть цели, тупика или повторяющегося состояния. Обратите внимание на то, что в алгоритме не осуществляется проверка, не является ли текущее состояние повторением какого-то состояния в каком-то другом пути от корня; этот вопрос исследуется в упр. 12.15.

Листинг 12.4. Алгоритм поиска в графах AND-OR, сформированных в условиях недетерминированных вариантов среды. Предполагается, что функция `Successors` возвращает список действий, каждое из которых связано с множеством возможных результатов. Задача состоит в том, чтобы найти условный план, который достигает целевого состояния во всех обстоятельствах `function And-Or-Graph-Search(problem) returns` условный план или индикатор неудачи `failure Or-Search(Initial-State[problem], problem, [])` `function Or-Search(state, problem, path) returns` условный план или индикатор неудачи `failure` `if Goal-Test[problem](state) then return` пустой план `if` состояние `state` входит в состав `path` `then return failure` `for each action, state_set in Successors[problem](state) do plan ← And-Search(state_set, problem, [state\path])` `if plan ≠ failure then return [action\plan]` `return failure` `function And-Search(state_set, problem, path) returns` условный план или индикатор неудачи `failure` `for each si in state_set do plani ← Or-Search(si, problem, path)` `if plani = failure then return failure` `return [if S1 then plan1 else if S2 then plan2 else ... if Sn then plann else failure]` Планы, возвращаемые алгоритмом `And-Or-Graph-Search`, содержат условные шаги, в которых проверяется описание всего состояния для выбора следующей ветви. Но во многих случаях можно справиться с этой работой с помощью менее исчерпывающих проверок. Например, план решения на рис. 12.6 может быть записан просто как `[Left, if CleanL then [] else Suck]`. Это связано с тем, что достаточно провести единственную проверку, `CleanL`, для разделения состояния в узле AND на два одноэлементных множества, чтобы после проверки агент мог точно определить, в каком состоянии он находится. В действительности проведение ряда проверок по принципу "if—then—else" отдельных переменных всегда позволяет разделить множество состояний на одноэлементные множества, при условии, что среда является полностью наблюдаемой. Поэтому можно без потери общности ограничиться проведением проверок отдельных переменных.

В недетерминированных проблемных областях часто возникает еще одна, последняя сложность: не всегда удастся выполнить текущую операцию с первого раза, поэтому приходится предпринимать еще одну попытку. Например, рассмотрим пылесос в мире с "тройным законом Мэрфи", который (кроме ранее указанных его недостатков) иногда отказывается переходить туда, куда требует переданная ему команда, например, действие `Left` может иметь дизъюнктивный результат `AtL v AtR`, как в уравнении 12.1. Теперь уже нельзя гарантировать, что план `[Left, if CleanL then [] else Suck]` будет работать. Часть нового графа поиска показана на рис. 12.7 ; очевидно, что больше нет каких-либо нециклических решений и алгоритм `And-Or-Graph-Search` выполняет возврат с индикатором неудачи. Тем не менее существует ^ циклическое решение, которое заключается в том, что нужно продолжать попытки выполнять действие `Left` до тех пор, пока одна из попыток не достигнет успеха. Это решение можно выразить, введя ^ метку для обозначения некоторой части плана и используя эту метку позднее, вместо повторения самого плана. Таким

образом, соответствующее циклическое решение может выглядеть таким образом: [Li: Left, if AtR then L\ else if CleanL then [] else Suck] Рис. 12.7. Первый уровень графа поиска для мира пылесоса, подчиняющегося "тройному закону Мэрфи", где циклы показаны явно. Все решения для данной задачи представляют собой циклические планы (Более удобный синтаксис для циклической части этого плана мог бы предусматривать применение конструкции "while AtR do Left".) Изменения, необходимые для внесения в алгоритм And-Or-Graph-Search, рассматриваются в упр.

12.16. Из этого следует основной вывод, что цикл в пространстве состояний, ведущий в состояние γ , преобразуется в цикл в плане, ведущий в ту точку, где выполнялся субплан для состояния L.

Теперь мы получили возможность синтезировать сложные планы, которые во многом выглядят как программы, с условными выражениями и циклами. К сожалению, эти циклы, в принципе, могут стать бесконечными. Например, в представлении действия для мира с "тройным законом Мэрфи" ничего не сказано, что действие Left в конечном итоге закончится успешно. Поэтому циклические планы не столь желательны, как ациклические, но вполне могут рассматриваться как решения, при условии, что каждый листовой узел представляет собой целевое состояние и любой листовой узел достижим из каждой точки в плане.

Условное планирование в частично наблюдаемых вариантах среды В предыдущем разделе рассматривались полностью наблюдаемые варианты среды, преимущество которых состоит в том, что во время условных проверок можно задавать любые вопросы и быть уверенным в том, что будет получен ответ. Но в реальном мире гораздо чаще встречается частичная наблюдаемость. В начальном состоянии частично наблюдаемой задачи планирования агент обладает лишь некоторым объемом знаний о действительном состоянии. Простейший способ промоделировать такую ситуацию состоит в том, чтобы принять предположение, что начальное состояние принадлежит к множеству состояний; множество состояний представляет собой способ описания начального доверительного состояния агента⁸.

Предположим следующее: агенту в мире пылесоса известно, что он находится в правом квадрате и что этот квадрат чист, но он не может определить с помощью датчиков наличие или отсутствие мусора в других квадратах. В таком случае, насколько известно агенту, он может находиться в одном из двух состояний: левый квадрат может быть либо чистым, либо грязным. Это доверительное состояние обозначено на рис. 12.8 буквой A На этом рисунке показана часть графа AND—OR для мира пылесоса с "альтернативным двойным законом Мэрфи", в котором мусор может иногда оставаться сзади, после того как агент покидает чистый квадрат⁹. Если бы этот мир был полностью наблюдаемым, то агент имел бы возможность

сформировать циклическое решение в такой форме: "Продолжать двигаться влево и вправо, всасывая мусор везде, где он появляется, до тех пор, пока оба квадрата не станут чистыми, а я не буду находиться в левом квадрате" (см. упр. 12.16). К сожалению, при использовании лишь локального датчика грязи этот план является невыполнимым, поскольку невозможно определить истинностное значение проверки "оба квадрата стали чистыми".

Рассмотрим, как формируется граф AND—OR. Из доверительного состояния A мы показываем результат перемещения с помощью действия Left (другие действия не имеют смысла). Поскольку агент может оставить за собой мусор, эти два возможных начальных состояния мира становятся четырьмя возможными состояниями, как показано в прямоугольниках B и C. Эти состояния формируют два различных доверительных состояния, которые классифицируются по доступной информации датчика¹⁰. В доверительном состоянии B агент имеет информацию CleanL, а в доверительном состоянии C— информацию -iCleanL. В результате уборки мусора в состоянии C агент переходит в состояние B. После перехода с помощью действия Right из состояния B агент может оставить или не оставить за собой мусор, поэтому снова возникают четыре возможных состояния мира, которые подразделяются в соответствии со знаниями агента о том, является ли правый квадрат чистым, CleanR (возврат в состояние A), или грязным, ^CleanR (переход в доверительное состояние D).

8 Эти понятия представлены в разделе 3.6, к которому читателю может потребоваться обратиться еще раз, прежде чем изучать дальнейший материал.

9 Этот феномен должен быть знаком родителям с маленькими детьми. Как обычно, приносим свои извинения тем, кто не является таковыми.

10 Обратите внимание на то, что эти состояния не классифицируются по тому признаку, остается ли сзади грязь после передвижения агента. Ветвление в пространстве доверительного состояния вызвано альтернативными результатами изменения знаний, а не альтернативными результатами физических изменений.

Рис. 12.8. Часть графа AND—OR для мира пылесоса с "альтернативным двойным законом Мэрфи", в котором грязь иногда может оставаться сзади, после того как агент покидает чистый квадрат. Агент не может получать с помощью датчиков информацию о наличии грязи в других квадратах. Подводя итог, можно отметить, что недетерминированные частично наблюдаемые варианты среды приводят к получению графа AND—OR доверительных состояний. Поэтому для поиска условных планов может применяться точно такой же алгоритм, как и в случае полностью наблюдаемой среды, а именно And-Or-GraphSearch. Еще один способ понять, что происходит, состоит в следующем: необходимо отметить, что доверительное состояние агента всегда является полностью наблюдаемым, т.е. агент

всегда знает о том, что он знает. Решение "стандартной" задачи в полностью наблюдаемой среде представляет собой частный случай, в котором каждое доверительное состояние является одноэлементным множеством, содержащим одно и только одно физическое состояние.

Можем ли мы на этом завершить описание данной темы? Не совсем! Еще необходимо определить, как должны быть представлены доверительные состояния, как работают средства сбора информации (датчики) и как в этой новой постановке задачи должны записываться описания действий.

Для описания доверительных состояний могут применяться три описанных ниже основных варианта.

1. Множества полных описаний состояния. Например, начальное доверительное состояние на рис. 12.8 может быть описано следующим образом: $\{ (AtR \wedge CleanR \wedge CleanL) , \{AtR \wedge CleanR \wedge \neg CleanL\} \}$ Это представление является простым в использовании, но очень дорогостоящим: если имеется p булевых высказываний, определяющих состояние, то доверительное состояние может содержать $O(p)$ описаний физических состояний, каждый из которых имеет размер $O(p)$. А если агент знает только небольшую часть этих высказываний, то могут возникать экспоненциально большие доверительные состояния, поскольку, чем меньше знает агент, тем больше количество возможных состояний, в которых он может находиться.

2. Логические высказывания, которые точно представляют множество возможных миров в доверительном состоянии. Например, определение начального состояния можно записать следующим образом:

$AtR \wedge CleanR$ Очевидно, что каждое доверительное состояние можно представить с помощью одного и только одного логического высказывания; в случае необходимости можно было бы использовать дизъюнкцию всех конъюнктивных описаний состояния, но рассматриваемый пример показывает, что могут существовать более компактные высказывания.

Один из недостатков общих логических высказываний состоит в том, что может существовать много различных, логически эквивалентных высказываний, которые описывают одно и то же доверительное состояние, поэтому проверка повторяющихся состояний в алгоритме поиска в графе может потребовать применения средств общего доказательства теорем. По этой причине желательно было бы использовать для высказываний каноническое представление, в котором каждое доверительное состояние соответствует одному и только одному высказыванию¹¹. В одном из таких представлений используется конъюнкция литералов, упорядоченных по именам термов; в качестве примера можно привести $AtR \wedge CleanR$. Такое представление можно рассматривать как стандартное

представление состояния с учетом предположения об открытом мире, описанного в главе 11. Не все логические высказывания могут быть записаны в указанной форме (например, не существует способа представления высказывания $AtL \vee CleanR$), но такой метод представления доверительных состояний может применяться во многих проблемных областях.

3. Высказывания с оценкой знаний, описывающие знания агента (см. также раздел 7.7). Для данного начального состояния такое высказывание имеет следующий вид:

$K(AtR) \wedge K(CleanR)$ 11 Наиболее известным каноническим представлением для пропозиционального высказывания общего вида являются бинарные диаграммы решения, или BDD (binary decision diagram) [200].

где k обозначает "знает, что", а $k(P)$ указывает, что агент знает¹² об истинности высказывания P . В сочетании с высказываниями с оценкой знаний используется предположение о замкнутом мире — если высказывания с оценкой знаний нет в списке, оно рассматривается как ложное. Например, в приведенном выше высказывании неявно присутствуют термы $\neg K(CleanL)$ и $\neg K(\neg CleanR)$, поэтому оно представляет тот факт, что агент не знает истинностного значения термина $CleanL$.

Как оказалось, второй и третий из описанных выше вариантов являются приблизительно эквивалентными, но мы будем использовать третий вариант — высказывания с оценкой знаний, поскольку они позволяют получить более реалистичное описание процесса получения информации с помощью датчиков и поскольку нам уже известно, как составлять описания Strips при использовании предположения о замкнутом мире.

В обоих вариантах каждый пропозициональный символ может быть представлен в одном из трех видов: положительный, отрицательный или неизвестный. Поэтому существует точно 3^n возможных доверительных состояний, которые могут быть описаны таким образом. С другой стороны, множество доверительных состояний представляет собой показательное множество (множество всех подмножеств) множества физических состояний. Количество физических состояний равно 2^P , поэтому количество доверительных состояний равно 2^P , что намного превышает 3^n ; это означает, что варианты 2 и 3 являются чрезвычайно ограниченными с точки зрения представления доверительных состояний. В настоящее время такая ситуация считается неизбежной, поскольку & любая схема, способная представить любое возможное доверительное состояние, в наихудшем случае для представления каждого из них потребует $O(\log 2^{2^P}) = O(P)$ битов. Наши простые схемы требуют только $O(n)$ битов для представления каждого доверительного состояния, поэтому позволяют достичь компромисса, в котором выразительность принесена в жертву компактности. В частности, если происходит некоторое действие, одно из

предусловий которого неизвестно, то результирующее доверительное состояние не будет точно представимым и результат действия также станет неизвестным.

Теперь необходимо решить, как должны действовать средства сбора информации с помощью датчиков. В данном случае имеются два варианта. Можно применить $\wedge$ Эк автоматический сбор информации с помощью датчиков, который означает, что на каждом временном этапе агент получает все доступные данные восприятия. В примере, приведенном на рис. 12.8, предполагается, что с помощью датчиков происходит автоматический сбор информации о местонахождении агента и наличии мусора в локальном квадрате. Еще один вариант состоит в том, что можно настаивать на использовании $\wedge$ средств активного сбора информации с помощью датчиков, а это означает, что данные восприятия приобретаются только в результате выполнения конкретных $\wedge$ действий по сбору информации с помощью датчиков, таких как 12 Это — такая же система обозначений, которая использовалась для агентов на основе логических схем, описанных в главе 7. Некоторые авторы применяют ее для передачи такого смысла, как "знает о том, является ли истинным высказывание p ". Задача определения преобразования между этими двумя представлениями является несложной.

CheckDirt и CheckLocation. Мы будем по очереди рассматривать каждый метод сбора информации с помощью датчиков.

Теперь сформулируем одно из описаний действия с использованием высказываний о знаниях. Предположим, что агент в мире "альтернативного двойного закона Мэрфи" перемещается с помощью действия Left, применяя автоматические средства сбора информации о наличии мусора в локальном квадрате; согласно правилам для этого мира, агент может оставить или не оставить за собой мусор, если квадрат был чистым. Этот результат, рассматриваемый как изменение физического состояния, должен быть дизъюнктивным, но если он рассматривается как результат изменения знаний, то просто сводится к тому, что у агента удаляются знания об истинности термина CleanR. Агент должен также знать, является ли истинным терм CleanL, получив эти знания тем или иным способом, поскольку для сбора информации о наличии мусора в локальном квадрате предусмотрены датчики; кроме того, агент знает, что он находится в квадрате AtL. Вся эта информация записывается следующим образом:

Action (Left, Precond: AtR Effect: $K\{AtL\} \vee \neg K\{AtR\}$ л when CleanR: $\neg K\{CleanR\}$ л when CleanL: $K\{CleanL\}$ л when $\neg CleanL$: $K(\neg CleanL)$) A2.2) Обратите внимание на то, что предусловия и условия when представляют собой простые высказывания, а не высказывания с оценкой знаний. Так и должно быть, поскольку результаты действий зависят от фактического состояния мира, но как мы могли бы проверить истинность этих условий, если все, что мы знаем, представляет собой доверительное состояние? Если агент знает истинность высказывания, например, $K\{AtR\}$ в текущем доверительном состоянии, то это высказывание должно быть истинным в текущем

физическом состоянии и, безусловно, соответствующее действие является применимым. Если же агент не знает истинность какого-то высказывания (например, условия CleanL конструкции when), то данное доверительное состояние должно включать миры, в которых терм CleanL является истинным, и миры, в которых CleanL является ложным. Именно поэтому в результате данного действия возникает множество доверительных состояний. Таким образом, если начальным состоянием является $(K(AtR) \wedge K(CleanR))$, то после движения Left двумя результирующими доверительными состояниями становятся $(K(AtL) \wedge K(CleanL))$ и $(K(AtL) \wedge K(\neg CleanL))$. В обоих случаях истинностное значение термина CleanL известно, поэтому в плане может использоваться проверка термина CleanL.

При использовании активных средств сбора информации с помощью датчиков (в отличие от автоматических средств сбора информации с помощью датчиков) агент получает новые данные восприятия только после выдачи запроса на их получение. Таким образом, после перемещения с помощью действия Left агент не знает, является ли левый квадрат грязным, поэтому два последних условных результата больше не появляются в описании действия в уравнении 12.2. Чтобы узнать, есть ли в квадрате мусор, агент может выполнить действие CheckDirt следующим образом:

Action (CheckDirt, Effect: when AtL $\wedge$ CleanL: $K(CleanL) \wedge$ when AtL $\wedge \neg CleanL$: $K(\neg CleanL) \wedge$ when AtR $\wedge$ CleanR: $K(CleanR) \wedge$ when AtR $\wedge \neg CleanR$: $K(\neg CleanR)$) A2.3)

Можно легко показать, что выполнение действия Left, за которым следует действие CheckDirt, при организации работы с помощью активных средств сбора информации приводит к получению тех же двух доверительных состояний, к которым приводило действие Left при использовании организации работы с помощью средств автоматического сбора информации. При активном сборе информации всегда имеет место то, что физические действия отображают доверительное состояние в единственное доверительное состояние-преемник. Многочисленные доверительные состояния могут быть введены только с помощью действий по сбору информации датчиками, которые позволяют получить конкретные знания и поэтому дают возможность использовать в планах условные проверки.

Выше был описан общий подход к условному планированию на основе поиска в пространстве состояний AND-OR. Такой подход зарекомендовал себя как чрезвычайно эффективный применительно к некоторым контрольным задачам, но другие задачи оказались трудноразрешимыми. Теоретически можно доказать, что условное планирование принадлежит к более трудному классу сложности, чем классическое планирование. Напомним, что определение класса задач NP состоит в том, что потенциальное решение может быть проверено для определения того, действительно ли оно является решением, за полиномиальное время. Это определение относится к классическим планам (по крайней мере, к планам, имеющим полиномиальные размеры), поэтому задача классического планирования

относится к числу NP-трудных. Но в условном планировании потенциальное решение должно быть проверено для определения того, что для всех возможных состояний существует некоторый путь через план, позволяющий достичь цели. Проверка такой комбинации "все/некоторые" не может быть выполнена за полиномиальное время, поэтому условное планирование труднее, чем NP. Единственный способ выхода из этой ситуации состоит в том, чтобы игнорировать некоторые из возможных непредвиденных ситуаций, которые могут рассматриваться на этапе планирования, и реагировать на них, только когда они действительно возникают. Именно этот подход рассматривается в следующем разделе.

12.5. Контроль выполнения и перепланирование Агент, контролирующий выполнение, проверяет свои восприятия для определения того, все ли идет в соответствии с планом. Закон Мэрфи говорит нам о том, что даже самые тщательно продуманные планы, которые составляют мыши, люди и агенты, занимающиеся условным планированием, часто оканчиваются неудачей. Проблема заключается в неограниченной недетерминированности — всегда могут возникнуть непредвиденные обстоятельства, для которых описания действий, подготовленные агентом, будут неправильными. Поэтому в реальных вариантах среды всегда требуется контроль выполнения. Мы будем рассматривать два способа организации контроля выполнения: простую, но слабую форму, называемую контролем действий, в которой агент проверяет среду для определения того, что следующее действие окажется применимым, и более сложную, но и более эффективную форму, называемую контролем плана, в которой агент проверяет весь оставшийся план.

Перепланирующий агент знает, что делать, когда происходит что-то непредвиденное: снова вызвать планировщик, чтобы он предложил ему новый план достижения

цели. Для предотвращения использования слишком больших затрат времени на планирование такая операция обычно осуществляется в виде попытки исправить старый план — найти способ перехода из текущего непредвиденного состояния обратно в одно из тех состояний, которые были предусмотрены в плане.

В качестве примера еще раз рассмотрим мир пылесоса с "двойным законом Мэрфи", показанный на рис. 12.6. В этом мире перемещение в чистый квадрат иногда приводит к тому, что в этом квадрате откладывается мусор, но что если агент не будет знать или задумываться об этом? Такой подход позволяет предложить очень простое решение: [Left]. Если мусор не был оставлен после прибытия агента в квадрат во время фактического выполнения плана, то агент обнаружит, что цель достигнута. В противном случае, поскольку предусловие CleanL неявного шага Finish не выполнено, агент выработает новый план: [Suck]. Выполнение этого плана всегда приводит к успеху.

Контроль выполнения и перепланирование, вместе взятые, образуют общую стратегию, которая может применяться и в полностью наблюдаемых, и в частично наблюдаемых вариантах среды, а также может распространяться на самые различные представления, применяемые в планировании, включая планы в пространстве состояний, планы с частичным упорядочением и условные планы. Один из простых подходов к планированию в пространстве состояний показан в листинге 12.5.

Планирующий агент начинает с цели и создает начальный план ее достижения. Затем агент приступает к последовательному выполнению действий. А перепланирующий агент, в отличие от других рассматриваемых нами планирующих агентов, следит за выполнением как оставшегося невыполненного сегмента плана `plan`, так и полного первоначального плана `whole_plan`. В нем используется ^.контроль действий: прежде чем выполнить следующее действие плана `plan`, агент проверяет свои результаты восприятия для определения того, не оказались ли вдруг невыполненными какие-либо предусловия плана. Если они действительно являются таковыми, то агент пытается снова попасть в колею, перепланируя последовательность действий, которые должны помочь ему вернуться в некоторый пункт плана `whole_plan`.

Листинг 12.5. Алгоритм агента, который занимается контролем действий и перепланированием.

В нем в качестве процедуры используется полный алгоритм планирования в пространстве состояний, называемый `Planner`. Если предусловия следующего действия не выполняются, агент проходит по циклу через возможные пункты `p` в плане `whole_plan`, пытаясь найти такой из них, путь к которому может быть запланирован с помощью алгоритма `Planner`. Этот путь называется `repair` (поправка). Если в алгоритме `Planner` удастся найти такую поправку `repair`, агент соединяет `repair` и остаток плана после пункта `p`, чтобы создать новый план. Затем агент возвращается к первому этапу нового плана `function Replanning-Agent(percept) returns действие`
`action static: KB, база знаний (включает описания действий) plan, план,`
первоначально представленный пустым списком `[] whole_plan, общий план,`
первоначально представленный пустым списком `[] goal, цель Tell{KB, Make-Percept-`
`Sentence(percept, t) ) current <— State-Description(KB, t) if plan = [] then`

`whole_plan <rplan <rPlanner (current, goal, KB) if предусловия Preconditions(First{plan}))`
не имеют в настоящее время значения `true` in `KB` then `candidates <— Sort(whole_jplan,`
упорядоченный по расстояниям до текущего состояния `current`) `find состояние s`
среди возможных кандидатов `candidates`, такое, что `failure Ф repair <-- Planner`
`(current, s, KB) continuation <— хвост общего плана whole_jolan, начинающийся с s`
`whole_plan <rplan <— Append {repair, continuation) return Pop(plan)` Схематически
этот процесс приведен на рис. 12.9. Перепланирующий агент обнаруживает, что в

текущем состоянии не выполнены предусловия первого действия в плане `plan`. Поэтому он вызывает алгоритм планирования, чтобы получить новый субплан `repair`, позволяющий перейти из текущего состояния в некоторое состояние `s` плана `whole_plan`. В данном примере оказалось, что состояние `s` находится на один шаг назад от текущего, оставшегося плана `plan` (это удалось определить благодаря тому, что осуществляется контроль за выполнением всего плана, а не только оставшегося). Вообще говоря, следует выбирать состояние `s` так, чтобы оно было как можно ближе к текущему состоянию. Соединение плана `repair` и части плана `whole_plan` от `s` и дальше, которую мы называем `continuation`, позволяет получить новый план `plan`, и агент становится готовым возобновить выполнение плана. `wholeplan plan ©-o—©--<1ь-o--o—©` О `continuation repair` Рис. 12.9. Перед выполнением планировщик предлагает план, называемый здесь `whole_plan`, для перехода из состояния `S` в состояние `G`. Агент выполняет этот план до тех пор, пока не встретится пункт `E`. Прежде чем выполнить оставшийся план `plan`, агент, как обычно, проверяет предусловия и обнаруживает, что он фактически находится в состоянии `O`, а не в состоянии `E`. Затем он вызывает свой алгоритм планирования для получения поправки `repair`, которая представляет собой план перехода из пункта `O` в некоторый пункт `p` первоначального плана `whole_plan`. Теперь новый план `plan` становится конкатенацией планов `repair` и `continuation` (продолжения первоначального плана `whole_plan`).

Теперь вернемся к примеру задачи, в которой нужно было добиться, чтобы цвет стула и стола совпадал, но на этот раз применим перепланирование. Предполагается наличие полностью наблюдаемой среды. В начальном состоянии стул окрашен синей краской, стол — зеленой, и имеются банка с синей краской и банка с красной краской.

Таким образом, может быть сформулировано следующее определение задачи:

```
Init(Color{Chair, Blue) л Color{Table, Green) а ContainsColor{BC, Blue) л PaintCan(BC)) а
ContainsColor(RC, Red) л PaintCan(RC) Goal{Color{Chair, x) л Color{Table, x) )
Action(Paint{object, color), Precond: HavePaint{color) , Effect: Color(object, color))
Action(Open(can), Precond: PaintCan(can) л ContainsColor(can, color), Effect:
HavePaint(color)) Алгоритм Planner агента должен предложить такой план: [Start;
Open(BC); Paint(Table, Blue); Finish] Теперь агент готов выполнить этот план.
Предположим, что все шло хорошо, поскольку агент успешно открыл банку с синей
краской и нанес эту краску на стол.
```

В этот момент агенты, описанные в предыдущих разделах, объявили бы о победе, поскольку они завершили все этапы в плане. Но агент, контролирующий выполнение, должен вначале проверить предусловие этапа `Finish`, в котором сказано, что эти два предмета мебели должны иметь одинаковый цвет.

Предположим, агент обнаружил, что стол и стул не имеют одинакового цвета, поскольку он не закрасил часть стола и осталось зеленое пятно. Теперь агент должен определить, в какую позицию плана `whole_jplan` он должен перейти и какая последовательность корректирующих действий позволяет в нее попасть. Агент обнаруживает, что текущее состояние идентично предусловию перед выполнением действия `Paint`, поэтому агент выбирает пустую последовательность для `repair` и принимает, что его план `plan` должен быть таким же, как и только что выполненная последовательность `[Paint, Finish]`. После принятия этого нового плана контроль выполнения возобновляется и предпринимается попытка выполнить действие `Paint`. Такое поведение повторяется в цикле до тех пор, пока результаты восприятия не будут указывать, что стол полностью перекрашен. Но заслуживает внимания то, что этот цикл создается процессом "запланировать—выполнить—перепланировать", а не явным циклом в плане.

Контроль действий — это очень простой метод контроля выполнения, но он может иногда приводить к поведению, которое нельзя назвать интеллектуальным.

Например, предположим, что агент формирует план решения задачи по перекраске, предусмотрев окрашивание стула и стола в красный цвет. Затем он открывает банку с красной краской и обнаруживает, что краски достаточно только для стула. При контроле действий неудача не была бы обнаружена до тех пор, пока не будет окрашен стул, поскольку в этот момент предусловие `HavePaint (Red)` становится ложным. А фактически требуется обнаруживать неудачное завершение каждый раз, когда состояние является таковым, что оставшийся план больше не будет работать.

В методе $\wedge$ контроля плана эта цель достигается путем проверки предусловий успеха всего оставшегося плана, т.е. предусловий каждого этапа в плане, за исключением тех предусловий, которые достигаются с помощью еще одного этапа в оставшемся плане. В условиях контроля плана выполнение поставленного под сомнение плана прекращается настолько быстро, насколько это возможно, а не продолжается до тех пор, пока фактически не возникнет неудачное завершение¹³. В некоторых случаях такое поведение позволяет спасти агента от несчастья, когда поставленный под сомнение план завел бы его к тупик, из которого цель будет недостижима.

Модифицировать алгоритм планирования таким образом, чтобы в нем в каждом пункте план сверялся с предусловиями успеха оставшегося плана, относительно несложно. Если же мы расширим контроль плана для проверки того, выполняются ли в текущем состоянии предусловия плана в любом будущем пункте, а не только в текущем пункте, то контроль плана позволит также воспользоваться преимуществом удачи, т.е. случайного успеха. Если кто-то вдруг придет на помощь и покрасит стол в красный цвет одновременно с тем, как агент красит в красный цвет

стул, то предусловия завершения плана будут выполнены (поскольку цель достигнута) и агент сможет раньше уйти с работы.

До сих пор методы контроля и перепланирования рассматривались в полностью наблюдаемых вариантах среды. А если среда является частично наблюдаемой, то ситуация может стать гораздо более сложной. Во-первых, обстоятельства могут так измениться в худшую сторону, что агент не сумеет этого обнаружить. Во-вторых, для "проверки предусловий" может потребоваться выполнение действий по получению информации с помощью датчиков, которые должны быть запланированы (либо на этапе планирования, что потребует от нас возврата к условному планированию, либо на этапе выполнения плана). В наихудшем случае для осуществления любого действия по получению информации с помощью датчиков может потребоваться составление сложного плана, который сам требует контроля и поэтому применения дополнительных действий по сбору информации, и т.д. Если агент упорно добивается того, чтобы были проверены все предусловия, то может так и не перейти к фактическому выполнению каких-либо действий. Агент должен предпочесть в такой ситуации проверку лишь переменных, которые имеют важное значение, характеризуются значительной вероятностью того, что их значения окажутся неправильными, и не требуют слишком больших затрат на сбор информации об их значениях. Это позволяет агенту отвечать должным образом на важные опасности, но не тратить время на проверку того, не падает ли небо на голову.

Теперь, после описания одного из методов контроля и перепланирования, мы должны ответить на вопрос: "Будет ли он работать?" Этот вопрос — на удивление сложный. Если под ним подразумевается: "Можем ли мы гарантировать, что агент всегда достигнет цели, даже в условиях неограниченной недетерминированности?", то ответ будет отрицательным, поскольку агент может неожиданно попасть в тупиковую ситуацию, как описано применительно к оперативному поиску в разделе 4.5.

Например, агент-пылесос может не знать, что его аккумуляторы способны разрядиться. Но мы исключим из рассмотрения тупиковые ситуации, т.е. предположим, что агент может сформировать план достижения цели из любого состояния в своей среде. Если принять предположение, что среда действительно является недетерми¹³ Благодаря использованию контроля плана наш агент становится немного умнее, чем навозный жук (см. с. 81). Наш агент заметил бы, что шарик навоза в его лапках отсутствует, и поэтому перепланировал бы свои действия, чтобы получить еще один шарик навоза и заткнуть им свою норку.

нированной, в том смысле, что подобный план всегда имеет какой-то шанс на успех в любой конкретной попытке его выполнения, то агент в конечном итоге достигнет цели. Поэтому пере планирующий агент обладает способностями, аналогичными тем, которыми обладает агент, занимающийся условным планированием. В

действительности можно модифицировать условный планировщик таким образом, чтобы он формировал только частичный план решения, который включает этапы в форме "if <test> then planA else replan". С учетом принятых выше предположений такой план может стать правильным решением первоначальной задачи, к тому же его формирование может оказаться намного дешевле по сравнению с созданием полного условного плана.

Неприятности возникают, когда повторные попытки достижения цели агентом становятся бесплодными — когда они блокируются каким-то предусловием или результатом, о котором агент не знает. Например, если агенту выдали неправильную магнитную карточку-ключ от его номера в гостинице, то он не сможет открыть дверь, даже вставляя эту карточку и вынимая бесконечное количество раз¹⁴. Одно из решений состоит в том, чтобы выбрать случайным образом один из множества возможных планов исправления ситуации, а не пытаться каждый раз выполнить один и тот же план. В данном случае полезной альтернативной был бы план исправления ситуации, состоящий в том, чтобы снова подойти к администратору гостиницы и получить правильную карточку-ключ от комнаты. Учитывая то, что агент может оказаться неспособным отличать друг от друга действительно недетерминированный случай и случай, в котором все попытки будут напрасными, такой способ организации выполнения плана, в котором предусмотрено использование нескольких вариантов исправления ситуации, в целом становится более целесообразным.

Еще одним решением проблемы неправильных описаний действий является обучение. После нескольких попыток обучающийся агент должен быть способен модифицировать описание действия, в котором указано, что данный ключ открывает дверь. С этого момента перепланировщик должен автоматически выработать альтернативный план, который состоит в получении нового ключа. Такого рода обучение рассматривается в главе 21.

Даже несмотря на все эти потенциальные усовершенствования, перепланирующий агент все еще обладает несколькими недостатками. Он не может действовать в тех вариантах среды, где все происходит в реальном времени, и не существует пределов количества времени, которое он может затратить на перепланирование, и поэтому нет пределов времени, в течение которого он решится на выполнение какого-то действия. Кроме того, данный агент не способен формулировать новые цели самостоятельно или принимать к исполнению новые цели, дополнительно к его текущим целям, поэтому такой агент не сможет долго существовать в сложной среде.

Указанные недостатки будут устранены в следующем разделе.

¹⁴ Бессмысленное повторение одного и того же способа исправления плана полностью повторяет поведение, которое демонстрирует оса-сфекс (см. с. 81).

12.6. Непрерывное планирование В этом разделе рассматривается проект агента, способного существовать в некоторой среде неопределенно долго. Поэтому он не является "решателем задач", который получает единственную цель, а затем составляет план и действует до тех пор, пока эта цель не будет достигнута; скорее, данный агент существует, проходя через целые ряды этапов формулировки постоянно меняющихся целей, планирования и осуществления действий. Вместо трактовки процессов планирования и контроля выполнения как отдельных процессов, один из которых передает свои результаты другому, их можно рассматривать как единый процесс, осуществляемый ^ непрерывно планирующим агентом (continuous planning agent).

Такого агента можно рассматривать как находящегося постоянно в ситуации осуществления плана — грандиозного плана прожития его жизни. Его деятельность включает выполнение некоторых этапов плана, готовых к выполнению, уточнение плана для удовлетворения открытых предусловий или разрешения конфликтов, а также модификацию плана в свете дополнительной информации, полученной во время его выполнения. Очевидно, что агент, впервые приступая к формулировке новой цели, не будет иметь в своем распоряжении действий, готовых для выполнения, поэтому он потратит некоторое время на выработку частичного плана. Однако вполне возможно, что агент начнет выполнение плана еще до того, как разработка плана будет полностью закончена, особенно если в нем имеются независимые подцели, которых требуется достичь уже сейчас. Непрерывно планирующий агент постоянно контролирует состояние мира, обновляя свою модель мира по новым результатам восприятия, даже если его размышления над планом еще продолжаются.

Вначале рассмотрим один пример, а затем опишем программу агента, которая получила название Continuous-POP-Agent, поскольку в ней для представления намеченной деятельности используются планы с частичным упорядочением. Чтобы упростить это представление, мы будем предполагать наличие полностью наблюдаемой среды. Те же самые методы могут быть распространены и на частично наблюдаемый случай.

В качестве примера рассматривается задача из проблемной области мира блоков (раздел 11.1). Начальное состояние показано на рис. 12.10, а. Нам потребуется действие $\text{Move}(x, y)$, в котором блок x перемещается на блок y , при условии, что обе их верхние поверхности являются свободными. Схема этого действия состоит в следующем:

Action {Move (x, y) , Precond: $\text{Clear}(x) \wedge \text{Clear}(y) \wedge \text{On}(x,z)$, Effect: $\text{On}(x,y) \wedge \text{Clear}(z) \wedge \neg \text{On}(x, z) \wedge \neg \text{Clear}(y)$ } Агенту вначале необходимо сформулировать для себя цель. Здесь мы не будем обсуждать процесс формулировки цели, а вместо этого предположим, что агенту было дано задание достичь цели $\text{On}(C, D) \wedge \text{On}(D, B)$ (или

он принял это решение самостоятельно). Агент начинает планировать действие по достижению данной цели. В отличие от всех других описанных в этой книге агентов, которые отключают все свои восприятия до тех пор, пока планировщик не вернет полное решение данной задачи, непрерывно планирующий агент формирует свой план поэтапно, затрачивая на каждый этап ограниченное количество времени. После каждого этапа планирования агент возвращает в качестве своего действия пустую операцию NoOp и снова

проверяет результаты своего восприятия. Предположим, что результаты восприятия не меняются и агент быстро составляет план, показанный на рис. 12.11. Обратите внимание на то, что предусловия обоих действий, $\text{Move}(D, B)$ и $\text{Move}(C, D)$, выполнены еще в состоянии Start , но в соответствии с существующим ограничением упорядочения первое из них вносится в план прежде, чем второе. Это требуется для обеспечения того, чтобы предикат $\text{Clear}(D)$ оставался истинным до выполнения действия $\text{Move}(D, B)$. На протяжении всего процесса непрерывного планирования в качестве метки для текущего состояния всегда используется Start . Агент обновляет это состояние после каждого действия. д Гв [e] Гс] ГБ1 [g] Гр] Гв] ш [c] Гп ттг71т на й ш а) б) в) Рис. 12.10. Последовательность состояний, возникающих в ходе того, как непрерывно планирующий агент пытается достичь целевого состояния $\text{On}(C, D)$ л $\text{On}(D, B)$, как показано на рис. 12.10, г: начальное состояние (а); вмешательство другого агента, поставившего блок D на блок B (б); агент выполнял действие $\text{Move}(C, D)$, но допустил ошибку и вместо перемещения блока C на блок D уронил блок C на блок A (в); агент предпринял повторную попытку выполнить действие $\text{Move}(C, D)$ и достиг целевого состояния (г) Start $\text{Ontable}(A)$ Опщё) $\text{Clear}(A)$ $\text{Clear}(C)$ $\text{Clear}(D)$ $\text{Clear}(B)$ $\text{On}(C,D)$ $\text{On}(D,B)$ Finish Рис. 12.11. Начальный план, сформированный непрерывно планирующим агентом. Пока что этот план нельзя отличить от плана, подготовленного обычным планировщиком с частичным упорядочением. Теперь план готов к выполнению, но прежде чем агент сможет приступить к действию, вмешивается природа. Какой-то внешний агент (возможно, учитель агента, потерявший терпение) перемещает блок D на блок B, после чего мир переходит в состояние, показанное на рис. 12.10, в. Агент воспринимает информацию об этом состоянии, учитывает, что предикаты $\text{Clear}(B)$ и $\text{On}(D, G)$ больше не являются истинными в текущем состоянии, и обновляет соответствующим образом модель текущего состояния. Причинные связи, которые создавали предусловия $\text{Clear}(B)$ и $\text{On}(D,G)$ для действия $\text{Move}(D, B)$, становятся недействительными и должны быть удалены из плана. Новый план показан на рис. 12.12. Состояние Start всегда представляет текущее состояние, но оно изменилось, поэтому на данном рисунке состояние Start отличается от соответствующего состояния на предыдущем рисунке. Обратите внимание на то, что теперь план стал неполным: два из предусловий для действия $\text{Move}(D, B)$ являются открытыми, а его предусловие $\text{On}(D,y)$ теперь

неконкретизировано, поскольку больше нет смысла предполагать, что это перемещение будет выполнено из позиции блока G.

Start On(C,D) On(D,B) Finish Clear(C) | Clear(D) « Clear(G) Clear(B) Move(D,B) Рис. 12.12. После того как некто посторонний перемещает блок D на блок B, неподдерживаемые связи, создающие предусловия Clear (B) и On (D,G), уничтожаются, в результате чего создается показанный здесь план. Теперь агент может воспользоваться преимуществом этого "полезного" вмешательства, обнаружив, что причинная связь $\text{Move}\{D, B\} \rightarrow \text{Finish}$ может быть заменена прямой связью от состояния Start к состоянию Finish. Этот процесс называется $\wedge$ продлением (extension) причинной связи и осуществляется каждый раз, когда некоторое условие может быть взято из более раннего, а не более позднего этапа без создания нового конфликта.

После удаления старой причинной связи от $\text{Move}\{D, B\}$ к Finish действие $\text{Move}\{D, B\}$ больше не поддерживает вообще какие-либо причинные связи. Теперь оно становится $\wedge$ избыточным этапом (redundant step). Все избыточные этапы и все поддерживающие их связи удаляются из плана. В результате создается план, показанный на рис. 12.13. $\text{On}(C, \text{Ю}) \wedge \text{Clear}(C) \wedge \text{Ontable}(A) \wedge \text{On}(B, \text{Ю}) \wedge \text{On}(CF) - \text{On}(D, B) \bullet$ Start(D) Clear(G) Clear(C) Clear(D) Рис. 12.13. Связь, поддерживаемая действием $\text{Move}(D, B)$, теперь заменена связью, проходящей от состояния Start, а этап $\text{Move}(D, B)$, ставший избыточным, удален. После этого стал готовым к выполнению этап $\text{Move}\{C, D\}$, поскольку все его предусловия выполнены на этапе Start, перед ним не требуется проходить какие-либо другие этапы и он не конфликтует с какой-либо другой связью в плане.

Данный этап удаляется из плана и выполняется. К сожалению, агент допустил оплошность и уронил блок C на блок A, вместо того чтобы поставить его на D, поэтому было получено состояние, показанное на рис. 12.10, в. Новое состояние плана приведено на рис. 12.14. Обратите внимание на то, что в плане нет новых действий, но все еще остается открытое условие для этапа Finish.

Start On(QD) $\sim \text{On}(ti, B)$ Finish OnTable(A) On(KE) On(CA) On(D,B) —————
- _____ Clear(C) Clear(D) Clear(G) Рис. 12.14. После того как действие $\text{Move}(CD)$ выполнено и удалено из плана, результаты этапа Start отражают тот факт, что в конечном итоге блок C оказался на блоке A, а не на блоке D, для установки на котором он предназначался. Предусловие цели $\text{On}(QfD)$ все еще открыто

Агент решает составить план для этого открытого условия. Опять-таки целевое условие будет выполнено с помощью действия $\text{Move}(C, D)$. Его предусловия удовлетворяются за счет новых причинных связей от этапа Start. Новый план приведен на рис. 12.15.

Start Ontable(A) On(B,E) On(CЯ) On(D,B) Clear(F) Clear(C) Clear(D) Clear(G) On(C,D) On(D,B) Finish Рис. 12.15. Открытое предусловие удовлетворяется за счет повторного введения в план действия Move (C, B) . Обратите внимание на то, что для предусловий применяются новые связывания Действие Move [C, D] снова готово к выполнению. На этот раз оно завершается успешно и достигается целевое состояние, показанное на рис. 12.10, г. После удаления данного этапа из плана целевое условие On {C, D} снова становится открытым. Но поскольку этап Start обновлен и в нем отражено новое состояние мира, целевое условие может быть немедленно выполнено с помощью связи от этапа Start. Таковым является обычный ход событий, когда какое-то действие оказывается успешным. Конечное состояние данного плана показано на рис. 12.16. Поскольку все целевые условия удовлетворяются этапом Start и больше нет оставшихся действий, агент теперь вправе удалить все подцели из состояния Finish и сформулировать новую цель.

Start Ontable(A) ~ TE) ^On(CD) -On(D,B) Finish On(B,L, On(Cf,D).

On(D,B) • Clear(F) Clear(C) Clear(A) Clear(G) Рис. 12.16. После того как действие Move (C, B) выполняется и удаляется из плана, оставшееся открытое условие On (C, D) может быть разрешено путем добавления причинной связи от нового этапа Start. Теперь выполнение плана закончено Из этого примера ясно, что непрерывное планирование весьма напоминает планирование с частичным упорядочением. В каждой итерации алгоритм находит что-то, касающееся плана, что требует исправления (так называемые ^ дефекты плана— plan flaw), и исправляет это положение. Сам алгоритм POP также может рассматриваться как алгоритм устранения дефектов, в котором учитываются два дефекта: открытые предусловия и конфликты причинных связей. Непрерывно планирующий агент, с другой стороны, справляется с гораздо более широким перечнем дефектов, которые описаны ниже.

- Недостающие цели. Агент может принять решение по добавлению в состояние Finish новой цели или целей. (При непрерывном планировании было бы больше смысла применять вместо названия Finish название Infinity, а вместо Start — Current, но авторы будут придерживаться установившейся традиции.)
- Открытые предусловия. Агент добавляет причинную связь к открытому предусловию, выбирая либо новое, либо существующее действие (как в алгоритме POP).

- Конфликты причинных связей. При наличии причинной связи $A \rightarrow B$ и действия C с результатом $\neg B$ агент для разрешения этого конфликта выбирает ограничение упорядочения или ограничение переменной (как в алгоритме POP).
- Неподдерживаемые связи. Если есть причинная связь $Start \rightarrow A$, где предикат p больше не является истинным в состоянии Start, агент удаляет эту связь (что позволяет предотвратить выполнения действия, предусловия которого являются ложными.)
- Избыточные действия. Если какое-то действие A не обеспечивает выполнения ни одной причинной связи, агент удаляет и это действие, и его связи

(что позволяет ему воспользоваться преимуществом удачно сложившихся обстоятельств.) • Невыполненные действия. Если некоторое действие A (отличное от Finish) имеет все выполненные предусловия в состоянии Start, не связано с упорядоченными перед ним другими действиями (кроме Start) и не конфликтует ни с одной причинной связью, агент удаляет из плана действие A вместе с его причинными связями и возвращает его как действие, которое должно быть выполнено. • Ненужные, существовавшие в прошлом цели. Если в плане нет открытых предусловий и нет действий (поэтому все причинные связи проходят непосредственно от состояния Start к состоянию Finish), то текущее множество целей достигнуто. Агент удаляет эти цели и связи, ведущие к ним, чтобы освободить место для новых целей.

Алгоритм Continuous-POP-Agent приведен в листинге 12.6. В нем имеется цикл "принять результаты восприятия, удалить дефекты, выполнить действие". Агент, действующий в соответствии с этим алгоритмом, хранит в своей базе знаний постоянно существующий план и в каждой операции удаляет из этого плана по одному дефекту. Затем он выполняет некоторое действие (хотя таким действием часто бывает пустая операция NoOp) и повторяет цикл. Этот агент способен преодолевать многие проблемы, перечисленные при обсуждении перепланирующего агента на с. 599. В частности, он может действовать в реальном времени, пользуется удачным стечением обстоятельств, способен формулировать цели самостоятельно и справляется с непредвиденными ситуациями, которые влияют на выполнение плана в будущем.

Листинг 12.6. Алгоритм непрерывно планирующего агента с частичным упорядочением Continuous-POP-Agent. После получения очередных результатов восприятия агент удаляет один дефект из своего постоянно обновляемого плана, а затем возвращает действие. Агенту часто приходится выполнять много этапов планирования, связанных с удалением дефектов, в течение которых он возвращает пустую операцию NoOp, пока он не будет готов выполнить какое-то реальное действие

```
function Continuous-POP-Agent(percept) returns действие
  static: plan,
  план, первоначально содержащий только действия Start и Finish
  action <— NoOp
  (действие NoOp применяется по умолчанию)
  Effects[Start] = Update(Effects[Start],
    percept)
  Remove-Flaw(plan) // Возможно, выполнение этой операции приведет //к
  обновлению действия
  action return action
```

12.7. МУЛЬТИАГЕНТНОЕ ПЛАНИРОВАНИЕ До сих пор нам приходилось иметь дело только с одноагентными вариантами среды, в которых рассматриваемый агент действует в одиночестве. А если в этой среде есть также другие агенты, наш агент может просто включить их в свою модель среды, не изменяя своих основных алгоритмов. Но во многих случаях такой подход приводит к низкой производительности, поскольку взаимодействие с другими агентами во многом

отличается от взаимодействия с природой. В частности, природа (как обычно принято считать) безразлична к намерениям агента¹⁵, а другие агенты — нет.

В данном разделе приведены основные сведения о мультиагентном планировании, которое предназначено для решения указанных проблем.

Как было показано в главе 2, мультиагентные варианты среды могут быть кооперативными или конкурентными. Начнем с простого кооперативного примера: планирование действий команды в парном теннисе. Для определения действий обоих игроков в команде могут быть составлены планы; в этом разделе будут описаны методы эффективного формирования таких планов. Эффективно сформированный план полезен, но не гарантирует успеха; прежде всего агенты должны согласиться использовать один и тот же план! Для этого требуется определенная форма координации, которая может быть достигнута с помощью общения.

Кооперация: совместные цели и планы Два агента, играющие в одной команде в парный теннис, имеют единую цель — выиграть матч, что приводит к возникновению различных подцелей. Предположим, что в какой-то момент игры они имеют общую цель — отбить мяч, который направлен на их половину поля, и обеспечить, чтобы по меньшей мере один из них играл под сеткой. Мы можем представить это общее намерение в виде задачи $\wedge$ мультиагентного планирования, как показано в листинге 12.7.

Листинг 12.7. Задача игры в парный теннис. Два агента играют в одной команде и могут присутствовать в одном из четырех мест: [Left,Baseline] (слева, у задней линии), [Right, Baseline] (справа, у задней линии), [Left, Net] (слева, под сеткой) и [Right, Net] (справа, под сеткой). Мяч может быть отбит, если в нужном месте находится один и только один игрок Agents(A, B) Init(At(A, [Left, Baseline]) $\wedge$ At(B, [Right, Net]) $\wedge$ Approaching(Ball, [Right, Baseline])) $\wedge$ Partner{A, B} $\wedge$ Partner(B,A) Goal[Returned(Ball) $\wedge$ At(agent, [x, Net])] Action(Hit(agent, Ball), Precond: Approaching(Ball, [x, y]) $\wedge$ At (agent, [x, y]) $\wedge$ Partner (agent, partner) $\wedge$ $\neg$ At (partner, [x,y]) , Effect: Returned(Ball)) Action(Go(agent, [x, y]) , Precond: At(agent, [a, b]) , Effect: At (agent, [x,y]) $\wedge$ $\neg$ At (agent, [a, b])))

В этой постановке задачи применяются два новых средства. Во-первых, в высказывании Agents {A, B} объявляется, что в плане участвуют два агента, A и B (по условиям данной задачи противостоящие им игроки не рассматриваются как агенты).

Во-вторых, в каждом действии в качестве формального параметра упоминается агент, поскольку нам необходимо следить за тем, что делает каждый агент.

Решением мультиагентной задачи планирования является $\wedge$ совместный план (joint plan), состоящий из действий для каждого агента. Совместный план представляет собой решение, если цель будет достигнута при условии, что каждый агент выполнит назначенные ему действия. Решением данной задачи игры в теннис является приведенный ниже план.

Plan 1: A: [Go(A, [Right, Baseline]), Hit(A, Ball)] B: [NoOp(B), NoOp(B)] Если оба агента имеют одинаковую базу знаний и если это решение является единственным, то все сложится идеально; каждый из агентов сможет определить это решение, а затем совместно выполнить его. Но к большому сожалению для этих агентов (и мы вскоре увидим, почему им приходится об этом сожалеть), существует другой план, позволяющий так же успешно достичь цели, как и первый:

Plan 2: A: [Go(A, [Left, Net]), NoOp(A)] B: [Go(B, [Right, baseline]), Hit(B, Ball)] Если агент A выберет план 2, а агент B — план 1, то никто из них не отобьет мяч.

И наоборот, если агент A выберет план 1, а агент B — план 2, то они, вероятно, столкнутся друг с другом; ни один из них не отобьет мяч, и к тому же пространство под сеткой может остаться неприкрытым. Поэтому само существование правильных совместных планов еще не означает, что цель будет достигнута. Агентам нужен некоторый механизм $\wedge$ координации для достижения одного и того же совместного плана; более того, оба агента должны обладать общими знаниями (см. главу 10) о том, что должен быть выполнен некоторый конкретный совместный план.

Многотельное планирование В данном разделе речь пойдет в основном о формировании правильных совместных планов, а вопросы координации мы пока отложим. Авторы называют данный подход $\wedge$ многотельным планированием (multibody planning); по сути именно в этом состоит задача планирования, с которой сталкивается также один централизованный агент, который может раздавать указания по выполнению действий каждой из нескольких физических сущностей. А если рассматривается случай, в котором действует несколько настоящих агентов, такое планирование дает возможность каждому агенту выяснить, каковы возможные совместные планы, которые позволили бы агентам добиться успеха, если бы они действовали согласованно.

Применяемый нами подход к многотельному планированию будет основан на планировании с частичным упорядочением, которое описано в разделе 11.3. Чтобы упростить решение этой проблемы, мы будем исходить из предположения о полной 15 С нами могут не согласиться жители Соединенного Королевства, где сами действия по подготовке пикника гарантируют дождь.

наблюдаемости среды. Есть еще один дополнительный вопрос, который не возникает в одноагентном случае: среда больше не является в полном смысле этого слова статической, поскольку другие агенты могут действовать, пока какой-то

конкретный агент размышляет. Поэтому необходимо позаботиться о $\wedge$ синхронизации. Для упрощения мы будем предполагать, что каждое действие занимает одно и то же количество времени и что действия, выполняемые в каждом пункте совместного плана, являются одновременными.

В любой момент времени каждый агент выполняет одно и только одно действие (возможно, включая пустую операцию NoOp). Такое множество одновременных действий называется $\wedge$ совместным действием (joint action). Например, совместным действием в проблемной области тенниса (с. 1) с агентами A и B является $\langle \text{NoOp}(A), \text{Hit}(B, \text{Ball}) \rangle$. Совместный план состоит из частично упорядоченного графа совместных действий. Например, план 2 для описанной выше задачи игры в теннис может быть представлен как такая последовательность совместных действий: $\langle \text{Go}\{A, [\text{Left}, \text{Net}]\}, \text{Go}\{B, [\text{Right}, \text{Baseline}]\} \rangle \langle \text{NoOp}(A), \text{Hit}\{B, \text{Ball}\} \rangle$. Это планирование может быть выполнено с помощью обычного алгоритма POP, применяемого к множеству всех возможных совместных действий. Единственная проблема состоит в огромных размерах этого множества: при наличии 10 действий и 5 агентов количество совместных действий составляет 105. Было бы очень утомительным правильное определение предусловий и результатов каждого действия, а разработка плана на таком большом множестве стала бы неэффективной.

Альтернативный подход заключается в том, что совместные действия определяются неявно путем описания того, как каждое отдельное действие сочетается с другими возможными действиями. Такой подход должен быть проще, поскольку большинство действий независимы друг от друга, поэтому нам потребуется перечислить лишь немного действий, которые действительно взаимодействуют друг с другом. Это можно сделать, дополнив обычные описания действий Strips или ADL одним новым средством — Ж списком одновременных действий (concurrent action list). Этот список аналогичен предусловию любого описания действия, за исключением того, что в нем описываются не переменные состояния, а действия, которые должны или не должны выполняться одновременно. Например, действие удара по мячу, Hit, может быть описано следующим образом:

Action{Hit(A, Ball), Concurrent: $\neg \text{Hit}(_, \text{Ball})$, Precond: Approaching(Ball, [x, y]) $\wedge$ At(A, [x, y]) , Effect: Returned(Ball)} В данном случае мы встречаемся с ограничением, запрещающим одновременное выполнение, согласно которому при выполнении действия Hit одним агентом не должно быть выполнения действия Hit другим агентом. А иногда как раз и требуются одновременные действия, например, если портативный холодильник, заполненный напитками, смогут отнести на теннисный корт только два агента. В описании для этого действия должно быть сказано, что агент A не сможет выполнить действие Carry, если нет другого агента, который одновременно выполнит бы действие Carry применительно к тому же холодильнику:

Action{Carry{A, cooler, here, there), Concurrent: Carry{B, cooler, here, there), Precond: At{A, here) л At{cooler, here) л Cooler{cooler), Effect: At {A, there) л At {cooler, there) л - iAfc(A, here) л —iAt {cooler, here)) При использовании такого представления появляется возможность создать планировщик, который весьма напоминает планировщик с частичным упорядочением, действующий по алгоритму POP. Но между этими двумя планировщиками существуют три различия, которые описаны ниже.

1. Кроме отношения временного упорядочения $A \prec$ в разрешается использовать отношения $A = B$ и $A \prec B$, которые означают "одновременно" и "прежде или одновременно", соответственно.
2. Если какое-то новое действие требует одновременных действий, необходимо конкретизировать эти действия, используя новые или существующие действия в плане.
3. Запрещенные одновременные действия являются дополнительным источником ограничений. Каждое ограничение должно быть разрешено путем наложения на конфликтующие действия таких ограничений, чтобы они могли быть выполнены либо прежде, либо позже другого конфликтующего действия.

Такое представление позволяет создать алгоритм планирования для многотельных проблемных областей, эквивалентный алгоритму POP. Мы могли бы дополнить этот подход с помощью уточнений, приведенных в последних двух главах (сетей HTN, частичной наблюдаемости, условных планов, контроля выполнения и перепланирования), но такая задача выходит за рамки настоящей книги.

Механизмы координации Простейший метод, с помощью которого группа агентов может обеспечить координацию при выполнении совместного плана, состоит в принятии определенного $\wedge$ соглашения (convention) до начала совместной деятельности. Соглашением является любое ограничение, касающееся выбора совместных планов, выходящее за рамки того основного ограничения, в соответствии с которым совместный план должен работать, если ему следуют все агенты. Например, соглашение "придерживаться своей стороны поля" станет причиной того, что партнеры в парном теннисе выберут план 2, а соглашение, что "один игрок всегда остается у сетки", приведет их к плану 1. Некоторые соглашения, такие как вождение по правильной стороне дороги, приняты настолько широко, что считаются $\wedge$ общественными законами. Естественные языки также могут рассматриваться как соглашения.

Соглашения, указанные в предыдущем абзаце, являются характерными для конкретной проблемной области и могут быть реализованы путем внесения в описания действий таких ограничений, которые позволили бы исключить нарушения соответствующего соглашения. Более общий подход состоит в

использовании соглашений, независимых от проблемной области. Например, если каждый агент эксплуатирует один и тот же алгоритм многократного планирования с одними и теми же входными данными, то может следовать соглашению о выполнении первого найденного осу-

ществимого совместного плана и быть уверенным в том, что другие агенты сделают такой же выбор. Более надежная, но и более дорогостоящая стратегия могла бы состоять в том, чтобы выработать все совместные планы, а затем выбрать, например, тот из них, внешнее представление для вывода на печать которого находится на первом месте в алфавитном порядке.

Соглашения могут также возникать благодаря эволюционным процессам.

Например, колонии общественных насекомых выполняют очень сложные совместные планы, а осуществление подобных действий к тому же обеспечивается благодаря общим генетическим характеристикам отдельных особей в этой колонии.

Согласованность действий может также быть обусловлена тем фактом, что отход от соглашений приводит к уменьшению эволюционной приспособляемости всех особей, поэтому любой осуществимый совместный план может стать шагом к стабильному равновесию. Аналогичные условия распространяются и на процесс развития любого человеческого языка, когда важным становится не тот факт, на каком языке должен говорить каждый индивидуум, а тот факт, что все индивидуумы должны говорить на одном и тот же языке. Наш последний пример относится к тому, как ведут себя птицы, сбиваясь в стаи во время полета. Вполне приемлемая модель такого процесса может быть получена, если каждый птицеподобный агент (который иногда именуется орнитоидом или [^] бондом от слова boid, или birdoid) выполняет три перечисленных ниже правила, применяя определенный метод их комбинирования.

1. Отделение. Направлять свой полет подальше от соседей, если ты начинаешь к ним слишком заметно приближаться.

2. Соединение в линию. Придерживаться направления полета, позволяющего занять среднюю позицию по отношению к соседям.

3. Выравнивание. Рулить в сторону средней ориентации (направления движения) соседей.

Если все птицы выполняют одни и те же описанные выше правила, то вся стая обнаруживает [^] эмерджентное поведение (от слова emergent— появляющийся), совершая полет в виде одного псевдоустойчивого тела приблизительно с постоянной плотностью, которое не рассеивается со временем. Как и в случае общественных насекомых, для каждого агента не требуется обладание совместным планом, который моделировал бы действия других агентов.

Как правило, принимаются такие соглашения, которые охватывают целый универсум отдельных задач мультиагентного планирования, а не требуют разработки с нуля перед началом решения каждой новой задачи. Это может приводить к недостаточной гибкости и нарушениям в работе, как иногда можно наблюдать в парном теннисе, когда мяч пролетает приблизительно на равных расстояниях между двумя партнерами. В отсутствие применимого соглашения агенты могут использовать общение для получения общих знаний об осуществимом совместном плане. Например, в парном теннисе игрок может крикнуть "Мой!" или "Твой!", имея в виду мяч, чтобы указать на предпочтительный для него совместный план. Механизмы общения рассматриваются более подробно в главе 22, в которой отмечается, что общение не обязательно требует устного обмена репликами. Например, один игрок может неявно сообщить о предпочтительном совместном плане другому, просто выполнив его первую часть. В нашей задаче игры в теннис, если агент А направился к сетке, то

агент А обязан отойти назад, к линии подачи, чтобы отбить мяч, поскольку план 2 является единственным совместным планом, который начинается с того, что агент А направляется к сетке. Такой подход к координированию действия, иногда называемый Ж распознаванием плана (plan recognition), является применимым, если для безошибочного определения нужного совместного плана достаточно одного действия (или краткой последовательности действий).

Вся нагрузка по определению того, как агенты должны приходить к успешному совместному плану, может быть возложена либо на проектировщиков агентов, либо на самих агентов. В первом случае проектировщик агента должен доказать, что правила и стратегии агентов будут успешными, еще до того, как агенты приступят к планированию. Сами агенты смогут правильно и успешно реагировать на обстановку, если заложенные в них правила и стратегии применимы для той среды, в которой они существуют, и им не нужно иметь явные модели, описывающие действия других агентов. А в последнем случае агентам приходится больше рассуждать; они должны доказывать или демонстрировать другими способами, что их планы будут успешными, принимая во внимание рассуждения других агентов. Например, в среде с двумя логическими агентами, А и Б, оба этих агента могут иметь следующее определение: $\langle \text{fp}, s \text{ Feasible} | p, s \rangle \Leftrightarrow \text{CommonKnowledge}\{A, B\}, \text{Achieves}(p, s, \text{Goal})$ В нем указано, что в любой ситуации s план p представляет собой осуществимый совместный план в данной ситуации, если оба агента располагают знаниями о том, что план p позволяет достичь цели. Нам потребуются дополнительные аксиомы для формирования общих знаний о $\wedge$ совместном намерении (Joint intention) выполнить конкретный совместный план; только после этого агенты могут приступать к действиям.

Конкуренция Не все мультиагентные варианты среды включают кооперирующих агентов.

Агенты с конфликтующими функциями полезности находятся в состоянии Ж конкуренции друг с другом. Одним из примеров этого являются игры между двумя игроками с нулевой суммой, такие как шахматы. Как было описано в главе 6, агенту, играющему в шахматы, приходится рассматривать возможные ходы противника на несколько этапов в будущее. Это означает, что любой агент в конкурентной среде должен, во-первых, признавать наличие других агентов, во-вторых, прогнозировать некоторые из возможных планов другого агента, в-третьих, определять, как планы другого агента повлияют на его собственные планы, и, в-четвертых, определять наилучшее действие с учетом указанных влияний. Поэтому в условиях конкуренции, как и в условиях кооперации, требуется модель с описанием планов другого агента.

С другой стороны, в конкурентной среде действия агентов не вносят свой вклад в совместный план.

В разделе 12.4 проводится аналогия между играми и задачами условного планирования. Алгоритм условного планирования, приведенный в листинге 12.4, позволяет составлять планы, действующие при наихудших предположениях о данной среде, поэтому эти планы могут применяться в таких конкурентных ситуациях, когда агента интересует только успех или неудача. А если для агента и его противников важна также стоимость плана, то применимым становится алгоритм минимакса. До сих пор еще очень мало сделано в области совместного использования алгоритма

минимакса с такими методами, как планирование POP и HTN, которые выходят за рамки модели поиска в пространстве состояний, применяемой в главе 6. Мы вернемся к вопросу конкуренции в разделе 17.6, где рассматривается теория игр.

12.8. Резюме В данной главе описаны некоторые сложности, связанные с планированием и осуществлением действий в реальном мире. Основные идеи, изложенные в этой главе, перечислены ниже. • Во многих действиях потребляются ресурсы, такие как деньги, топливо или сырье. Эти ресурсы удобно в целом рассматривать как числовые величины, а не пытаться рассуждать, скажем, о каждой отдельной монете или купюре во всем мире. Действия способны вырабатывать и потреблять ресурсы, поэтому обычно дешевле и эффективнее проверять частичные планы на предмет удовлетворения в них ресурсных ограничений, прежде чем предпринимать попытки их дальнейшего уточнения. • Время — это один из наиболее важных ресурсов. За его расходованием можно следить, применяя специализированные алгоритмы составления расписаний или объединяя составление расписаний с планированием. • Планирование с помощью иерархической сети задач (Hierarchical Task Network — HTN) позволяет агенту

получить совет от проектировщика проблемной области в форме правил декомпозиции. Такой подход обеспечивает создание очень больших планов, требуемых для многих реальных приложений. • В стандартных алгоритмах планирования предполагается наличие полной и правильной информации, а также детерминированной, полностью наблюдаемой среды. Это предположение является недействительным во многих проблемных областях. • С проблемой неполной информации в планировании можно справиться, используя действия по применению датчиков для получения необходимой информации. Условные планы позволяют агенту получать с помощью датчиков информацию о мире во время выполнения плана для определения того, по какой ветви плана он должен следовать дальше. В некоторых случаях может использоваться планирование без использования датчиков или согласованное планирование для формирования плана, который во время своего выполнения не требует применения результатов восприятия. И планы без использования датчиков, и условные планы могут быть сформированы по методу поиска в пространстве доверительных состояний. • Неправильная информация приводит к тому, что предусловия действий и планов остаются невыполненными. Контроль выполнения позволяет обнаруживать нарушения предусловий, создавая предпосылки успешного осуществления плана. • В перепланирующем агенте используется контроль выполнения и по мере необходимости осуществляются восстановительные действия для возврата к первоначальному плану.

- Непрерывно планирующий агент создает новые цели в процессе своей деятельности и реагирует на изменения ситуации в реальном времени. • Мультиагентное планирование необходимо, если в среде имеются другие агенты, с которыми приходится кооперировать, конкурировать или координировать свои действия. В многотельном планировании формируются совместные планы с использованием эффективной декомпозиции описаний совместных действий, но это планирование должно дополняться определенной формой координации, если два кооперативных агента должны согласовать друг с другом, какой из совместных планов следует выполнить.

Библиографические и исторические заметки Планирование с непрерывным учетом времени было впервые реализовано в программе Deviser [1541]. Проблема систематического представления времени в планах была решена Дином и др. [358] в системе Forbin. Программы Nonlin+ [1496] и Sipe [1593], [1594] обладают способностью формировать рассуждения о распределении ограниченных ресурсов по различным этапам плана. Программа O-Plan [93] (планировщик HTN) поддерживает равномерное, общее представление для ограничений, распространяющихся на время и ресурсы. Кроме приложения для Hitachi, упомянутого в этой главе, программа O-Plan применялась для планирования

поставок программного обеспечения в компании Price Waterhouse и для планирования сборки заднего моста автомобиля в компании Jaguar Cars. Кроме того, был разработан целый ряд гибридных систем планирования и составления расписаний: система Isis [489], [490] использовалась при составлении производственных расписаний компании Westinghouse, система Gari [392] осуществляла планирование машинной обработки и конструирования механических деталей, система Forbin применялась для управления фабрикой, а система Nonlin+ служила средством планирования поставок для военно-морского флота.

После первоначального стремительного наплыва теоретических работ в области временного планирования в конце 1980-х годов наступило затишье, и лишь недавно интерес к этой теме возобновился в связи с тем, что появились новые алгоритмы и возросли обрабатывающие мощности, что привело к появлению возможности создавать новые практические приложения. В двух разработанных недавно планировщиках, Sara [401] и T4 [627], используется прямой поиск в пространстве состояний в сочетании со сложными эвристическими функциями для учета действий, характеризующихся различными продолжительностями и требующих разных ресурсов.

Альтернативой этого подхода является применение очень выразительных языков действий, но поиск в подобных системах должен осуществляться под управлением составленных людьми эвристик, характерных для данной проблемной области, как это сделано в системах ASPEN [511], HSTS [744] и IxTeT [549].

Разработки в области составления расписаний для аэрокосмических проектов имеют долгую историю. Программа T-Sched [412] использовалась для составления расписаний, представляющих собой последовательности принципиально важных команд для спутника Uosat-II. Программы Optimum-AIV [1] и Plan-ERSI [509], основанные на программе O-Plan, применялись в Европейском космическом агентст-

ве, соответственно, для сборки космических аппаратов и планирования наблюдений. Программа Spike [741] использовалась для планирования наблюдений с помощью космического телескопа Хаббл в NASA, а система Space Shuttle Ground Processing Scheduling System [355] осуществляла составление производственного расписания с охватом вплоть до 16 000 рабочих смен. Программа Remote Agent [1108] стала первой автономной программой планирования и составления расписаний, применяемой для управления космическим аппаратом, которая работала на борту космического аппарата Deep Space One в 1999 году. В [1527] приведен обзор литературы по применению средств составления производственных расписаний в области исследования операций; теоретические результаты приведены в [993].

Применяемые в программе Strips средства для изучения $\wedge$ макроопераций ("микрооператоров", состоящих из последовательности примитивных этапов) могут рассматриваться как первый механизм иерархического планирования [465].

Иерархия использовалась также в системе Lawaly [1410]. В системе Abstrips [1336] была реализована идея $\wedge$ иерархии абстракции, на основе которой было разрешено игнорировать предусловия действий низкого уровня при планировании на более высоких уровнях, чтобы можно было проще выявить общую структуру рабочего плана. В тезисах докторской диссертации Остина Тэйта [1494] и в работе Эрла Сакердоти [1338] были разработаны основные идеи планирования HTN в его современной форме. Многие практически применяемые планировщики, включая O-Plan и Sipe, представляют собой планировщики HTN. В [1628] обсуждаются свойства действий, благодаря которым планирование HTN становится эффективным. В [443], [444] представлен планировщик с полной иерархической декомпозицией, а также приведен ряд результатов анализа сложности для чистых планировщиков HTN. Другие авторы [25], [72], [766], [1635] предложили гибридный подход, принятый в данной главе, согласно которому декомпозиции просто представляют собой еще одну форму уточнения, которая может использоваться в планировании с частичным упорядочением.

Начиная с исследований по использованию микрооператоров в языке Strips, одной из целей иерархического планирования было повторное использование ранее накопленного опыта планирования в форме обобщенных планов. В некоторых системах, включая Soar [881] и Prodigy [221], в качестве средства обобщения ранее вычисленных планов применялся метод обучения на основе объяснений (explanation-based learning), описанный более подробно в главе 19. Альтернативный подход состоит в том, что ранее вычисленные планы должны сохраняться в их первоначальной форме, а затем использоваться повторно для решения новых подобных проблем по аналогии с первоначальной проблемой. Именно этот подход принят в области, получившей название $\wedge$ планирование на основе прецедентов (case-based planning) [23], [220], [609]. В [765] приведено обоснование того мнения, что планирование на основе прецедентов должно рассматриваться как форма планирования на основе уточнения и что этот метод может служить средством формального описания для планирования с частичным упорядочением на основе прецедентов.

На самых ранних этапах осуществления робототехнических проектов, в которых использовались методы планирования, включая Shakey [465] и Freddy [1045], была обнаружена проблема непредсказуемости и частичной наблюдаемости реальных вариантов среды. Эта проблема стала привлекать больше внимания после публикации статьи Макдермотта Planning and Acting [1021], оказавшей значительное влияние на ход дальнейших исследований.

В ранних планировщиках, в которых отсутствовали условные выражения и циклы, не была явно реализована концепция условного планирования, но несмотря на это, некоторые из таких планировщиков в ответ на неопределенность среды иногда прибегали к стилю, предусматривающему принудительный переход в заданное состояние. В программе Noah, разработанной Сакердоти, такой принудительный переход использовался в предлагаемом программой решении задачи с "ключами и ящиками". Это — известная задача в области планирования, в которой планировщик имеет мало информации о начальном состоянии. Мэйсон [996] доказал, что в робототехническом планировании часто можно и нужно отказываться от сбора информации с помощью датчиков, и описал план без использования датчиков, позволяющий передвинуть инструмент в заданную позицию на столе с помощью последовательности раскачивающих действий, независимой от начальной позиции.

Мы опишем эту идею в контексте робототехники (рис. 25.16).

Голдмен и Бодди [572] ввели термин согласованное планирование (conformant planning) применительно к планировщикам без использования датчиков, которые позволяют справиться с неопределенностью, принудительно переводя мир в известные состояния, и отметили, что планы без использования датчиков часто эффективнее, даже если агент имеет датчики. Первым достаточно эффективным согласованным планировщиком была программа Conformant Graphplan, или CGP, Смита и Уэлда [1437]. Феррарис и Гуинкилья [464], а также Ринтанен [1289] независимо разработали согласованные планировщики на основе алгоритма SATplan. В [148] описан согласованный планировщик, основанный на эвристическом поиске в пространстве доверительных состояний, который представляет собой результат дальнейшего развития идей, впервые реализованных в 1960-х годах в частично наблюдаемых марковских процессах принятия решения, или POMDP (Partially Observable Markov Decision Processes) (см. главу 17). В настоящее время в самых быстрых согласованных планировщиках, осуществляющих поиск в пространстве доверительных состояний, таких как HSCP [118], для представления доверительных состояний используются двоичные диаграммы решений (Binary Decision Diagram — BDD) [200]; такие планировщики характеризуются быстроедействием, примерно на пять порядков превышающим быстроедействие алгоритма CGP.

Планировщик Warplan-C [1555] представляет собой один из вариантов планировщика Warplan, который принадлежит к числу первых планировщиков, основанных на использовании условных действий (conditional action). В [1153] описаны основные проблемы, касающиеся условного планирования.

Подход к организации условного планирования, описанный в данной главе, основан на эффективных алгоритмах поиска для циклических графов AND—OR, разработанных Хименесом и Торрасом [737], а также Хансенom и Зильберштейном

[613]. В [119] описан подход на основе BDD, позволяющий составлять условные планы с циклами. В программе C-Buridan [413] осуществляется условное планирование для действий с вероятностными результатами; это — та же проблема, попытки решения которой предпринимались с помощью процессов POMDP (см. главу 17).

Существует тесная связь между условным планированием и автоматизированным синтезом программ; большое количество ссылок по этой теме приведено в главе 9.

Но работы в этих двух научных областях осуществлялись отдельно из-за невероятной разницы в стоимостях между выполнением машинных команд и осуществлением действий с помощью транспортных средств или манипуляторов роботов. В [934] предпринята явная попытка добиться взаимного обогащения идеями между этими двумя областями.

Теперь в ретроспективе можно непредвзято наблюдать за тем, как появление основных классических алгоритмов планирования привело к созданию расширенных версий этих алгоритмов для проблемных областей, связанных с учетом неопределенности. Разработка методов на основе поиска привела к созданию методов поиска в пространстве доверительных состояний [148]; опыт в области использования алгоритма SATplan привел к созданию стохастического алгоритма SATplan [970] и к разработке средств планирования с применением булевых логических выражений с кванторами [1289]; работы в области планирования с частичным упорядочением привели к созданию программ UWL [446], CNLP [1206] и Cassandra [1241]. Опыт эксплуатации алгоритма Graphplan привел к появлению программы Sensory Graphplan, или SGP [1569], но задача разработки полностью вероятностного алгоритма Graphplan еще не совсем решена.

Одной из самых первых значительных работ по контролю выполнения было создание программы Planex [465], которая применялась в сочетании с планировщиком Strips для управления роботом Shakey. В программе Planex использовались треугольные таблицы (по сути — это эффективный механизм хранения для предусловий плана в каждом пункте плана), что обеспечивает возобновление работы после частичного неудачного завершения при выполнении плана без полного перепланирования. Модель выполнения, применяемая в работе Shakey, рассматривается более подробно в главе 25. В планировщике Nası [1021] проблема планирования рассматривается просто как спецификация выполнения сложного действия, что позволяет полностью унифицировать этапы выполнения и планирования. В этом планировщике для формирования рассуждений о таких сложных действиях используются средства доказательства теорем.

Одним из первых планировщиков, в котором осуществлен систематический подход к решению задач перепланирования, была программа Sipe (System for Interactive

Planning and Execution monitoring) [1593], [1594]. Эта программа использовалась в демонстрационных проектах в нескольких проблемных областях, включая планирование операций на взлетной палубе авианосца и составление производственного расписания для одного из пивзаводов в Австралии. Еще в одном исследовании программа Sipe применялась для планирования строительства многоэтажных зданий, что представляет собой одну из наиболее сложных проблемных областей, в которых когда-либо применялись программы-планировщики.

Система Ipem (Integrated Planning, Execution, and Monitoring) [25] оказалась первой системой, в которой было применено сочетание планирования с частичным упорядочением и выполнения для создания непрерывно планирующего агента.

В алгоритме Continuous-POP-Agent, приведенном в этой книге, объединены идеи, взятые из системы Ipem, планировщика Puccini [570] и системы Cypress [1595].

В середине 1980-х годов некоторые специалисты считали, что планирование с частичным упорядочением и связанные с ним методы никогда не позволят добиться достаточно высокого быстродействия для формирования эффективного поведения агента в реальном мире [7]. Вместо этого были предложены системы $\wedge$ реактивного планирования; в своей основной форме эти системы представляли собой рефлексных агентов, возможно, с учетом внутреннего состояния, которые могут быть реализованы с помощью любого из самых различных представлений для правил "условие—действие". В архитектуре обобщения Брукса [189] (см. главы 7 и 25) использовались многоуровневые конечные автоматы для управления движениями и

обхода препятствий шагающими и колесными роботами. Система Pengi [7] обладала способностью играть в (полностью наблюдаемую) видеоигру с использованием логических схем в сочетании с "визуальным" представлением текущих целей и внутреннего состояния агента.

В качестве метода поиска по таблице для реактивного планирования были разработаны "универсальные планы" Шопперса [1366], но, как оказалось, они представляли собой повторное открытие идеи $\wedge$ правил, которые уже давно использовались в марковских процессах принятия решений. Универсальный план (или правило) содержит отображение из любого состояния в действие, которое должно быть выполнено в этом состоянии. Гинсберг [557] предпринял яростную эмоциональную атаку на идею универсальных планов и, в частности, показал неразрешимость результатов для некоторых формулировок задачи реактивного планирования. Шопперс [1367] ответил на эту статью не менее эмоционально.

Как часто случается, сложившееся противоречие удалось разрешить с помощью гибридного подхода. Использование тщательно разработанных иерархий,

планировщиков HTN, таких как PRS [542] и RAP [471], а также непрерывно планирующих агентов позволяет добиться сокращения времени отклика, свойственного реактивным подходам, и осуществления сложного долгосрочного планирующего поведения во многих проблемных областях.

Популярность мультиагентного планирования особенно возросла в последние годы, но само это направление имеет долгую историю. В [833] предложена формализация мультиагентного планирования в логике первого порядка, а описание в стиле Strips дано в [1195]. Понятие совместного намерения, которое приобретает особую важность, если агенты должны выполнить совместный план, впервые было сформулировано в исследованиях по актам общения [276], [277]. Приведенное в данной главе описание многотельного планирования с частичным упорядочением основано на [157].

В этой главе была лишь слегка затронута тема согласования в мультиагентном планировании. В [424] обсуждается вопрос о том, как может осуществляться совместное использование задач агентами с помощью согласования действий между ними. В [858] описана система для ведения Diplomacy, настольной игры, в которой допускается согласование действий, формирование и разрушение коалиций, а также нарушение взятых на себя обязательств. В [1468] показано, что агенты могут взаимодействовать как члены одной команды в конкурентной, динамической, частично наблюдаемой среде робототехнического футбола. В [1564] приведен обзор мультиагентных систем, который занимает целую книгу.

Модель орнитоида, приведенная на с. 609, разработана Рейнольдсом [1284], который получил премию Оскар Американской академии киноискусства за успешное применение этой модели во время съемок полчищ летучих мышей и стай пингвинов в кинофильме "Batman Returns" (Бэтмен возвращается).

Упражнения 12.1. Тщательно изучите представление времени и ресурсов, приведенное в разделе 12.1. а) Почему целесообразно сделать предикат Duration (d) одним из результатов действия, а не предусмотреть в действии отдельное поле в форме

Duration: d? (Подсказка. Рассмотрите условные результаты и дизъюнктивные результаты.) б) Почему Resource: r задано как отдельное поле в действии, а не оформлено как результат?

12.2. ^ Потребляемым ресурсом называется ресурс, который (частично) расходуется при выполнении действия. Например, для крепления двигателей к автомобилям требуются винты. Винты после их использования становятся недоступными для создания других креплений. а) Объясните, как модифицировать представление, приведенное в листинге 12.2, чтобы в нем первоначально было 100 винтов, для двигателя E₁ требовалось 40 винтов, а для двигателя E₂ — 50. В литералах результата

для ресурсов могут использоваться функциональные символы + и -. б) Объясните, как должно быть модифицировано определение конфликта между причинными связями и действиями в планировании с частичным упорядочением для учета потребляемых ресурсов. в) Некоторые действия (например, пополнение на заводе запасов винтов или повторная заправка топливом автомобиля) способны увеличивать доступность ресурсов. Если ни одно действие не увеличивает доступность ресурса, он становится монотонно невозрастающим. Объясните, как можно использовать это свойство для отсечения ветвей в пространстве поиска.

12.3. Приведите декомпозиции для этапов HireBuilder и GetPermit, показанных на рис. 12.4, и объясните, как декомпонованные субпланы соединяются в общий план.

12.4. Приведите пример двух абстрактных субпланов из проблемной области строительства дома, которые не могут быть объединены в согласованный план без совместно выполняемых этапов. (Подсказка. Участками, в которых обычно должны взаимодействовать два субплана, являются те места, в которых совместно устанавливаются две физические детали дома.) 12.5. Некоторые специалисты утверждают, что одним из преимуществ планирования HTN является то, что оно позволяет решать задачи типа "совершить прямую и обратную поездку из Лос-Анджелеса в Нью-Йорк", которые трудно выразить в системах обозначения, отличных от HTN, поскольку начальное и целевое состояния должны быть одинаковыми ($At(LA)$). Можете ли вы предложить способ отображения и решения таких задач без сетей HTN?

12.6. Покажите, как может быть перезаписано стандартное описание действия Strips в виде декомпозиции HTN с использованием обозначения Achieve (p) для указания на деятельность по достижении условия p .

12.7. Некоторые операции в стандартных языках программирования можно промоделировать как действия, которые меняют состояние мира. Например, в операции присваивания копируется содержимое участка памяти, а в операции печати изменяется состояние выходного потока. Программа, состоящая из этих операций, может также рассматриваться как план, цель которого задана в спецификации программы. Поэтому алгоритмы планирования могут использоваться для создания программ, которые реализуют заданную спецификацию.

а) Запишите операторную схему для оператора присваивания (в котором значение одной переменной присваивается другой). Помните, что первоначальное значение должно быть при этом перезаписано! б) Покажите, как может использоваться в планировщике процедура создания объекта для разработки плана обмена значениями между двумя переменными с применением временной переменной.

12.8. Ознакомьтесь со следующими доводами: "В инфраструктуре, которая допускает наличие неопределенных начальных состояний, дизъюнктивные результаты

являются просто удобным способом обозначения, а не источником дополнительной выразительной мощи. Для любой схемы действий a с дизъюнктивным результатом $P \vee Q$ можно всегда заменить его условным результатом $\text{when } R: P \text{ л } \text{when } \neg R: Q$, который, в свою очередь, можно свести к двум обычным действиям. Высказывание a обозначает случайное высказывание, которое не известно в начальном состоянии и для которого не предусмотрено каких-либо действий по получению информации с помощью датчиков".

Являются ли эти доводы правильными? Рассмотрите отдельно два случая, в одном из которых в плане имеется только один экземпляр схемы действия a , тогда как во втором имеется больше одного экземпляра.

12.9. Почему условное планирование не позволяет справиться с неограниченной недетерминированностью?

12.10. В мире блоков мы были вынуждены ввести два действия Strips (Move и MoveToTable), чтобы иметь возможность сопровождать предикат Clear должным образом. Покажите, как можно использовать условные результаты для представления обоих этих случаев с помощью одного действия.

12.11. Условные результаты были проиллюстрированы на примере действия Suck в мире пылесоса — от того, в каком квадрате находится робот, зависит, какой из квадратов станет чистым. Можете ли вы предложить новое множество пропозициональных переменных для определения состояний мира пылесоса, таких, что действие Suck имеет безусловное описание? С использованием этих пропозициональных переменных составьте описания действий Suck , Left и Right и покажите, что их достаточно для представления всех возможных состояний мира.

12.12. Составьте полное описание действия Suck для пылесоса в мире "двойного закона Мэрфи", который иногда оставляет мусор после его перемещения в чистый квадрат назначения, а иногда оставляет мусор после выполнения действия Suck в чистом квадрате.

12.13. Найдите достаточно грязный ковер, свободный от препятствий, и очистите его пылесосом. Нарисуйте путь, проделанный чистящей головкой пылесоса, настолько точно, насколько сможете. Объясните характер этого пути, ссылаясь на формы планирования, описанные в этой главе.

12.14. Приведенные ниже цитаты взяты из инструкций на задних стенках бутылок с шампунем. Обозначьте каждую из этих инструкций как безусловный план, условный план или план с контролем выполнения: а) "Разотрите пену.

Прополосните волосы. Повторите процедуру"; б) "Нанесите шампунь на волосы

и оставьте на несколько минут. Прополосните волосы и повторите процедуру в случае необходимости"; в) "Если не удастся устранить проблемы, обратитесь к врачу".

12.15. В алгоритме And-Or-Graph-Search, приведенном в листинге 12.4, осуществляется проверка повторяющихся состояний только в пути от корня до текущего состояния. Предположим, что, кроме того, в этом алгоритме было бы предусмотрено хранение в виде списка каждого когда-либо посещенного состояния и проверка по этому списку (см., например, алгоритм GraphSearch, приведенный в листинге 3.6). Определите, какая информация должна записываться в список и как она должна использоваться в алгоритме при обнаружении повторяющегося состояния. {Подсказка. Вам потребуется, по меньшей мере, проводить различие между состояниями, для которых перед этим был успешно сконструирован субплан, и состояниями, для которых не удалось найти субплана.) Объясните, как можно использовать метки для предотвращения необходимости иметь несколько копий субпланов.

12.16. (И) Объясните точно, как следует модифицировать алгоритм And-Or-GraphSearch для выработки циклического плана, если не существует ни одного ациклического плана. Вам придется рассмотреть три проблемы: применение меток для этапов плана так, чтобы циклический план мог указывать обратно на какую-то предыдущую часть плана, модификация алгоритма Or-Search так, чтобы он продолжал поиск ациклических планов после нахождения циклического плана, а также дополнение средств представления плана, чтобы они позволяли указать, является ли план циклическим. Покажите, как действует ваш алгоритм: а) в мире пылесоса с тройным законом Мэрфи и б) в альтернативном мире пылесоса с двойным законом Мэрфи. Вам может потребоваться реализация алгоритма на компьютере для проверки полученных результатов.

Может ли план для случая б) быть записан с использованием стандартного синтаксиса цикла?

12.17. Полностью определите процедуру обновления доверительного состояния для частично наблюдаемых вариантов среды. Под этим подразумевается метод вычисления нового представления доверительного состояния (как списка высказываний с оценкой знаний) из текущего представления доверительного состояния и описания действия с условными результатами.

12.18. Составьте описания действий, аналогичные уравнению 12.2, для действий Right и Suck. Запишите также описание для действия CheckLocation, аналогичное приведенному уравнению 12.3. Повторите это упражнение, используя альтернативное множество высказываний из упр. 12.11.

12.19. Рассмотрите приведенный на с. 599 список действий, которые не могут быть выполнены перепланирующим агентом. Составьте набросок алгоритма, который позволяет выполнить одно или несколько таких действий.

12.20. Рассмотрите следующую задачу: пациент поступил в приемную врача с симптомами, которые могут быть вызваны либо обезвоживанием, либо болезнью D (но не обеими этими причинами вместе). Существуют два возможных действия: Drink (Питьё), которое, безусловно, позволяет справиться с обезвоживанием, и Medicate (Медикаментозное лечение), излечивающее заболевание

Д но имеющее нежелательный побочный эффект, который возникает, если организм пациента обезвожен. Составьте описание этой проблемы на языке PDDL и разработайте план без использования датчиков, позволяющий решить эту задачу, перечислив все относящиеся к ней возможные миры.

12.21. В задаче из области медицины, приведенной в предыдущем упражнении, добавьте действие Test (Лабораторный анализ), которое имеет условный результат CultureGrowth (Развитие бактериальной культуры), если предикат Disease имеет истинное значение, и в любом случае имеет результат из области восприятия Known (CultureGrowth). Составьте условный план, который решает эту задачу и сводит к минимуму использование действия Medicate.